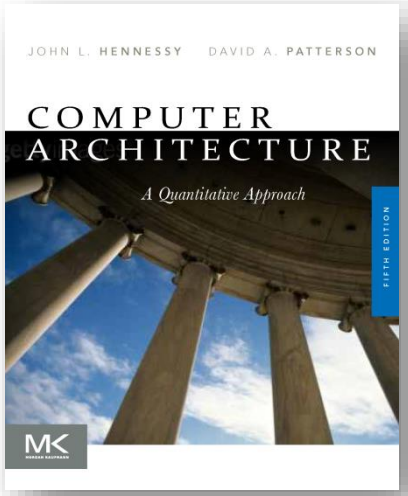


Chapter 1

Fundamentals of Quantitative Design and Analysis

Copyright © 2019, Elsevier Inc. All rights reserved.

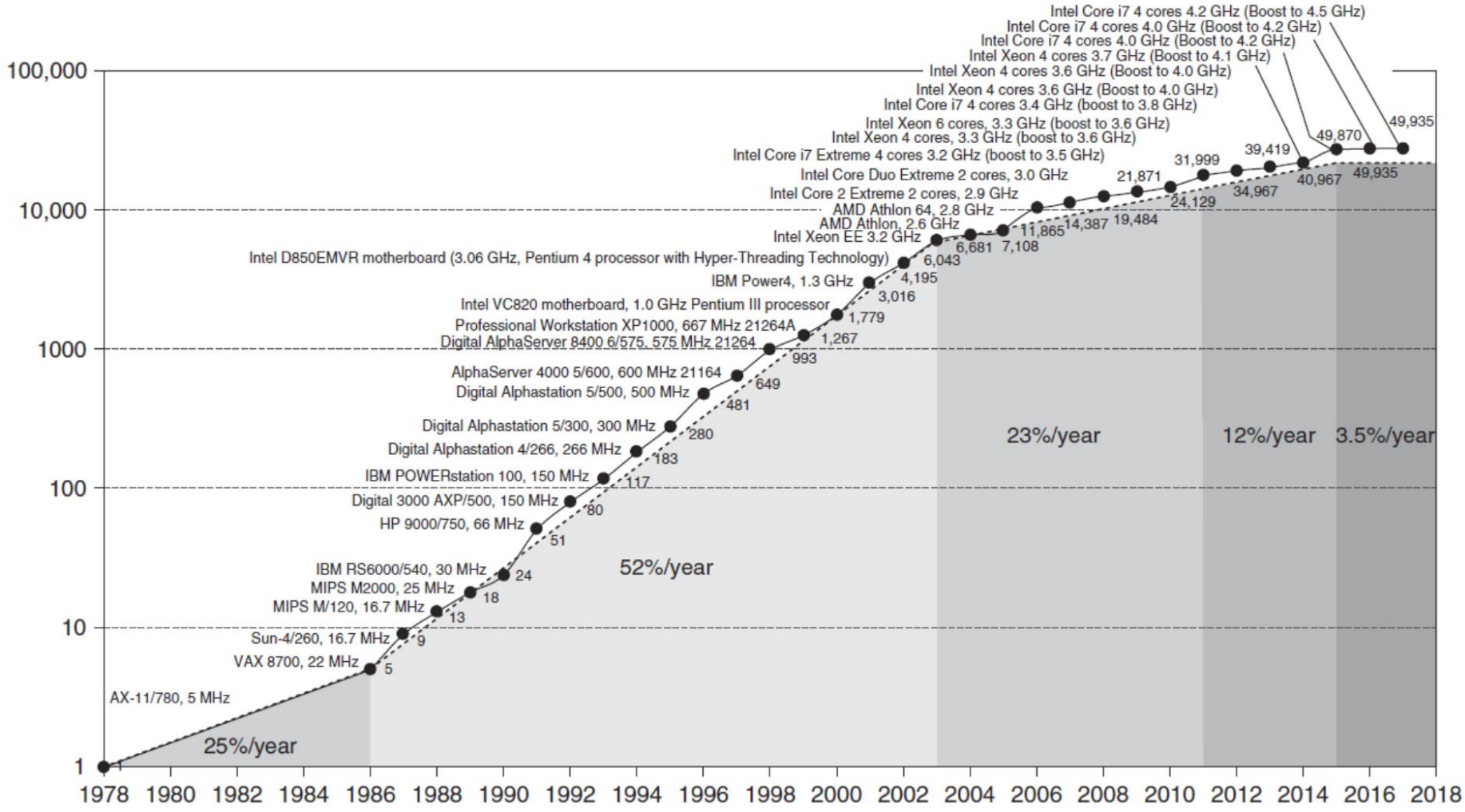
Computer Technology

- Performance improvements:
  - Improvements in semiconductor technology
    - Feature size, clock speed
  - Improvements in computer architectures
    - Enabled by HLL compilers, UNIX
    - Lead to RISC architectures
  - Together have enabled:
    - Lightweight computers
    - Productivity-based managed/interpreted programming languages

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Single Processor Performance



MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Current Trends in Architecture

- Cannot continue to leverage Instruction-Level parallelism (ILP)
  - Single processor performance improvement ended in 2003
- New models for performance:
  - Data-level parallelism (DLP)
  - Thread-level parallelism (TLP)
  - Request-level parallelism (RLP)
- These require explicit restructuring of the application

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Classes of Computers

- Personal Mobile Device (PMD)
  - e.g. start phones, tablet computers
  - Emphasis on energy efficiency and real-time
- Desktop Computing
  - Emphasis on price-performance
- Servers
  - Emphasis on availability, scalability, throughput
- Clusters / Warehouse Scale Computers
  - Used for “Software as a Service (SaaS)”
  - Emphasis on availability and price-performance
  - Sub-class: Supercomputers, emphasis: floating-point performance and fast internal networks
- Internet of Things/Embedded Computers
  - Emphasis: price

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Parallelism

- Classes of parallelism in applications:
  - Data-Level Parallelism (DLP)
  - Task-Level Parallelism (TLP)
- Classes of architectural parallelism:
  - Instruction-Level Parallelism (ILP)
  - Vector architectures/Graphic Processor Units (GPUs)
  - Thread-Level Parallelism
  - Request-Level Parallelism

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Flynn's Taxonomy

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data streams (SIMD)
  - Vector architectures
  - Multimedia extensions
  - Graphics processor units
- Multiple instruction streams, single data stream (MISD)
  - No commercial implementation
- Multiple instruction streams, multiple data streams (MIMD)
  - Tightly-coupled MIMD
  - Loosely-coupled MIMD

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Defining Computer Architecture

- "Old" view of computer architecture:
  - Instruction Set Architecture (ISA) design
  - i.e. decisions regarding:
    - registers, memory addressing, addressing modes, instruction operands, available operations, control flow instructions, instruction encoding
- "Real" computer architecture:
  - Specific requirements of the target machine
  - Design to maximize performance within constraints: cost, power, and availability
  - Includes ISA, microarchitecture, hardware

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Instruction Set Architecture

- Class of ISA
  - General-purpose registers
  - Register-memory vs load-store
- RISC-V registers
  - 32 g.p., 32 f.p.

Register	Name	Use	Saver
x0	zero	constant 0	n/a
x1	ra	return addr	caller
x2	sp	stack ptr	callee
x3	gp	gbl ptr	
x4	tp	thread ptr	
x5-x7	t0-t2	temporaries	caller
x8	s0/fp	saved/ frame ptr	callee
x9	s1	saved	callee
x10-x17	a0-a7	arguments	caller
x18-x27	s2-s11	saved	callee
x28-x31	t3-t6	temporaries	caller
f0-f7	ft0-ft7	FP temps	caller
f8-f9	fs0-fs1	FP saved	callee
f10-f17	fa0-fa7	FP arguments	callee
f18-f27	fs2-fs21	FP saved	callee
f28-f31	ft8-ft11	FP temps	caller

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Instruction Set Architecture

- Memory addressing
  - RISC-V: byte addressed, aligned accesses faster
- Addressing modes
  - RISC-V: Register, immediate, displacement (base+offset)
  - Other examples: autoincrement, indexed, PC-relative
- Types and size of operands
  - RISC-V: 8-bit, 32-bit, 64-bit

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Instruction Set Architecture

- Operations
  - RISC-V: data transfer, arithmetic, logical, control, floating point
  - See Fig. 1.5 in text
- Control flow instructions
  - Use content of registers (RISC-V) vs. status bits (x86, ARMv7, ARMv8)
  - Return address in register (RISC-V, ARMv7, ARMv8) vs. on stack (x86)
- Encoding
  - Fixed (RISC-V, ARMv7/v8 except compact instruction set) vs. variable length (x86)

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Trends in Technology

- Integrated circuit technology (Moore's Law)
  - Transistor density: 35%/year
  - Die size: 10-20%/year
  - Integration overall: 40-55%/year
- DRAM capacity: 25-40%/year (slowing)
  - 8 Gb (2014), 16 Gb (2019), possibly no 32 Gb
- Flash capacity: 50-60%/year
  - 8-10X cheaper/bit than DRAM
- Magnetic disk capacity: recently slowed to 5%/year
  - Density increases may no longer be possible, maybe increase from 7 to 9 platters
  - 8-10X cheaper/bit then Flash
  - 200-300X cheaper/bit than DRAM

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

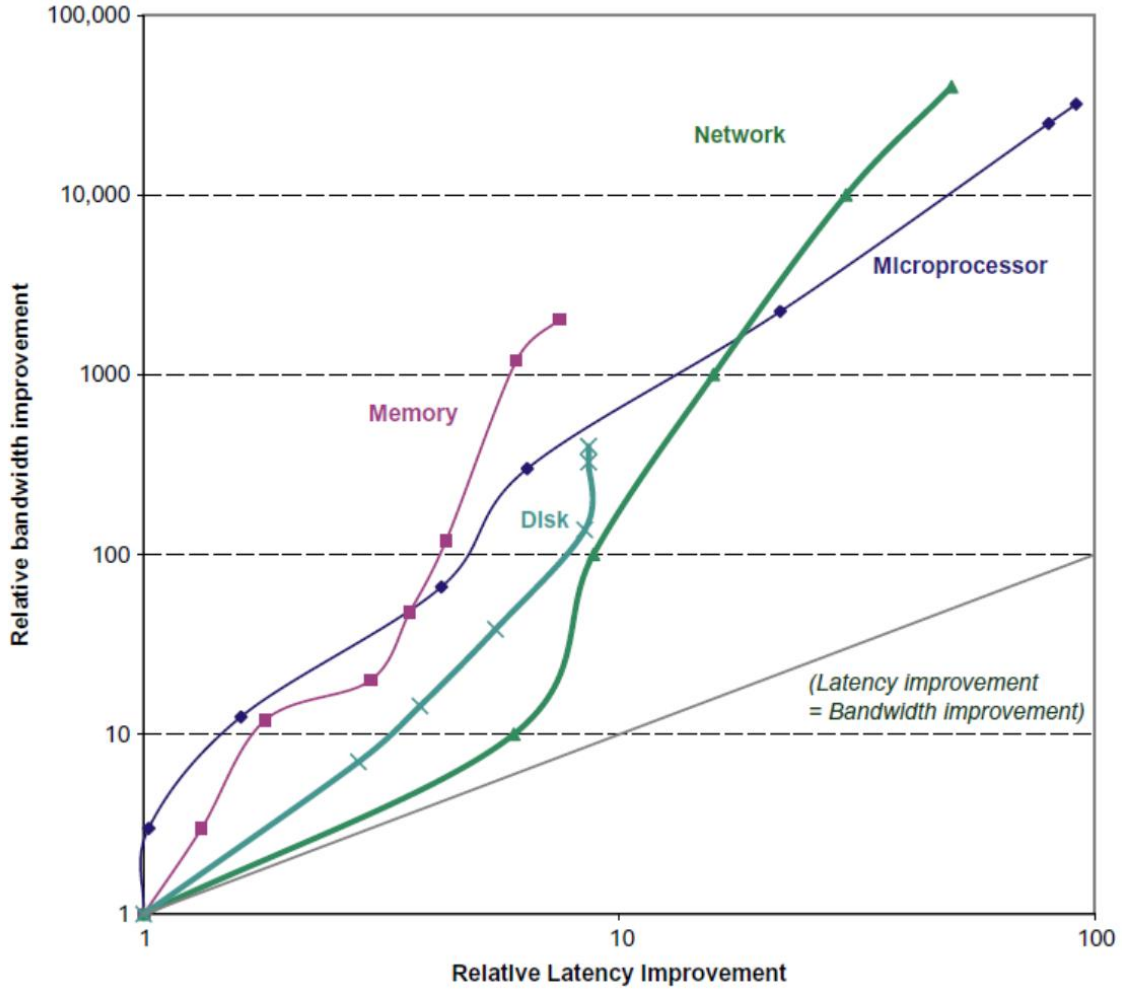
Bandwidth and Latency

- Bandwidth or throughput
  - Total work done in a given time
  - 32,000-40,000X improvement for processors
  - 300-1200X improvement for memory and disks
- Latency or response time
  - Time between start and completion of an event
  - 50-90X improvement for processors
  - 6-8X improvement for memory and disks

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Bandwidth and Latency



Log-log plot of bandwidth and latency milestones

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Transistors and Wires

- Feature size
  - Minimum size of transistor or wire in x or y dimension
  - 10 microns in 1971 to .011 microns in 2017
  - Transistor performance scales linearly
    - Wire delay does not improve with feature size!
  - Integration density scales quadratically

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Power and Energy

- Problem: Get power in, get power out
- Thermal Design Power (TDP)
  - Characterizes sustained power consumption
  - Used as target for power supply and cooling system
  - Lower than peak power (1.5X higher), higher than average power consumption
- Clock rate can be reduced dynamically to limit power consumption
- Energy per task is often a better measurement

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

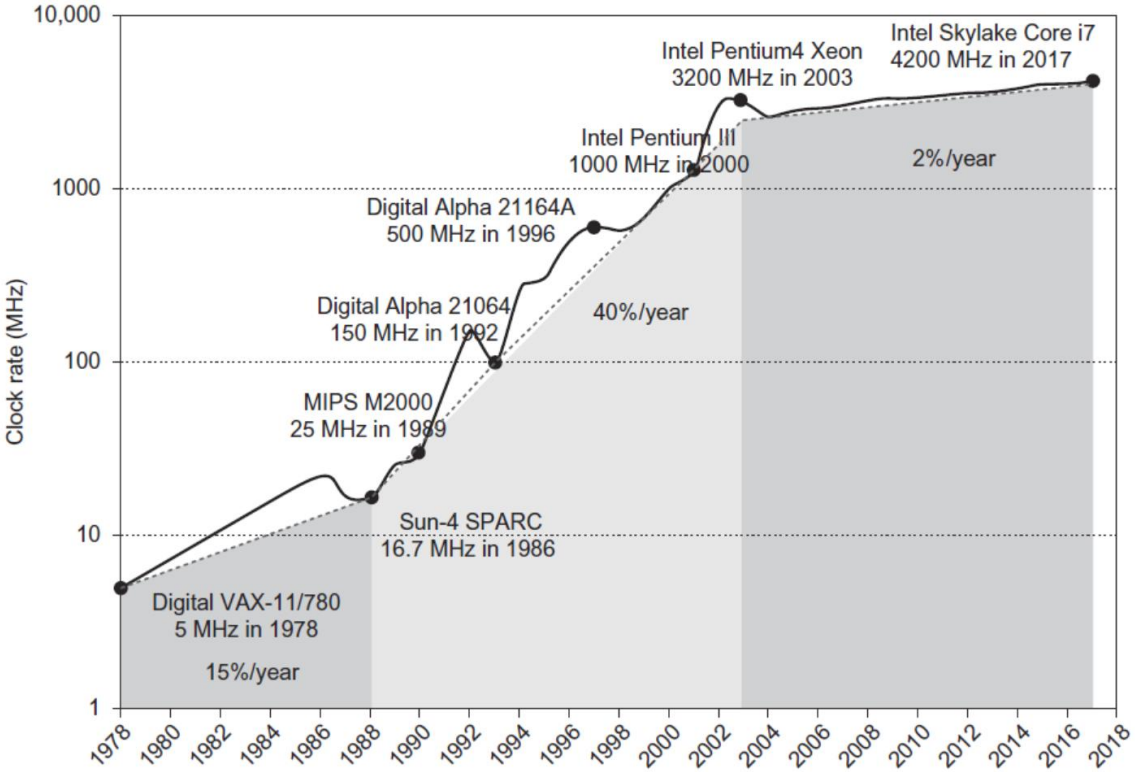
Dynamic Energy and Power

- Dynamic energy
  - Transistor switch from 0 -> 1 or 1 -> 0
  - ½ x Capacitive load x Voltage^2
- Dynamic power
  - ½ x Capacitive load x Voltage^2 x Frequency switched
- Reducing clock rate reduces power, not energy

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights reserved.

Power

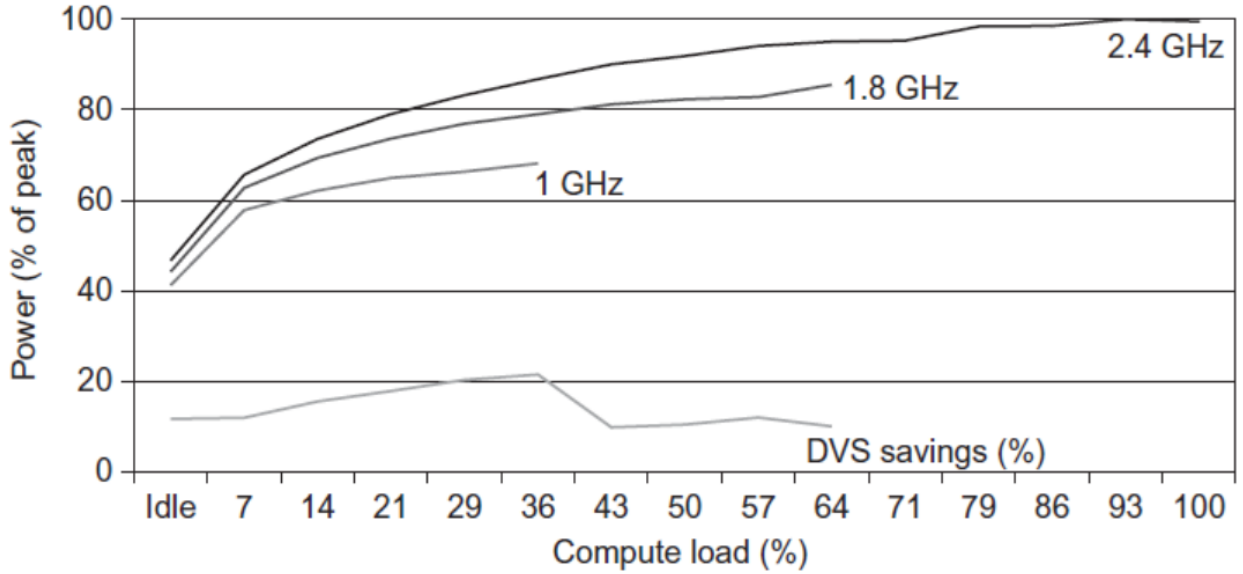
- Intel 80386 consumed ~ 2 W
- 3.3 GHz Intel Core i7 consumes 130 W
- Heat must be dissipated from 1.5 x 1.5 cm chip
- This is the limit of what can be cooled by air



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights reserved.

Reducing Power

- Techniques for reducing power:
  - Do nothing well
  - Dynamic Voltage-Frequency Scaling



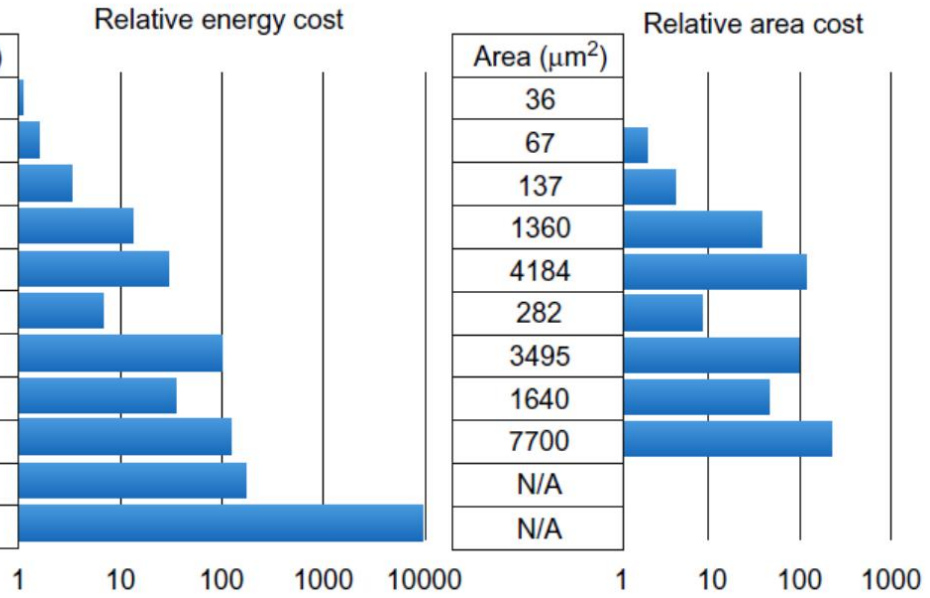
  - Low power state for DRAM, disks
  - Overclocking, turning off cores

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights reserved.

Static Power

- Static power consumption
  - 25-50% of total power
  - $Current_{static}$ x Voltage
  - Scales with number of transistors
  - To reduce: power gating

Operation:	Energy (pJ)	Area (μm^2)
8b Add	0.03	36
16b Add	0.05	67
32b Add	0.1	137
16b FB Add	0.4	1360
32b FB Add	0.9	4184
8b Mult	0.2	282
32b Mult	3.1	3495
16b FB Mult	1.1	1640
32b FB Mult	3.7	7700
32b SRAM Read (8KB)	5	N/A
32b DRAM Read	640	N/A



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights reserved.

Trends in Cost

- Cost driven down by learning curve
  - Yield
- DRAM: price closely tracks cost
- Microprocessors: price depends on volume
  - 10% less for each doubling of volume

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Integrated Circuit Cost

- Integrated circuit

$$Cost\ of\ integrated\ circuit = \frac{Cost\ of\ die + Cost\ of\ testing\ die + Cost\ of\ packaging\ and\ final\ test}{Final\ test\ yield}$$

$$Cost\ of\ die = \frac{Cost\ of\ wafer}{Dies\ per\ wafer\ x\ Die\ yield}$$

$$Dies\ per\ wafer = \frac{\pi \times (Wafer\ diameter/2)^2}{Die\ area} - \frac{\pi \times Wafer\ diameter}{\sqrt{2 \times Die\ area}}$$

- Bose-Einstein formula:

$$Die\ yield = Wafer\ yield \times 1/(1 + Defects\ per\ unit\ area \times Die\ area)^N$$

- Defects per unit area = 0.016-0.057 defects per square cm (2010)
- N = process-complexity factor = 11.5-15.5 (40 nm, 2010)

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Dependability

- Module reliability
  - Mean time to failure (MTTF)
  - Mean time to repair (MTTR)
  - Mean time between failures (MTBF) = MTTF + MTTR
  - Availability = MTTF / MTBF

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights reserved.

Measuring Performance

- Typical performance metrics:
  - Response time
  - Throughput
- Speedup of X relative to Y
  - Execution time$_Y$ / Execution time$_X$
- Execution time
  - Wall clock time: includes all system overheads
  - CPU time: only computation time
- Benchmarks
  - Kernels (e.g. matrix multiply)
  - Toy programs (e.g. sorting)
  - Synthetic benchmarks (e.g. Dhrystone)
  - Benchmark suites (e.g. SPEC06fp, TPC-C)

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Principles of Computer Design

- Take Advantage of Parallelism
  - e.g. multiple processors, disks, memory banks, pipelining, multiple functional units
- Principle of Locality
  - Reuse of data and instructions
- Focus on the Common Case
  - Amdahl's Law

$$Execution\ time_{new} = Execution\ time_{old} \times \left((1 - Fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}} \right)$$

$$Speedup_{overall} = \frac{1}{(1 - Fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}}$$

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Principles of Computer Design

- The Processor Performance Equation

$$CPU\ time = CPU\ clock\ cycles\ for\ a\ program \times Clock\ cycle\ time$$

$$CPU\ time = \frac{CPU\ clock\ cycles\ for\ a\ program}{Clock\ rate}$$

$$CPI = \frac{CPU\ clock\ cycles\ for\ a\ program}{Instruction\ count}$$

$$CPU\ time = Instruction\ count \times Cycles\ per\ instruction \times Clock\ cycle\ time$$

$$\frac{Instructions}{Program} \times \frac{Clock\ cycles}{Instruction} \times \frac{Seconds}{Clock\ cycle} = \frac{Seconds}{Program} = CPU\ time$$

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Principles of Computer Design

- Different instruction types having different CPIs

$$CPU\ clock\ cycles = \sum_{i=1}^{n} IC_i \times CPI_i$$

$$CPU\ time = \left(\sum_{i=1}^{n} IC_i \times CPI_i \right) \times Clock\ cycle\ time$$

MK MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Principles of Computer Design

- Different instruction types having different CPIs

$$CPU\ clock\ cycles = \sum_{i=1}^{n} IC_i \times CPI_i$$

$$CPU\ time = \left(\sum_{i=1}^{n} IC_i \times CPI_i \right) \times Clock\ cycle\ time$$

MK MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights reserved.

Fallacies and Pitfalls

- All exponential laws must come to an end
  - Dennard scaling (constant power density)
    - Stopped by threshold voltage
  - Disk capacity
    - 30-100% per year to 5% per year
  - Moore's Law
    - Most visible with DRAM capacity
    - ITRS disbanded
    - Only four foundries left producing state-of-the-art logic chips
    - 11 nm, 3 nm might be the limit

MK MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights reserved.

Fallacies and Pitfalls

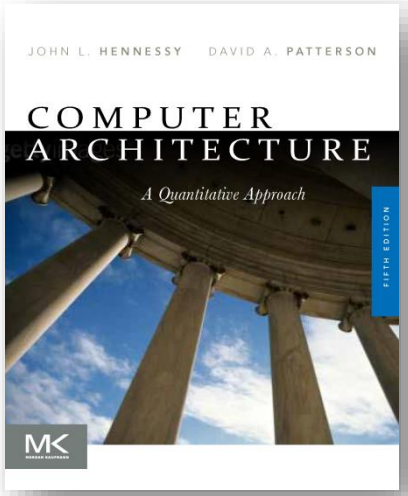
- Microprocessors are a silver bullet
  - Performance is now a programmer's burden
- Falling prey to Amdahl's Law
- A single point of failure
- Hardware enhancements that increase performance also improve energy efficiency, or are at worst energy neutral
- Benchmarks remain valid indefinitely
  - Compiler optimizations target benchmarks

Copyright © 2019, Elsevier Inc. All rights reserved.

Fallacies and Pitfalls

- The rated mean time to failure of disks is 1,200,000 hours or almost 140 years, so disks practically never fail
  - MTTF value from manufacturers assume regular replacement
- Peak performance tracks observed performance
- Fault detection can lower availability
  - Not all operations are needed for correct execution

Copyright © 2019, Elsevier Inc. All rights reserved.



Chapter 2

Memory Hierarchy Design

Copyright © 2019, Elsevier Inc. All rights Reserved

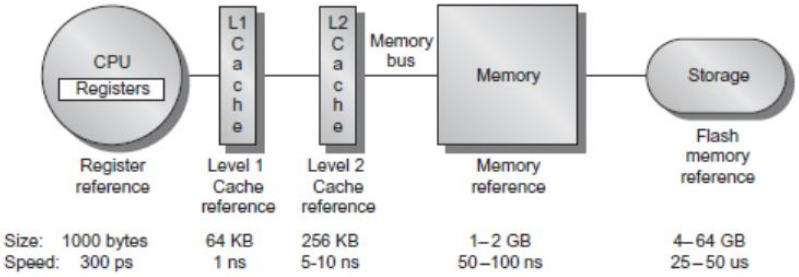
Introduction

- Programmers want unlimited amounts of memory with low latency
- Fast memory technology is more expensive per bit than slower memory
- Solution: organize memory system into a hierarchy
  - Entire addressable memory space available in largest, slowest memory
  - Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor
- Temporal and spatial locality insures that nearly all references can be found in smaller memories
  - Gives the allusion of a large, fast memory being presented to the processor

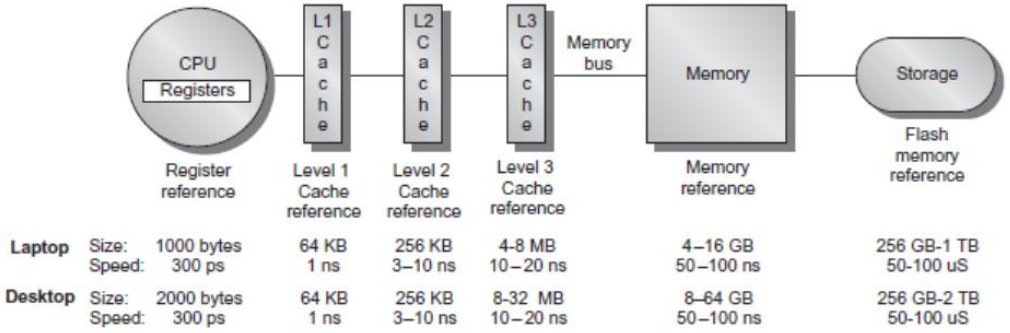
MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

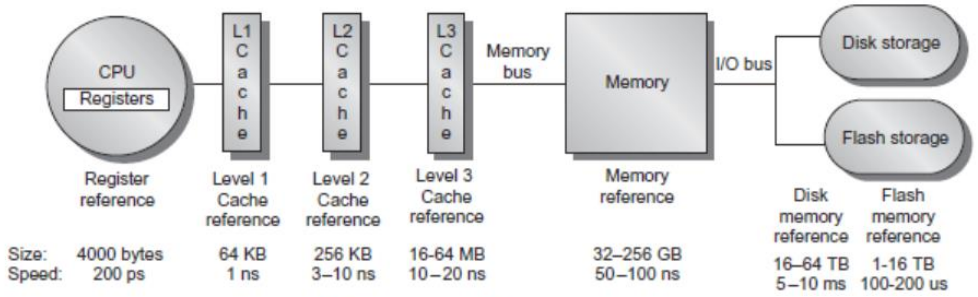
Memory Hierarchy



(A) Memory hierarchy for a personal mobile device



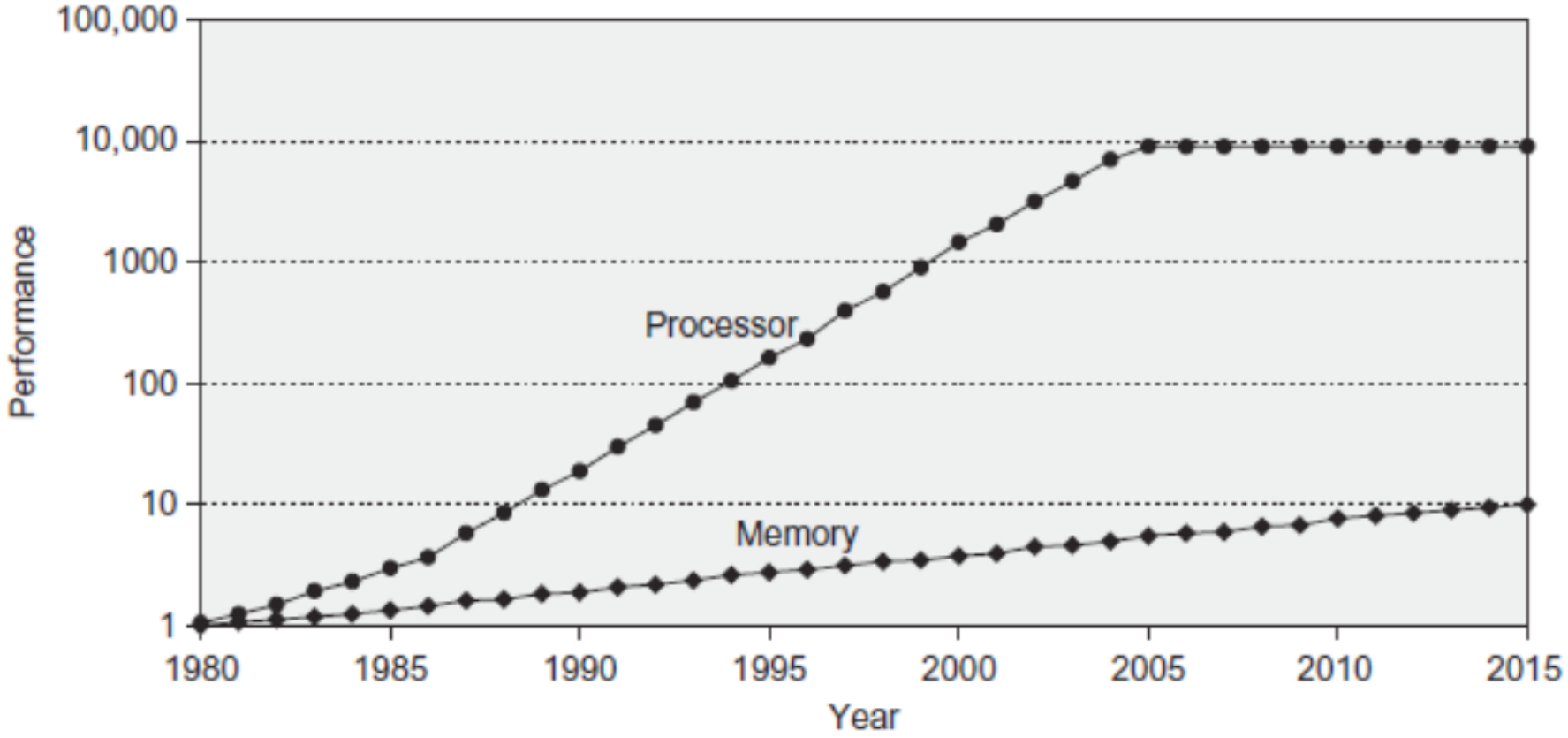
(B) Memory hierarchy for a laptop or a desktop



(C) Memory hierarchy for server

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Performance Gap



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Hierarchy Design

- Memory hierarchy design becomes more crucial with recent multi-core processors:
  - Aggregate peak bandwidth grows with # cores:
    - Intel Core i7 can generate two references per core per clock
    - Four cores and 3.2 GHz clock
      - 25.6 billion 64-bit data references/second +
      - 12.8 billion 128-bit instruction references/second
      - = 409.6 GB/s!
    - DRAM bandwidth is only 8% of this (34.1 GB/s)
    - Requires:
      - Multi-port, pipelined caches
      - Two levels of cache per core
      - Shared third-level cache on chip

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Performance and Power

- High-end microprocessors have >10 MB on-chip cache
  - Consumes large amount of area and power budget

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Hierarchy Basics

- When a word is not found in the cache, a *miss* occurs:
  - Fetch word from lower level in hierarchy, requiring a higher latency reference
  - Lower level may be another cache or the main memory
  - Also fetch the other words contained within the *block*
    - Takes advantage of spatial locality
  - Place block into cache in any location within its *set*, determined by address
    - block address MOD number of sets in cache

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Hierarchy Basics

- *n* sets => *n-way set associative*
  - *Direct-mapped cache* => one block per set
  - *Fully associative* => one set

- Writing to cache: two strategies
  - *Write-through*
    - Immediately update lower levels of hierarchy
  - *Write-back*
    - Only update lower levels of hierarchy when an updated block is replaced
  - Both strategies use *write buffer* to make writes asynchronous

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Hierarchy Basics

- Miss rate
  - Fraction of cache access that result in a miss
- Causes of misses
  - Compulsory
    - First reference to a block
  - Capacity
    - Blocks discarded and later retrieved
  - Conflict
    - Program makes repeated references to multiple addresses from different blocks that map to the same location in the cache

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Hierarchy Basics

$$\frac{Misses}{Instruction} = \frac{Miss\,rate \times Memory\,accesses}{Instruction\,count} = Miss\,rate \times \frac{Memory\,accesses}{Instruction}$$

$$Average\,memory\,access\,time = Hit\,time + Miss\,rate \times Miss\,penalty$$

- Speculative and multithreaded processors may execute other instructions during a miss
  - Reduces performance impact of misses

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Hierarchy Basics

- Six basic cache optimizations:
  - Larger block size
    - Reduces compulsory misses
    - Increases capacity and conflict misses, increases miss penalty
  - Larger total cache capacity to reduce miss rate
    - Increases hit time, increases power consumption
  - Higher associativity
    - Reduces conflict misses
    - Increases hit time, increases power consumption
  - Higher number of cache levels
    - Reduces overall memory access time
  - Giving priority to read misses over writes
    - Reduces miss penalty
  - Avoiding address translation in cache indexing
    - Reduces hit time

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Technology and Optimizations

- Performance metrics
  - Latency is concern of cache
  - Bandwidth is concern of multiprocessors and I/O
  - Access time
    - Time between read request and when desired word arrives
  - Cycle time
    - Minimum time between unrelated requests to memory
- SRAM memory has low latency, use for cache
- Organize DRAM chips into many banks for high bandwidth, use for main memory

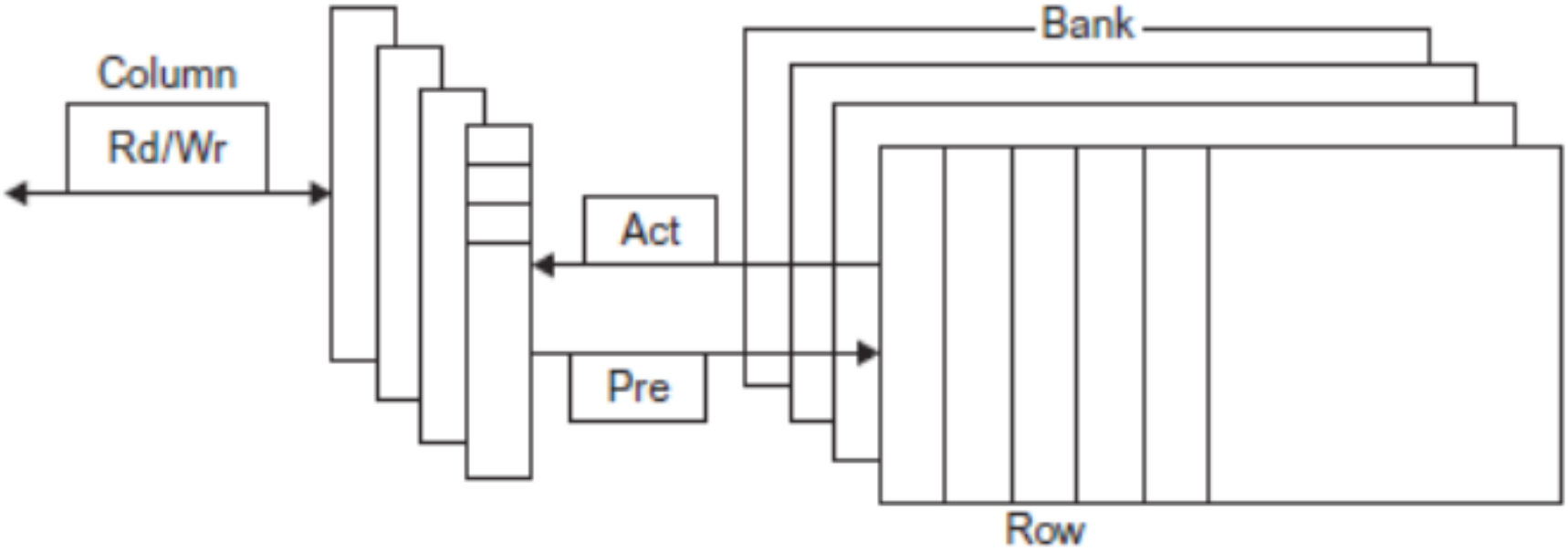
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Technology

- SRAM
  - Requires low power to retain bit
  - Requires 6 transistors/bit
- DRAM
  - Must be re-written after being read
  - Must also be periodically refeshed
    - Every ~ 8 ms (roughly 5% of time)
    - Each row can be refreshed simultaneously
  - One transistor/bit
  - Address lines are multiplexed:
    - Upper half of address: row access strobe (RAS)
    - Lower half of address: column access strobe (CAS)

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Internal Organization of DRAM



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Technology

- Amdahl:
  - Memory capacity should grow linearly with processor speed
  - Unfortunately, memory capacity and speed has not kept pace with processors
- Some optimizations:
  - Multiple accesses to same row
  - Synchronous DRAM
    - Added clock to DRAM interface
    - Burst mode with critical word first
  - Wider interfaces
  - Double data rate (DDR)
  - Multiple banks on each DRAM device

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Optimizations

			Best case access time (no precharge)			Precharge needed
Production year	Chip size	DRAM type	RAS time (ns)	CAS time (ns)	Total (ns)	Total (ns)
2000	256M bit	DDR1	21	21	42	63
2002	512M bit	DDR1	15	15	30	45
2004	1G bit	DDR2	15	15	30	45
2006	2G bit	DDR2	10	10	20	30
2010	4G bit	DDR3	13	13	26	39
2016	8G bit	DDR4	13	13	26	39

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Optimizations

Standard	I/O clock rate	M transfers/s	DRAM name	MiB/s/DIMM	DIMM name
DDR1	133	266	DDR266	2128	PC2100
DDR1	150	300	DDR300	2400	PC2400
DDR1	200	400	DDR400	3200	PC3200
DDR2	266	533	DDR2-533	4264	PC4300
DDR2	333	667	DDR2-667	5336	PC5300
DDR2	400	800	DDR2-800	6400	PC6400
DDR3	533	1066	DDR3-1066	8528	PC8500
DDR3	666	1333	DDR3-1333	10,664	PC10700
DDR3	800	1600	DDR3-1600	12,800	PC12800
DDR4	1333	2666	DDR4-2666	21,300	PC21300

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Optimizations

- DDR:
  - DDR2
    - Lower power (2.5 V -> 1.8 V)
    - Higher clock rates (266 MHz, 333 MHz, 400 MHz)
  - DDR3
    - 1.5 V
    - 800 MHz
  - DDR4
    - 1-1.2 V
    - 1333 MHz
- GDDR5 is graphics memory based on DDR3

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

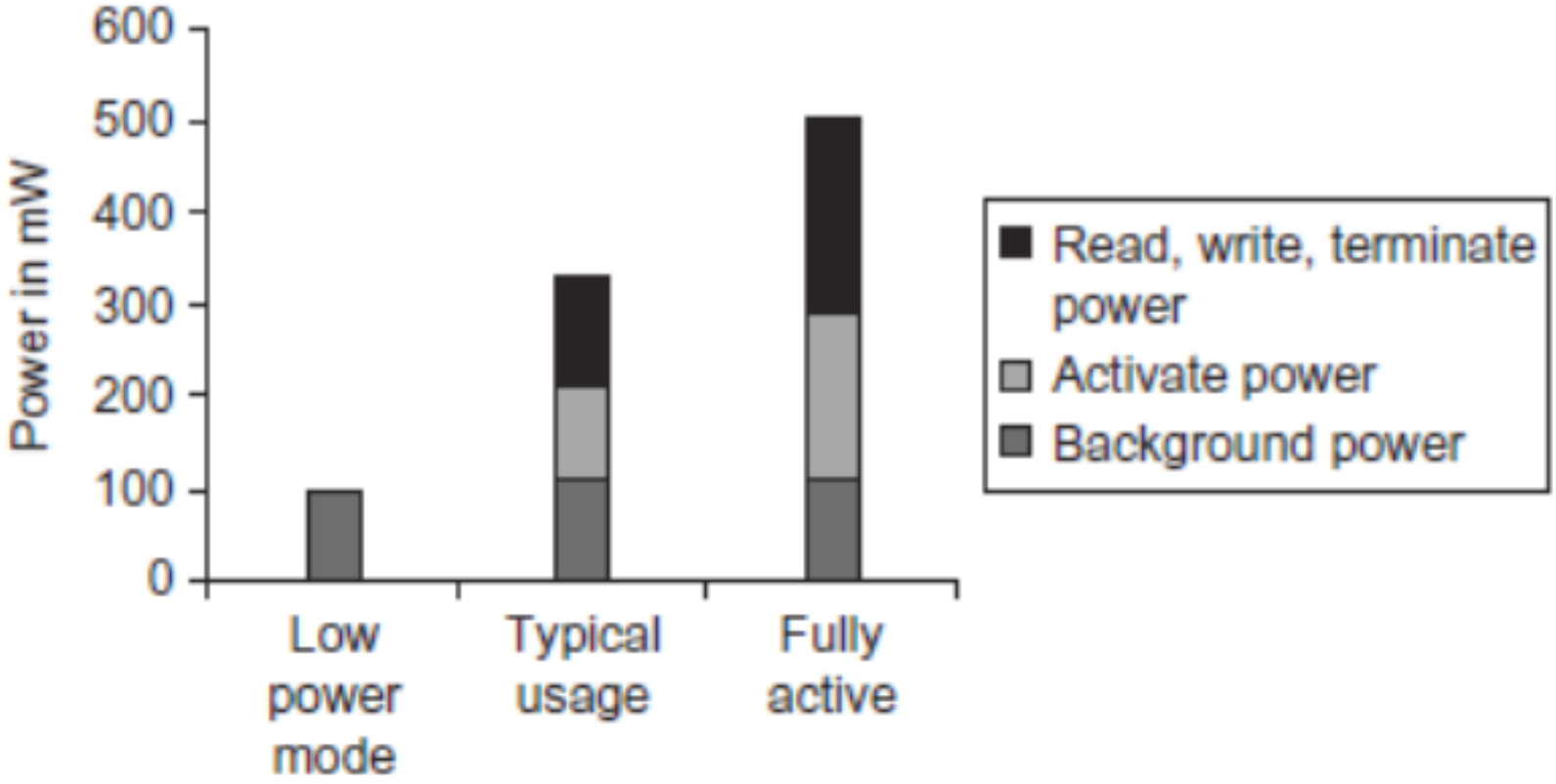
Memory Optimizations

- Reducing power in SDRAMs:
  - Lower voltage
  - Low power mode (ignores clock, continues to refresh)
- Graphics memory:
  - Achieve 2-5 X bandwidth per DRAM vs. DDR3
    - Wider interfaces (32 vs. 16 bit)
    - Higher clock rate
      - Possible because they are attached via soldering instead of socketted DIMM modules

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

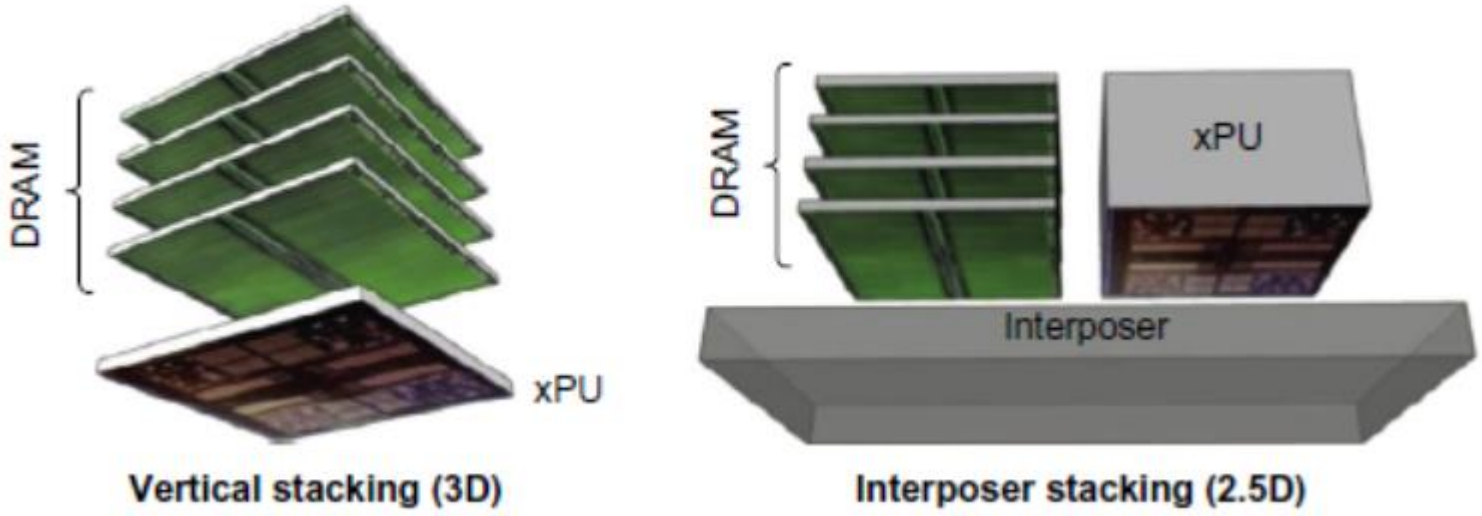
Memory Power Consumption



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Stacked/Embedded DRAMs

- Stacked DRAMs in same package as processor
  - High Bandwidth Memory (HBM)



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Flash Memory

- Type of EEPROM
- Types: NAND (denser) and NOR (faster)
- NAND Flash:
  - Reads are sequential, reads entire page (.5 to 4 KiB)
  - 25 us for first byte, 40 MiB/s for subsequent bytes
  - SDRAM: 40 ns for first byte, 4.8 GB/s for subsequent bytes
  - 2 KiB transfer: 75 uS vs 500 ns for SDRAM, 150X slower
  - 300 to 500X faster than magnetic disk

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

NAND Flash Memory

- Must be erased (in blocks) before being overwritten
- Nonvolatile, can use as little as zero power
- Limited number of write cycles (~100,000)
- $2/GiB, compared to $20-40/GiB for SDRAM and $0.09 GiB for magnetic disk

- Phase-Change/Memrister Memory
  - Possibly 10X improvement in write performance and 2X improvement in read performance

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Dependability

- Memory is susceptible to cosmic rays
- *Soft errors*: dynamic errors
  - Detected and fixed by error correcting codes (ECC)
- *Hard errors*: permanent errors
  - Use spare rows to replace defective rows
- Chipkill: a RAID-like error recovery technique

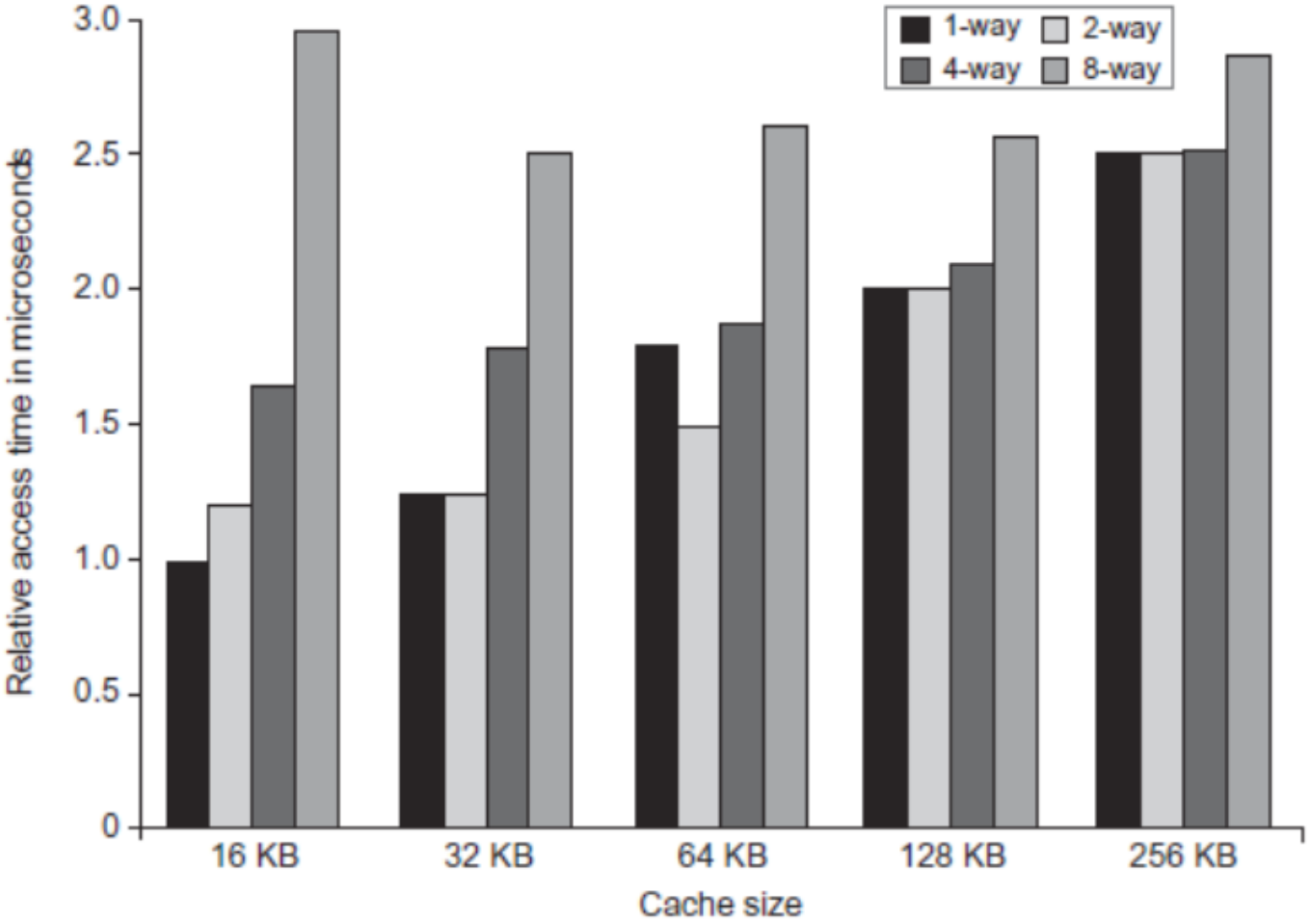
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Advanced Optimizations

- Reduce hit time
  - Small and simple first-level caches
  - Way prediction
- Increase bandwidth
  - Pipelined caches, multibanked caches, non-blocking caches
- Reduce miss penalty
  - Critical word first, merging write buffers
- Reduce miss rate
  - Compiler optimizations
- Reduce miss penalty or miss rate via parallelization
  - Hardware or compiler prefetching

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

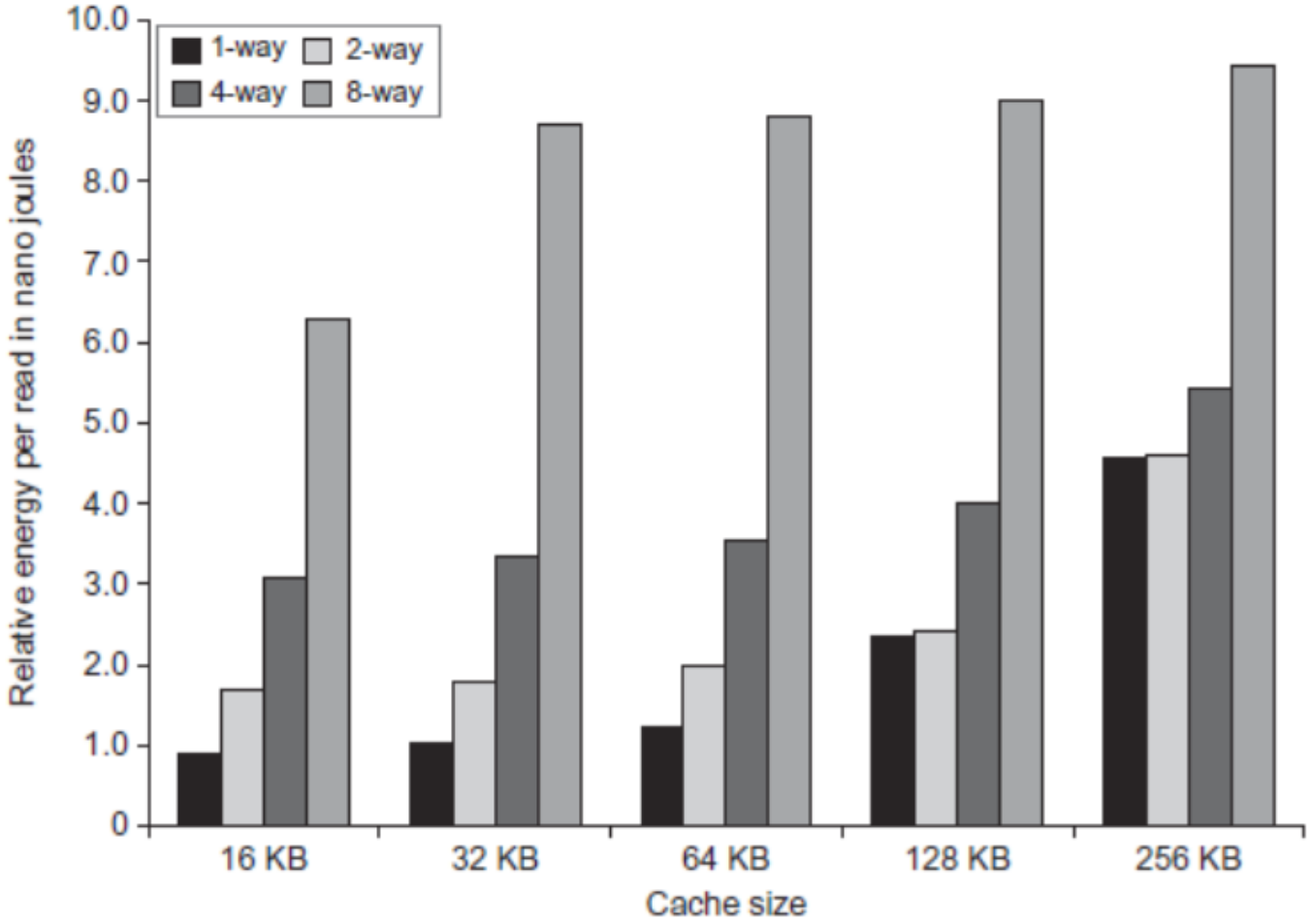
L1 Size and Associativity



Access time vs. size and associativity

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

L1 Size and Associativity



Energy per read vs. size and associativity

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Way Prediction

- To improve hit time, predict the way to pre-set mux
  - Mis-prediction gives longer hit time
  - Prediction accuracy
    - > 90% for two-way
    - > 80% for four-way
    - I-cache has better accuracy than D-cache
  - First used on MIPS R10000 in mid-90s
  - Used on ARM Cortex-A8
- Extend to predict block as well
  - “Way selection”
  - Increases mis-prediction penalty

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Pipelined Caches

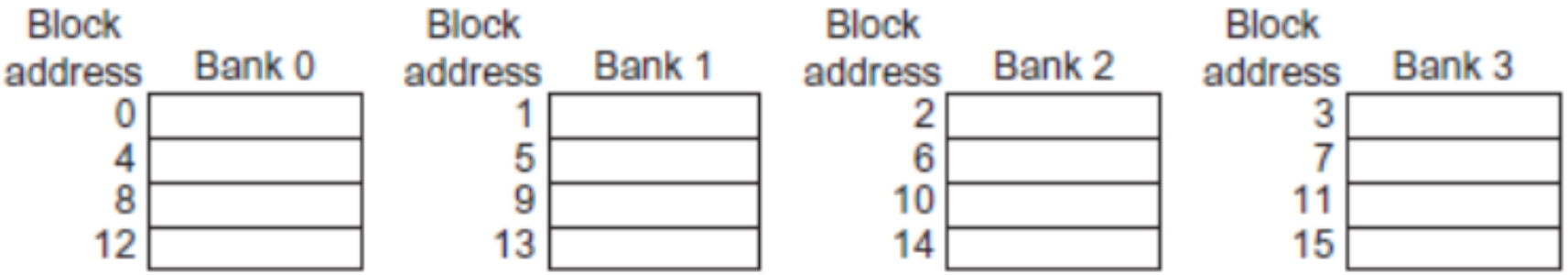
- Pipeline cache access to improve bandwidth
  - Examples:
    - Pentium: 1 cycle
    - Pentium Pro – Pentium III: 2 cycles
    - Pentium 4 – Core i7: 4 cycles
- Increases branch mis-prediction penalty
- Makes it easier to increase associativity

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Multibanked Caches

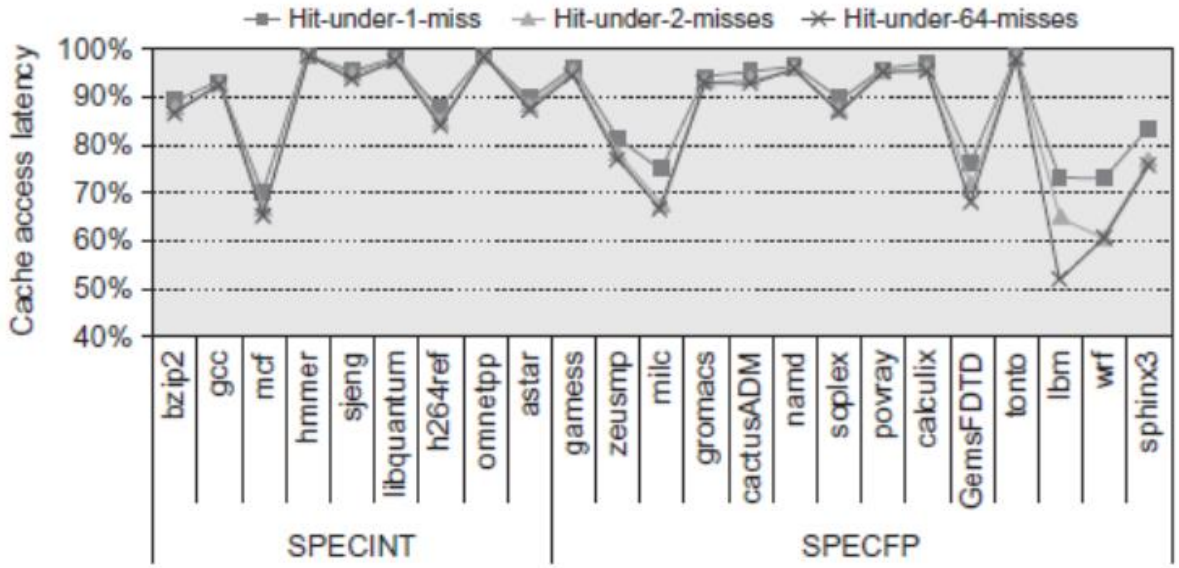
- Organize cache as independent banks to support simultaneous access
  - ARM Cortex-A8 supports 1-4 banks for L2
  - Intel i7 supports 4 banks for L1 and 8 banks for L2
- Interleave banks according to block address



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Nonblocking Caches

- Allow hits before previous misses complete
  - "Hit under miss"
  - "Hit under multiple miss"
- L2 must support this
- In general, processors can hide L1 miss penalty but not L2 miss penalty



Copyright © 2019, Elsevier Inc. All rights Reserved

Critical Word First, Early Restart

- Critical word first
  - Request missed word from memory first
  - Send it to the processor as soon as it arrives
- Early restart
  - Request words in normal order
  - Send missed work to the processor as soon as it arrives

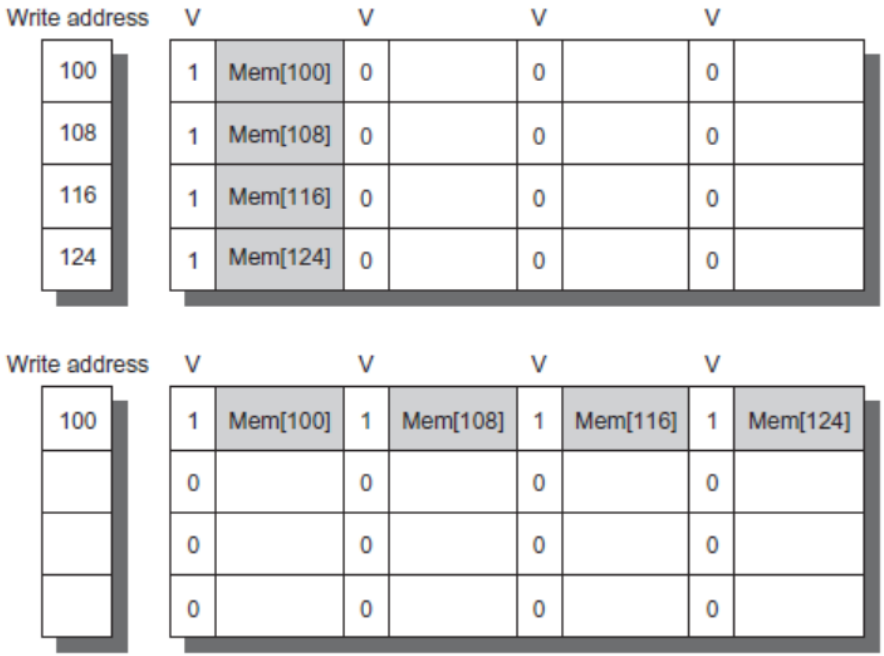
- Effectiveness of these strategies depends on block size and likelihood of another access to the portion of the block that has not yet been fetched

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Merging Write Buffer

- When storing to a block that is already pending in the write buffer, update write buffer
- Reduces stalls due to full write buffer
- Do not apply to I/O addresses



Write address	V		V		V		V	
100	1	Mem[100]	0		0		0	
108	1	Mem[108]	0		0		0	
116	1	Mem[116]	0		0		0	
124	1	Mem[124]	0		0		0	

No write buffering

Write address	V		V		V		V	
100	1	Mem[100]	1	Mem[108]	1	Mem[116]	1	Mem[124]
	0		0		0		0	
	0		0		0		0	
	0		0		0		0	

Write buffering

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Compiler Optimizations

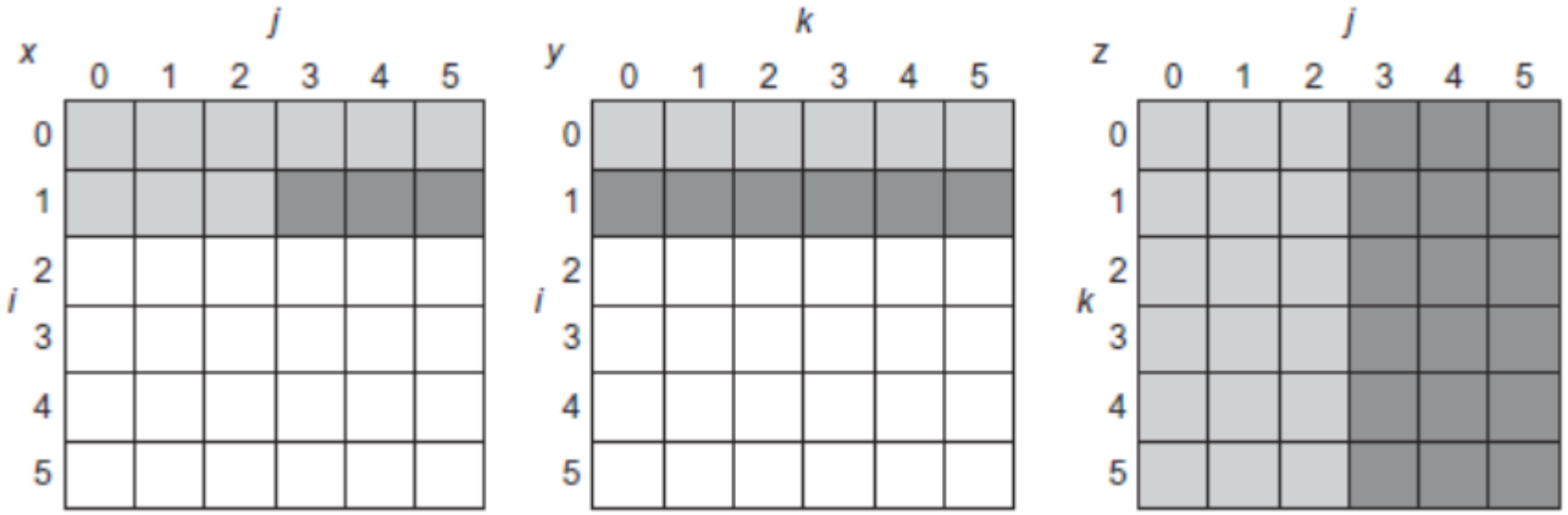
- Loop Interchange
  - Swap nested loops to access memory in sequential order
- Blocking
  - Instead of accessing entire rows or columns, subdivide matrices into blocks
  - Requires more memory accesses but improves locality of accesses

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Blocking

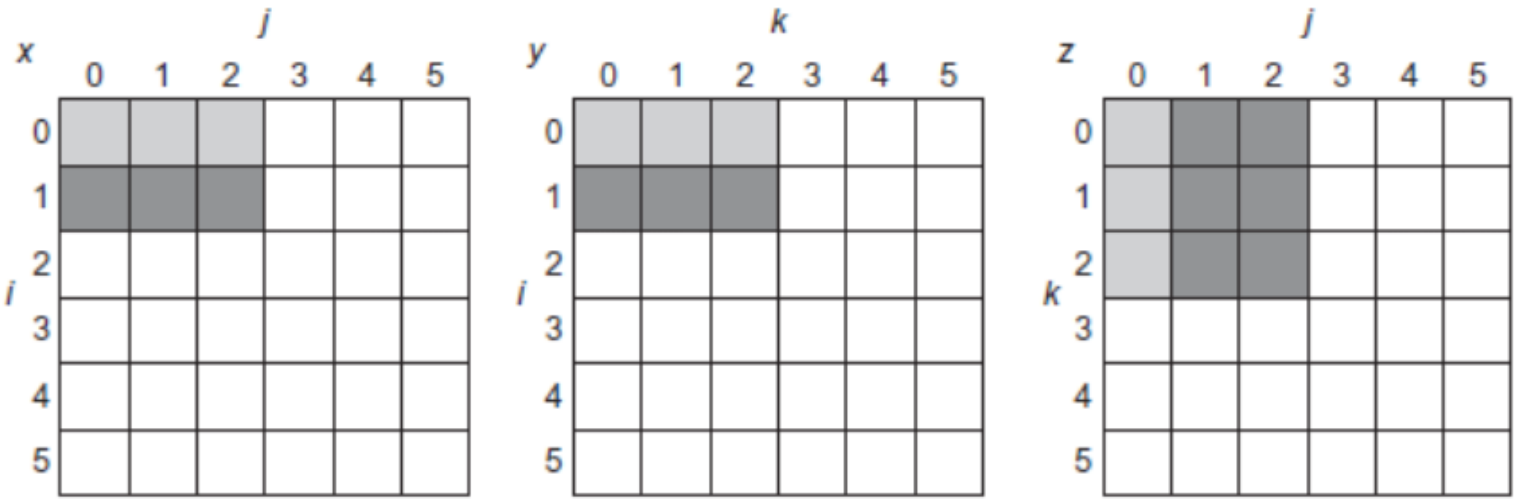
```
for (i = 0; i < N; i = i + 1)
  for (j = 0; j < N; j = j + 1)
  {
    r = 0;
    for (k = 0; k < N; k = k + 1)
      r = r + y[i][k]*z[k][j];
    x[i][j] = r;
};
```



Copyright © 2019, Elsevier Inc. All rights Reserved

Blocking

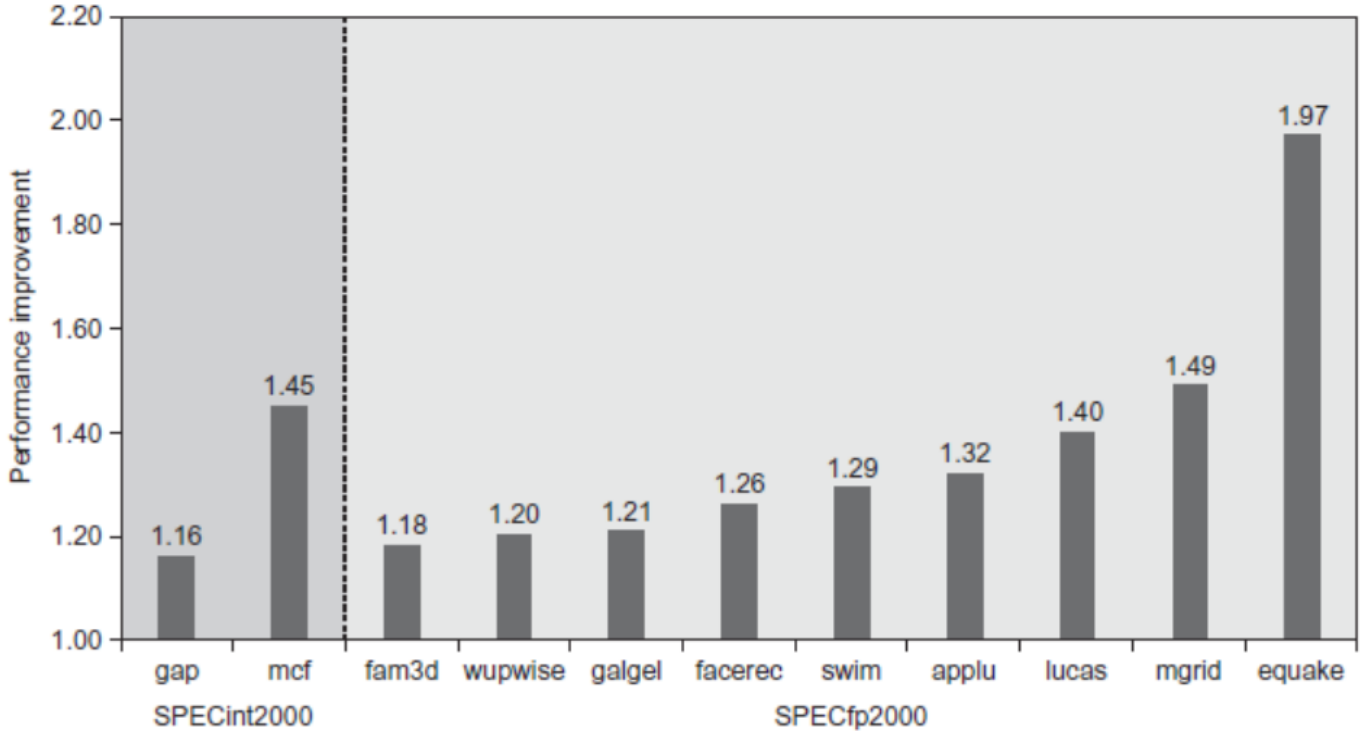
```
for (jj = 0; jj < N; jj = jj + B)
  for (kk = 0; kk < N; kk = kk + B)
    for (i = 0; i < N; i = i + 1)
      for (j = jj; j < min(jj + B,N); j = j + 1)
      {
        r = 0;
        for (k = kk; k < min(kk + B,N); k = k + 1)
          r = r + y[i][k]*z[k][j];
        x[i][j] = x[i][j] + r;
};
```



Copyright © 2019, Elsevier Inc. All rights Reserved

Hardware Prefetching

- Fetch two blocks on miss (include next sequential block)



Pentium 4 Pre-fetching

Copyright © 2019, Elsevier Inc. All rights Reserved

Compiler Prefetching

- Insert prefetch instructions before data is needed
- Non-faulting: prefetch doesn't cause exceptions

- Register prefetch
  - Loads data into register
- Cache prefetch
  - Loads data into cache

- Combine with loop unrolling and software pipelining

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Use HBM to Extend Hierarchy

- 128 MiB to 1 GiB
- Smaller blocks require substantial tag storage
- Larger blocks are potentially inefficient

- One approach (L-H):
  - Each SDRAM row is a block index
  - Each row contains set of tags and 29 data segments
  - 29-set associative
  - Hit requires a CAS

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

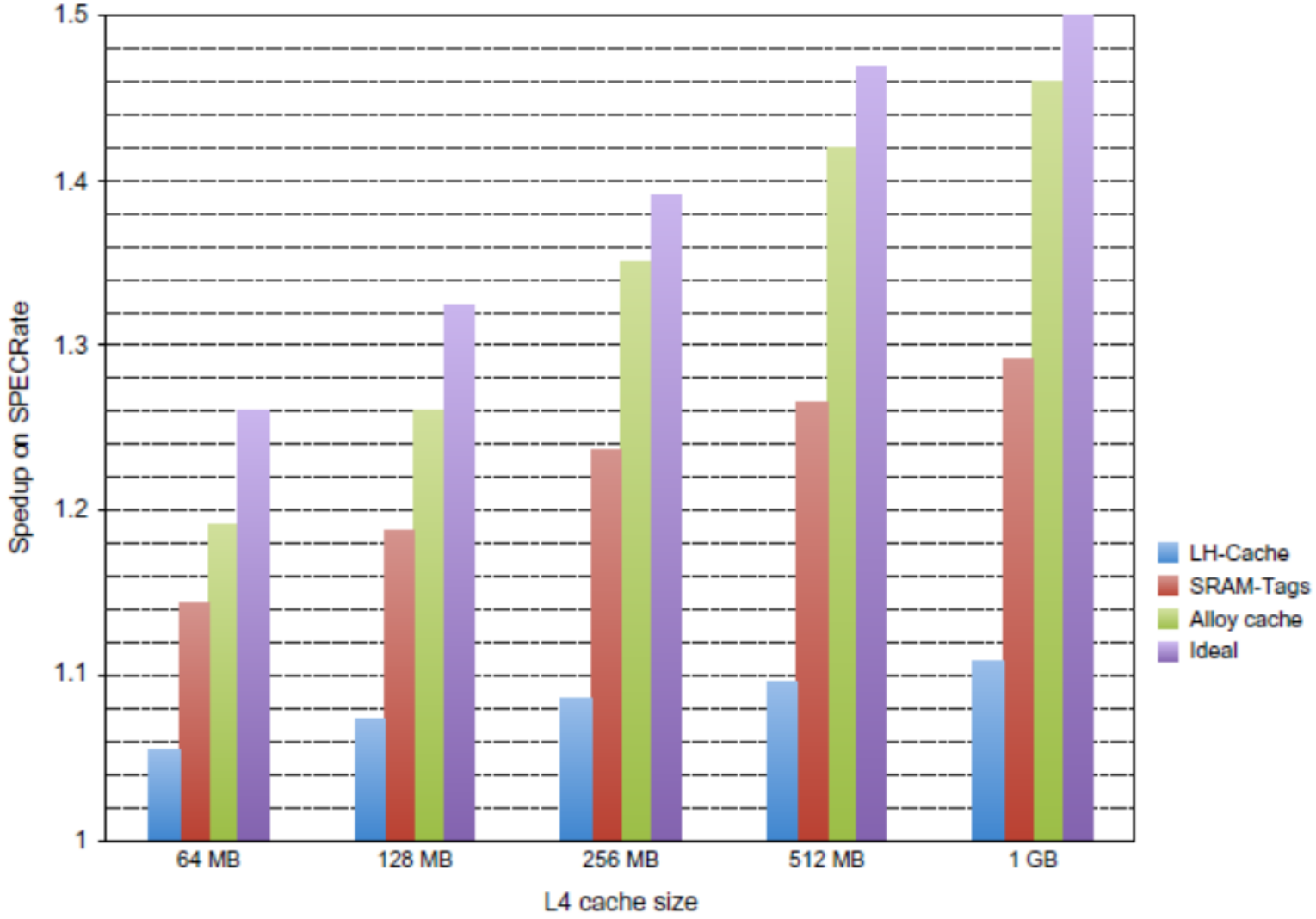
Use HBM to Extend Hierarchy

- Another approach (Alloy cache):
  - Mold tag and data together
  - Use direct mapped
- Both schemes require two DRAM accesses for misses
  - Two solutions:
    - Use map to keep track of blocks
    - Predict likely misses

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Use HBM to Extend Hierarchy



MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Summary

Technique	Hit time	Band-width	Miss penalty	Miss rate	Power consumption	Hardware cost/ complexity	Comment
Small and simple caches	+			–	+	0	Trivial; widely used
Way-predicting caches	+				+	1	Used in Pentium 4
Pipelined & banked caches	–	+				1	Widely used
Nonblocking caches		+	+			3	Widely used
Critical word first and early restart			+			2	Widely used
Merging write buffer			+			1	Widely used with write through
Compiler techniques to reduce cache misses				+		0	Software is a challenge, but many compilers handle common linear algebra calculations
Hardware prefetching of instructions and data			+	+	–	2 instr., 3 data	Most provide prefetch instructions; modern high-end processors also automatically prefetch in hardware
Compiler-controlled prefetching			+	+		3	Needs nonblocking cache; possible instruction overhead; in many CPUs
HBM as additional level of cache		+/–	–	+	+	3	Depends on new packaging technology. Effects depend heavily on hit rate improvements

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Virtual Memory and Virtual Machines

- Protection via virtual memory
  - Keeps processes in their own memory space
- Role of architecture
  - Provide user mode and supervisor mode
  - Protect certain aspects of CPU state
  - Provide mechanisms for switching between user mode and supervisor mode
  - Provide mechanisms to limit memory accesses
  - Provide TLB to translate addresses

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Virtual Machines

- Supports isolation and security
- Sharing a computer among many unrelated users
- Enabled by raw speed of processors, making the overhead more acceptable

- Allows different ISAs and operating systems to be presented to user programs
  - “System Virtual Machines”
  - SVM software is called “virtual machine monitor” or “hypervisor”
  - Individual virtual machines run under the monitor are called “guest VMs”

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Requirements of VMM

- Guest software should:
  - Behave on as if running on native hardware
  - Not be able to change allocation of real system resources
- VMM should be able to “context switch” guests
- Hardware must allow:
  - System and use processor modes
  - Privileged subset of instructions for allocating system resources

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Impact of VMs on Virtual Memory

- Each guest OS maintains its own set of page tables
  - VMM adds a level of memory between physical and virtual memory called “real memory”
  - VMM maintains shadow page table that maps guest virtual addresses to physical addresses
    - Requires VMM to detect guest’s changes to its own page table
    - Occurs naturally if accessing the page table pointer is a privileged operation

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Extending the ISA for Virtualization

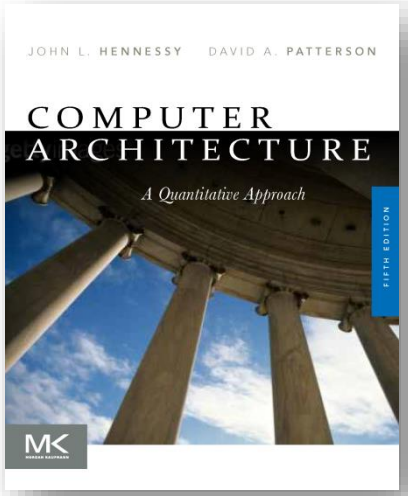
- Objectives:
  - Avoid flushing TLB
  - Use nested page tables instead of shadow page tables
  - Allow devices to use DMA to move data
  - Allow guest OS's to handle device interrupts
  - For security: allow programs to manage encrypted portions of code and data

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- Predicting cache performance of one program from another
- Simulating enough instructions to get accurate performance measures of the memory hierarchy
- Not deliverying high memory bandwidth in a cache-based system

Copyright © 2019, Elsevier Inc. All rights Reserved



Chapter 3

Instruction-Level Parallelism and Its Exploitation

Copyright © 2019, Elsevier Inc. All rights Reserved

Introduction

- Pipelining become universal technique in 1985
  - Overlaps execution of instructions
  - Exploits “Instruction Level Parallelism”
- Beyond this, there are two main approaches:
  - Hardware-based dynamic approaches
    - Used in server and desktop processors
    - Not used as extensively in PMP processors
  - Compiler-based static approaches
    - Not as successful outside of scientific applications

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Instruction-Level Parallelism

- When exploiting instruction-level parallelism, goal is to maximize CPI
  - Pipeline CPI =
    - Ideal pipeline CPI +
    - Structural stalls +
    - Data hazard stalls +
    - Control stalls
- Parallelism with basic block is limited
  - Typical size of basic block = 3-6 instructions
  - Must optimize across branches

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Data Dependence

- Loop-Level Parallelism
  - Unroll loop statically or dynamically
  - Use SIMD (vector processors and GPUs)
- Challenges:
  - Data dependency
    - Instruction *j* is data dependent on instruction *i* if
      - Instruction *i* produces a result that may be used by instruction *j*
      - Instruction *j* is data dependent on instruction *k* and instruction *k* is data dependent on instruction *i*
- Dependent instructions cannot be executed simultaneously

Copyright © 2019, Elsevier Inc. All rights Reserved

Data Dependence

- Dependencies are a property of programs
- Pipeline organization determines if dependence is detected and if it causes a stall

- Data dependence conveys:
  - Possibility of a hazard
  - Order in which results must be calculated
  - Upper bound on exploitable instruction level parallelism

- Dependencies that flow through memory locations are difficult to detect

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Name Dependence

- Two instructions use the same name but no flow of information
  - Not a true data dependence, but is a problem when reordering instructions
  - Antidependence: instruction j writes a register or memory location that instruction i reads
    - Initial ordering (i before j) must be preserved
  - Output dependence: instruction i and instruction j write the same register or memory location
    - Ordering must be preserved
- To resolve, use register renaming techniques

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Other Factors

- Data Hazards
  - Read after write (RAW)
  - Write after write (WAW)
  - Write after read (WAR)
- Control Dependence
  - Ordering of instruction i with respect to a branch instruction
    - Instruction control dependent on a branch cannot be moved before the branch so that its execution is no longer controlled by the branch
    - An instruction not control dependent on a branch cannot be moved after the branch so that its execution is controlled by the branch

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Examples

- Example 1:

```
    add x1,x2,x3
    beq x4,x0,L
    sub x1,x1,x6
L:  …
    or x7,x1,x8
```

- or instruction dependent on add and sub

- Example 2:

```
    add x1,x2,x3
    beq x12,x0,skip
    sub x4,x5,x6
    add x5,x4,x9
skip:
    or x7,x8,x9
```

- Assume x4 isn't used after skip
  - Possible to move sub before the branch

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Compiler Techniques for Exposing ILP

- Pipeline scheduling
  - Separate dependent instruction from the source instruction by the pipeline latency of the source instruction
- Example:

  for (i=999; i>=0; i=i-1)

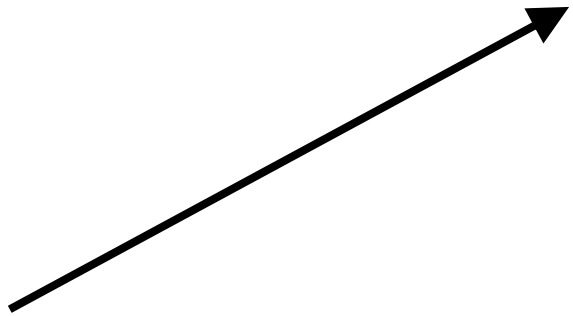
Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Pipeline Stalls

```
Loop:   fld      f0,0(x1)
        stall
        fadd.d f4,f0,f2
        stall
        stall
        fsd f4,0(x1)
        addi x1,x1,-8
        bne x1,x2,Loop
```



```
Loop:   fld      f0,0(x1)
        addi x1,x1,-8
        fadd.d f4,f0,f2
        stall
        stall
        fsd f4,0(x1)
        bne x1,x2,Loop
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Loop Unrolling

- Loop unrolling
  - Unroll by a factor of 4 (assume # elements is divisible by 4)
  - Eliminate unnecessary instructions

```
Loop:   fld f0,0(x1)
        fadd.d f4,f0,f2
        fsd f4,0(x1) //drop addi & bne
        fld f6,-8(x1)
        fadd.d f8,f6,f2
        fsd f8,-8(x1) //drop addi & bne
        fld f0,-16(x1)
        fadd.d f12,f0,f2
        fsd f12,-16(x1) //drop addi & bne
        fld f14,-24(x1)
        fadd.d f16,f14,f2
        fsd f16,-24(x1)
        addi x1,x1,-32
        bne x1,x2,Loop
```

- note: number of live registers vs. original loop

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Loop Unrolling/Pipeline Scheduling

- Pipeline schedule the unrolled loop:

```
Loop:   fld f0,0(x1)
        fld f6,-8(x1)
        fld f8,-16(x1)
        fld f14,-24(x1)
        fadd.d f4,f0,f2
        fadd.d f8,f6,f2
        fadd.d f12,f0,f2
        fadd.d f16,f14,f2
        fsd f4,0(x1)
        fsd f8,-8(x1)
        fsd f12,-16(x1)
        fsd f16,-24(x1)
        addi x1,x1,-32
        bne x1,x2,Loop
```

- 14 cycles
- 3.5 cycles per element

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Strip Mining

- Unknown number of loop iterations?
  - Number of iterations = *n*
  - Goal: make *k* copies of the loop body
  - Generate pair of loops:
    - First executes *n* mod *k* times
    - Second executes *n* / *k* times
    - “Strip mining”

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

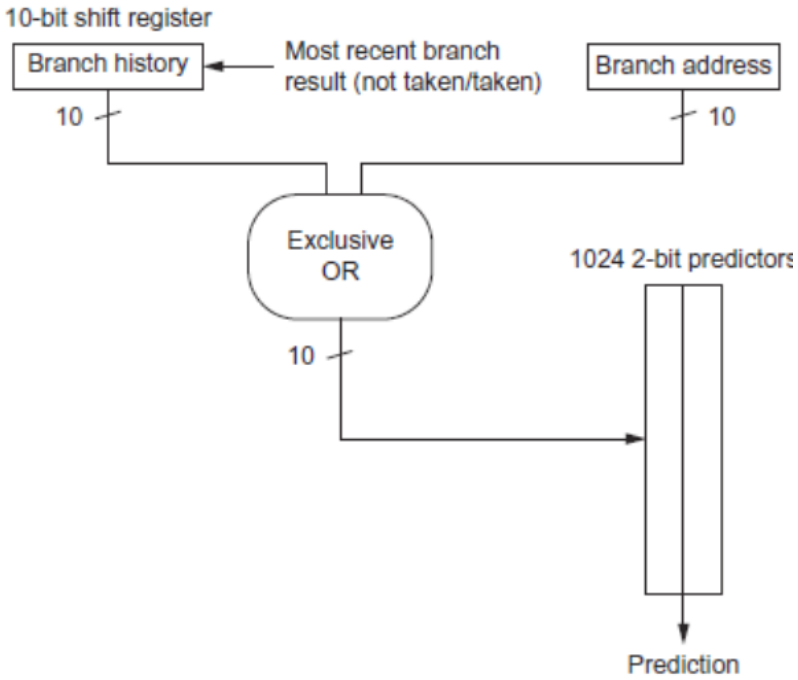
Branch Prediction

- Basic 2-bit predictor:
  - For each branch:
    - Predict taken or not taken
    - If the prediction is wrong two consecutive times, change prediction
- Correlating predictor:
  - Multiple 2-bit predictors for each branch
  - One for each possible combination of outcomes of preceding *n* branches
    - (*m*,*n*) predictor: behavior from last *m* branches to choose from 2^m *n*-bit predictors
- Tournament predictor:
  - Combine correlating predictor with local predictor

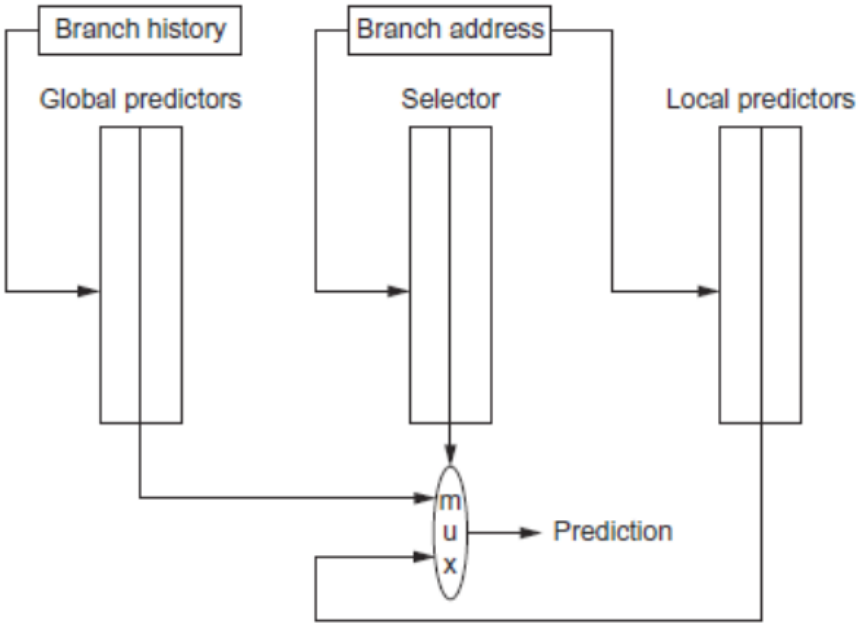
MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Branch Prediction



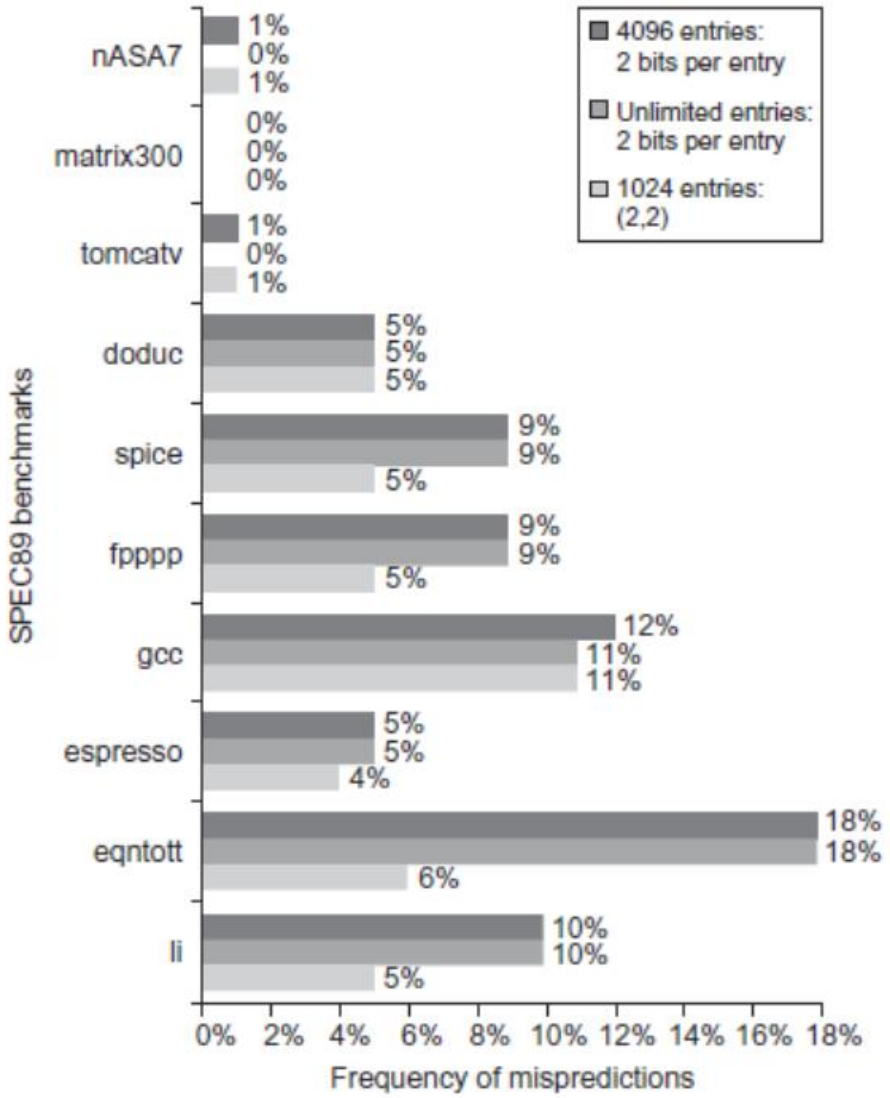
gshare



tournament

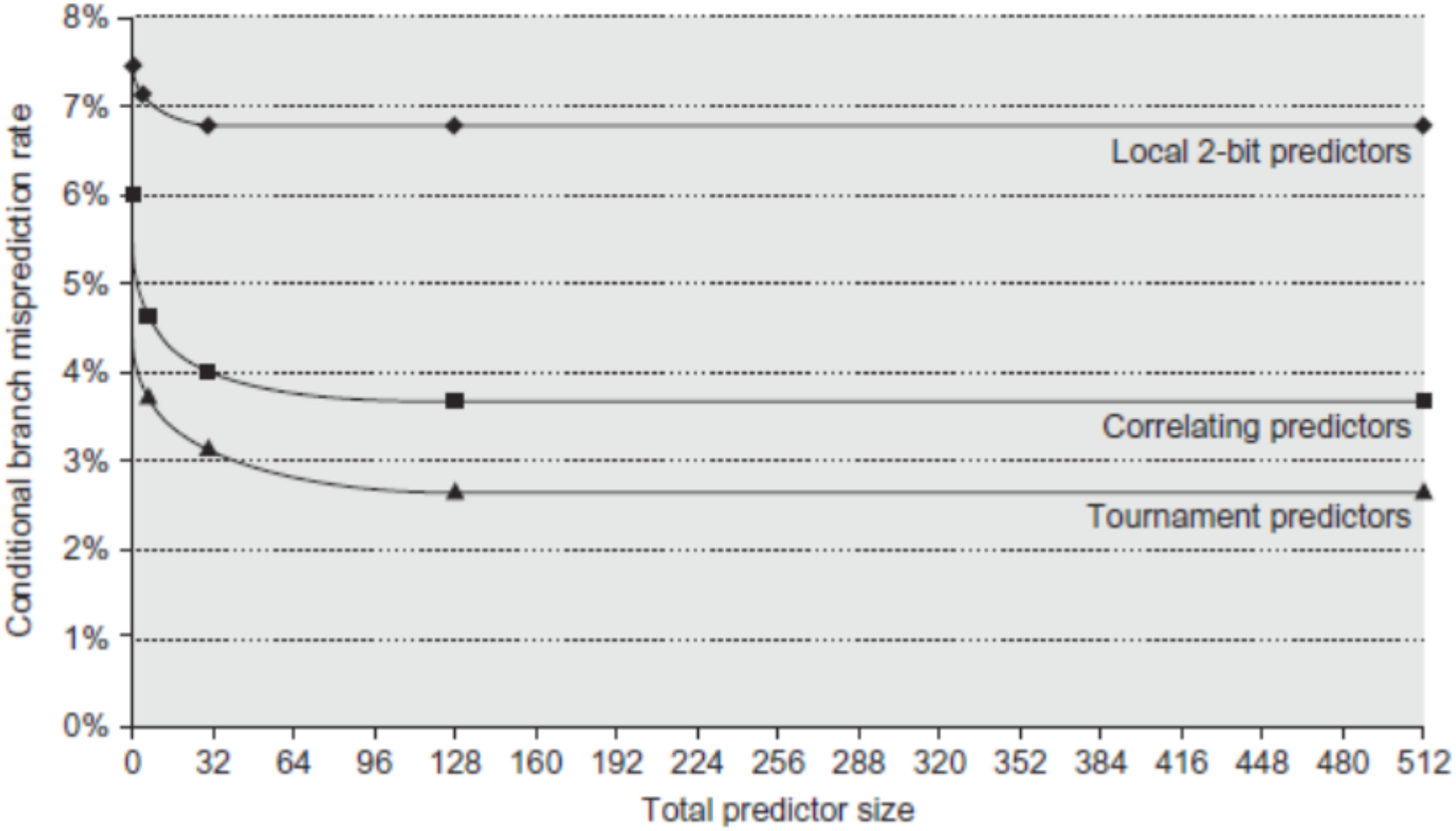
Copyright © 2019, Elsevier Inc. All rights Reserved

Branch Prediction Performance



Copyright © 2019, Elsevier Inc. All rights Reserved

Branch Prediction Performance



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

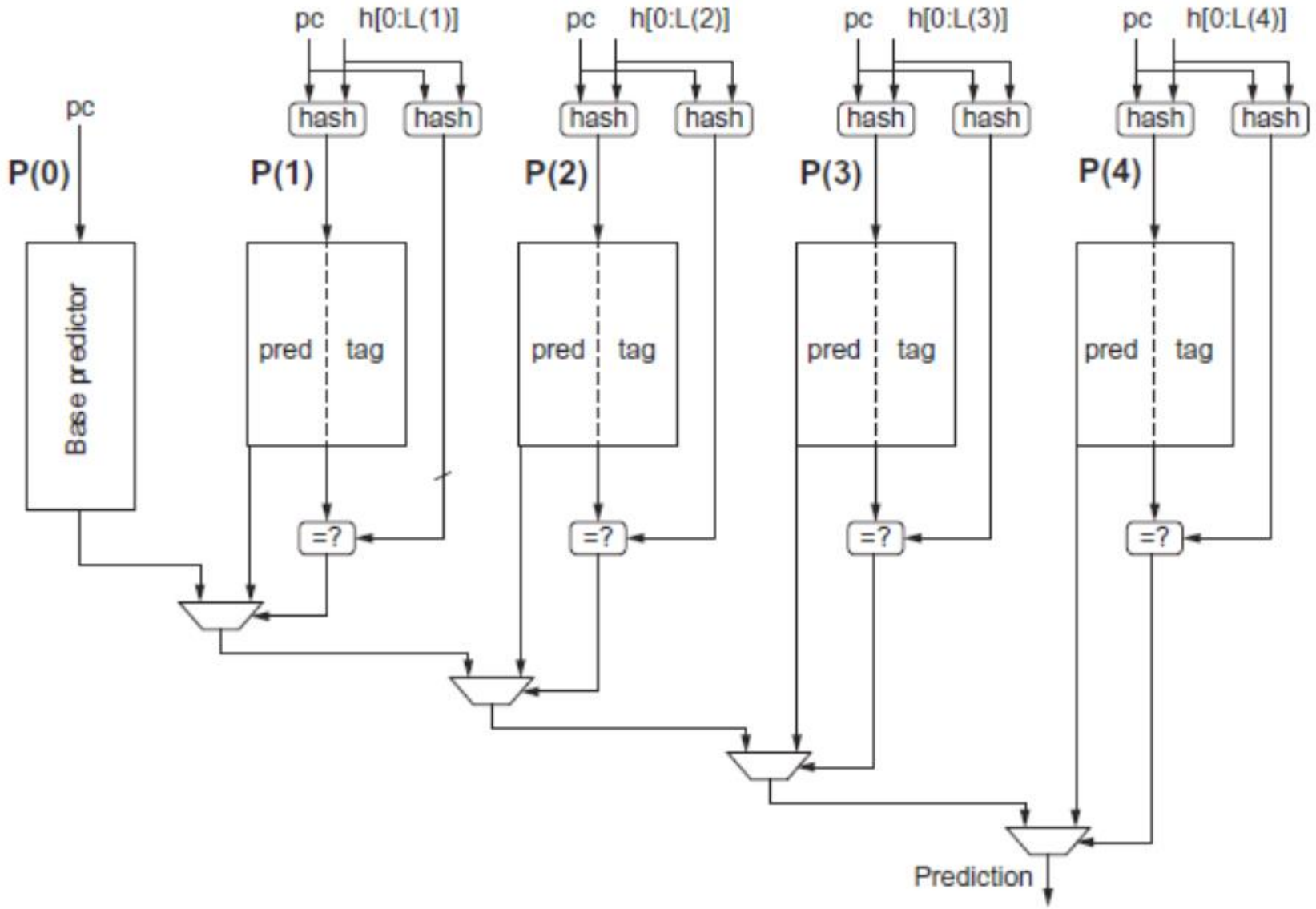
Tagged Hybrid Predictors

- Need to have predictor for each branch and history
  - Problem: this implies huge tables
  - Solution:
    - Use hash tables, whose hash value is based on branch address and branch history
    - Longer histories may lead to increased chance of hash collision, so use multiple tables with increasingly shorter histories

MORGAN KAUFMANN

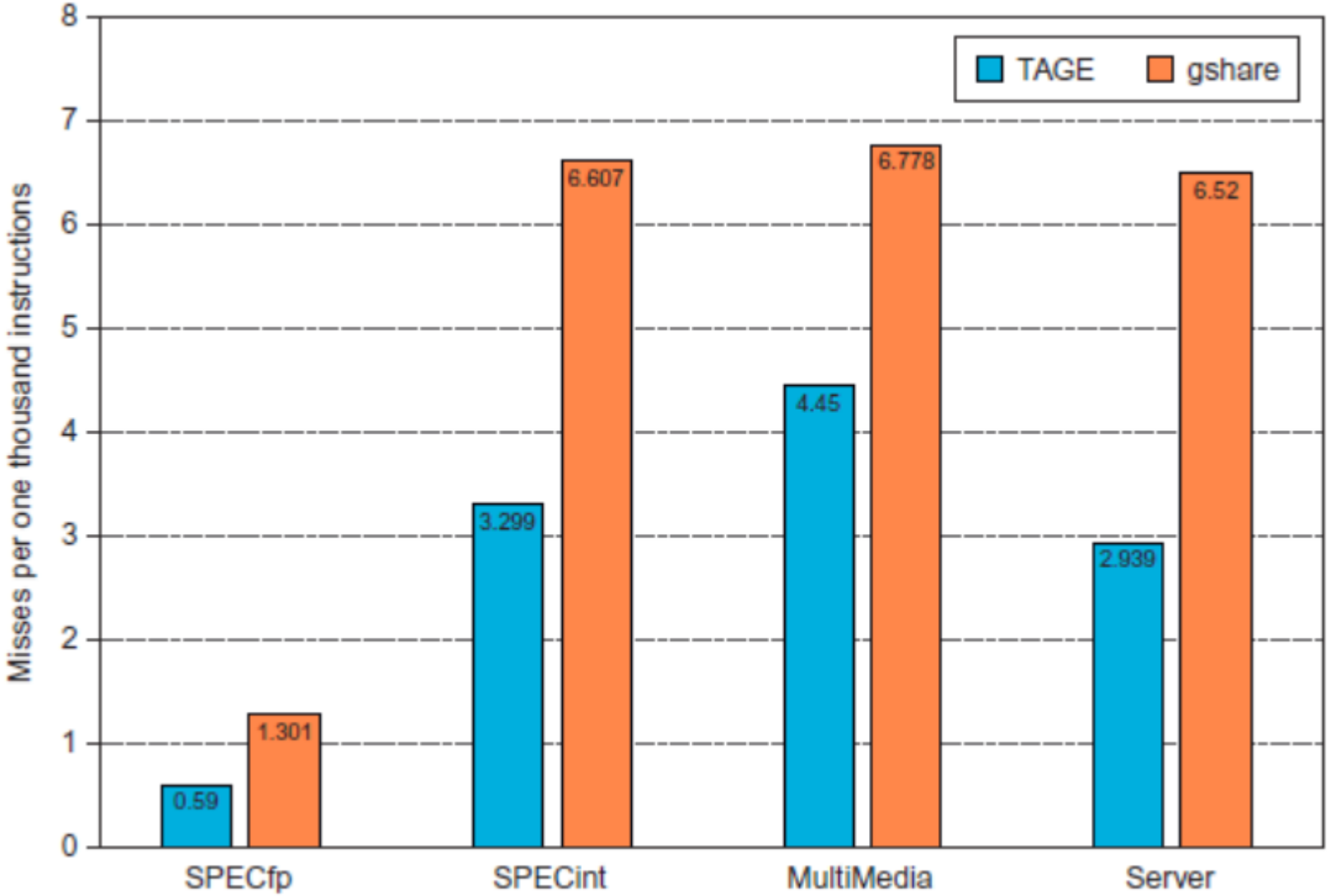
Copyright © 2019, Elsevier Inc. All rights Reserved

Tagged Hybrid Predictors



Copyright © 2019, Elsevier Inc. All rights Reserved

Tagged Hybrid Predictors



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Dynamic Scheduling

- Rearrange order of instructions to reduce stalls while maintaining data flow
- Advantages:
  - Compiler doesn't need to have knowledge of microarchitecture
  - Handles cases where dependencies are unknown at compile time
- Disadvantage:
  - Substantial increase in hardware complexity
  - Complicates exceptions

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Dynamic Scheduling

- Dynamic scheduling implies:
  - Out-of-order execution
  - Out-of-order completion

- Example 1:

  fdiv.d f0,f2,f4

  fadd.d f10,f0,f8

  fsub.d f12,f8,f14

  - fsub.d is not dependent, issue before fadd.d

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Dynamic Scheduling

- Example 2:

  fdiv.d f0,f2,f4

  fmul.d f6,f0,f8

  fadd.d f0,f10,f14

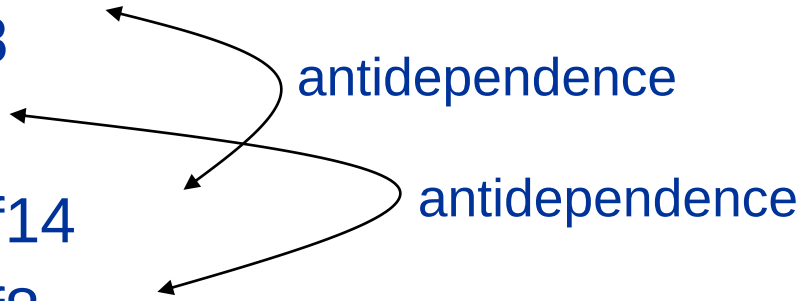
  - fadd.d is not dependent, but the antidependence makes it impossible to issue earlier without register renaming

Copyright © 2019, Elsevier Inc. All rights Reserved

Register Renaming

- Example 3:

```
fdiv.d f0,f2,f4
fadd.d f6,f0,f8
fsd f6,0(x1)
fsub.d f8,f10,f14
fmul.d f6,f10,f8
```



  - name dependence with f6

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Register Renaming

- Example 3:

```
fdiv.d f0,f2,f4
fadd.d S,f0,f8
fsd S,0(x1)
fsub.d T,f10,f14
fmul.d f6,f10,T
```

- Now only RAW hazards remain, which can be strictly ordered

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Register Renaming

- Tomasulo's Approach
  - Tracks when operands are available
  - Introduces register renaming in hardware
    - Minimizes WAW and WAR hazards
- Register renaming is provided by reservation stations (RS)
  - Contains:
    - The instruction
    - Buffered operand values (when available)
    - Reservation station number of instruction providing the operand values

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

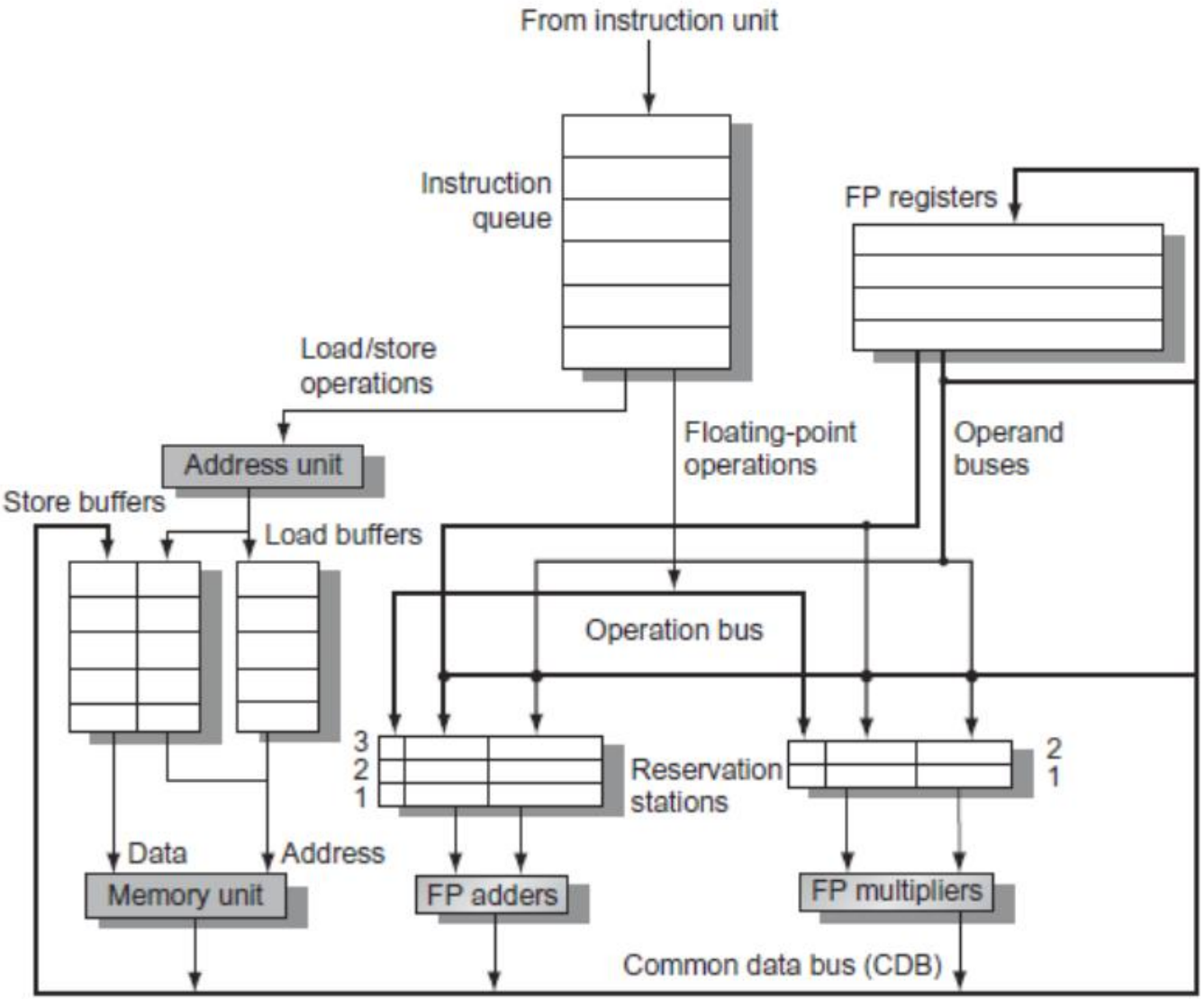
Register Renaming

- RS fetches and buffers an operand as soon as it becomes available (not necessarily involving register file)
- Pending instructions designate the RS to which they will send their output
  - Result values broadcast on a result bus, called the common data bus (CDB)
- Only the last output updates the register file
- As instructions are issued, the register specifiers are renamed with the reservation station
- May be more reservation stations than registers
- Load and store buffers
  - Contain data and addresses, act like reservation stations

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Tomasulo's Algorithm



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Tomasulo's Algorithm

- Three Steps:
  - Issue
    - Get next instruction from FIFO queue
    - If available RS, issue the instruction to the RS with operand values if available
    - If operand values not available, stall the instruction
  - Execute
    - When operand becomes available, store it in any reservation stations waiting for it
    - When all operands are ready, issue the instruction
    - Loads and store maintained in program order through effective address
    - No instruction allowed to initiate execution until all branches that proceed it in program order have completed
  - Write result
    - Write result on CDB into reservation stations and store buffers
      - (Stores must wait until address and value are received)

Copyright © 2019, Elsevier Inc. All rights Reserved

Example

		Instruction status		
Instruction		Issue	Execute	Write result
fld	f6,32(x2)	√	√	√
fld	f2,44(x3)	√	√	
fmul.d	f0,f2,f4	√		
fsub.d	f8,f2,f6	√		
fdiv.d	f0,f0,f6	√		
fadd.d	f6,f8,f2	√		

	Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

	Register status								
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Qi	Mult1	Load2		Add2	Add1	Mult2			

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Tomasulo's Algorithm

- Example loop:

```
Loop:   fld f0,0(x1)
fmul.d f4,f0,f2
fsd f4,0(x1)
addi x1,x1,8
bne x1,x2,Loop // branches if x16 != x2
```

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Tomasulo's Algorithm

		Instruction status			
Instruction		From iteration	Issue	Execute	Write result
fld	f0,0(x1)	1	√	√	
fmul.d	f4,f0,f2	1	√		
fsd	f4,0(x1)	1	√		
fld	f0,0(x1)	2	√	√	
fmul.d	f4,f0,f2	2	√		
fsd	f4,0(x1)	2	√		

	Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	Yes	Load					Regs[x1] + 0
Load2	Yes	Load					Regs[x1] – 8
Add1	No						
Add2	No						
Add3	No						
Mult1	Yes	MUL		Regs[f2]	Load1		
Mult2	Yes	MUL		Regs[f2]	Load2		
Store1	Yes	Store	Regs[x1]			Mult1	
Store2	Yes	Store	Regs[x1] – 8			Mult2	

	Register status								
Field	f0	f2	f4	f6	f8	f10	f12	…	f30
Qi	Load2		Mult2						

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Hardware-Based Speculation

- Execute instructions along predicted execution paths but only commit the results if prediction was correct
- Instruction commit: allowing an instruction to update the register file when instruction is no longer speculative
- Need an additional piece of hardware to prevent any irrevocable action until an instruction commits
  - I.e. updating state or taking an execution

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Reorder Buffer

- Reorder buffer – holds the result of instruction between completion and commit
- Four fields:
  - Instruction type: branch/store/register
  - Destination field: register number
  - Value field: output value
  - Ready field: completed execution?
- Modify reservation stations:
  - Operand source is now reorder buffer instead

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Reorder Buffer

- Issue:
  - Allocate RS and ROB, read available operands
- Execute:
  - Begin execution when operand values are available
- Write result:
  - Write result and ROB tag on CDB
- Commit:
  - When ROB reaches head of ROB, update register
  - When a mispredicted branch reaches head of ROB, discard all entries

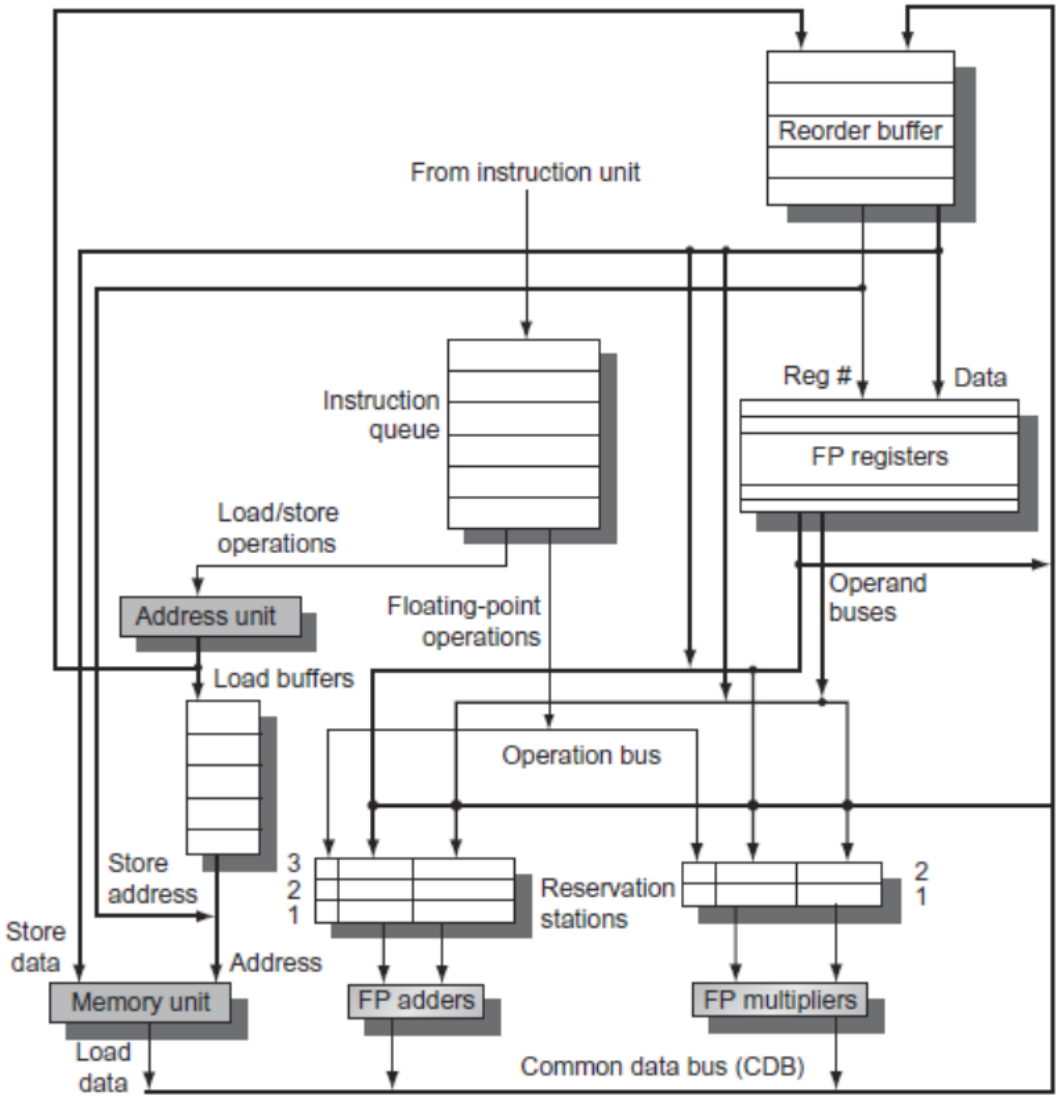
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Reorder Buffer

- Register values and memory values are not written until an instruction commits
- On misprediction:
  - Speculated entries in ROB are cleared
- Exceptions:
  - Not recognized until it is ready to commit

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Reorder Buffer



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Reorder Buffer

	Reorder buffer					
Entry	Busy	Instruction		State	Destination	Value
1	No	fld	f6,32(x2)	Commit	f6	Mem[32 + Regs[x2]]
2	No	fld	f2,44(x3)	Commit	f2	Mem[44 + Regs[x3]]
3	Yes	fmul.d	f0,f2,f4	Write result	f0	#2 × Regs[f4]
4	Yes	fsub.d	f8,f2,f6	Write result	f8	#2 – #1
5	Yes	fdiv.d	f0,f0,f6	Execute	f0	
6	Yes	fadd.d	f6,f8,f2	Write result	f6	#4 + #2

	Reservation stations							
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	fmul.d	Mem[44 + Regs[x3]]	Regs[f4]			#3	
Mult2	Yes	fdiv.d		Mem[32 + Regs[x2]]	#3		#5	

	FP register status									
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	…	Yes	Yes

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Multiple Issue and Static Scheduling

- To achieve CPI < 1, need to complete multiple instructions per clock

- Solutions:
  - Statically scheduled superscalar processors
  - VLIW (very long instruction word) processors
  - Dynamically scheduled superscalar processors

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Multiple Issue

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the Cortex-A53
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

VLIW Processors

- Package multiple operations into one instruction
- Example VLIW processor:
  - One integer instruction (or branch)
  - Two independent floating-point operations
  - Two independent memory references
- Must be enough parallelism in code to fill the available slots

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

VLIW Processors

Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
fld f0,0(x1)	fld f6,-8(x1)			
fld f10,-16(x1)	fld f14,-24(x1)			
fld f18,-32(x1)	fld f22,-40(x1)	fadd.d f4,f0,f2	fadd.d f8,f6,f2	
fld f26,-48(x1)		fadd.d f12,f0,f2	fadd.d f16,f14,f2	
		fadd.d f20,f18,f2	fadd.d f24,f22,f2	
fsd f4,0(x1)	fsd f8,-8(x1)	fadd.d f28,f26,f24		
fsd f12,-16(x1)	fsd f16,-24(x1)			addi x1,x1,-56
fsd f20,24(x1)	fsd f24,16(x1)			
fsd f28,8(x1)				bne x1,x2,Loop

- Disadvantages:
  - Statically finding parallelism
  - Code size
  - No hazard detection hardware
  - Binary code compatibility

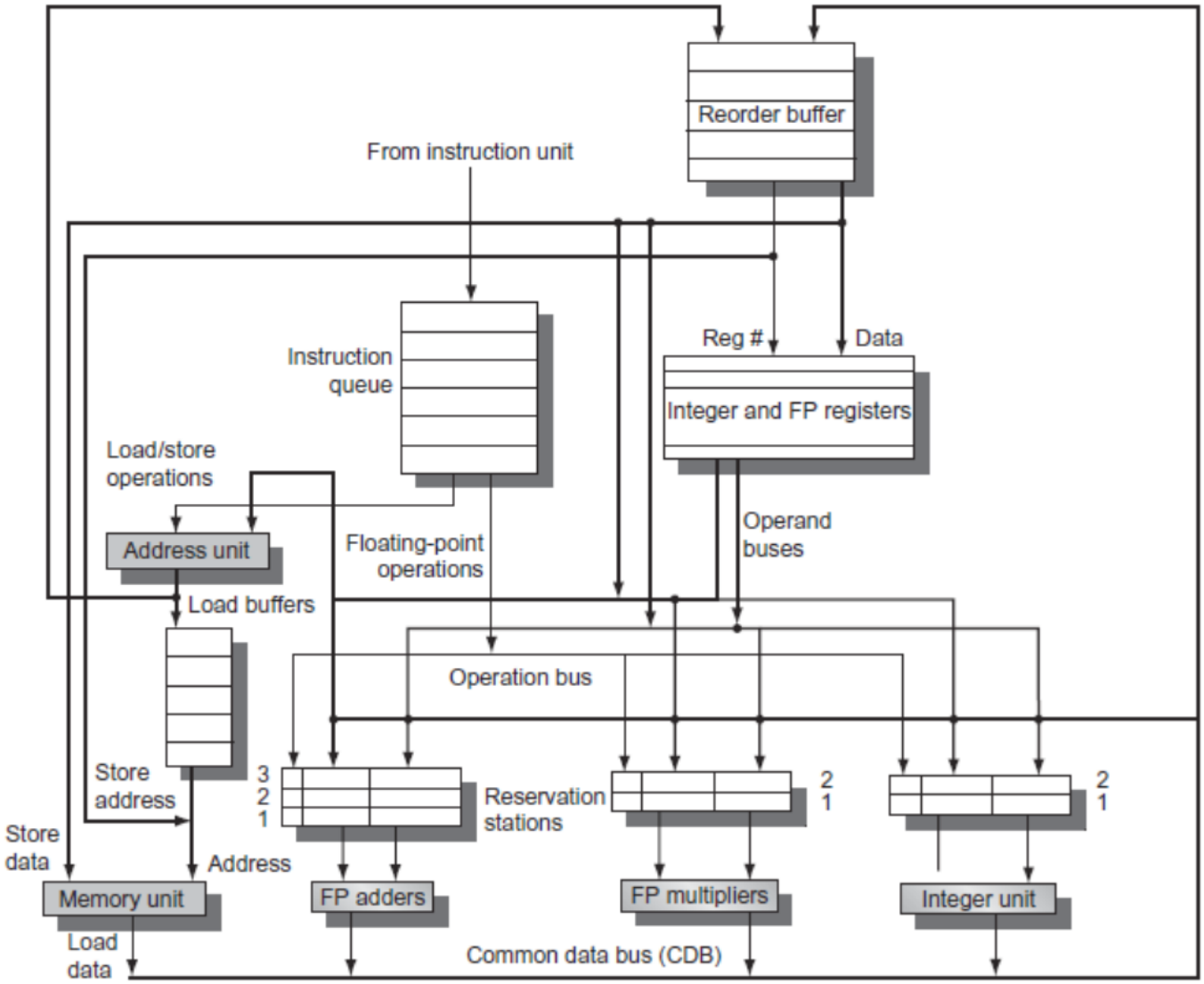
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Dynamic Scheduling, Multiple Issue, and Speculation

- Modern microarchitectures:
  - Dynamic scheduling + multiple issue + speculation
- Two approaches:
  - Assign reservation stations and update pipeline control table in half clock cycles
    - Only supports 2 instructions/clock
  - Design logic to handle any possible dependencies between the instructions
- Issue logic is the bottleneck in dynamically scheduled superscalars

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Overview of Design



Copyright © 2019, Elsevier Inc. All rights Reserved

Multiple Issue

- Examine all the dependencies amoung the instructions in the bundle
- If dependencies exist in bundle, encode them in reservation stations
- Also need multiple completion/commit
- To simplify RS allocation:
  - Limit the number of instructions of a given class that can be issued in a “bundle”, i.e. on FP, one integer, one load, one store

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Example

```
Loop:  ld x2,0(x1)          //x2=array element
addi x2,x2,1          //increment x2
sd x2,0(x1)           //store result
addi x1,x1,8          //increment pointer
bne x2,x3,Loop        //branch if not last
```


MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Example (No Speculation)

Iteration number	Instructions	Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	ld x2,0(x1)	1	2	3	4	First issue
1	addi x2,x2,1	1	5		6	Wait for ld
1	sd x2,0(x1)	2	3	7		Wait for addi
1	addi x1,x1,8	2	3		4	Execute directly
1	bne x2,x3,Loop	3	7			Wait for addi
2	ld x2,0(x1)	4	8	9	10	Wait for bne
2	addi x2,x2,1	4	11		12	Wait for ld
2	sd x2,0(x1)	5	9	13		Wait for addi
2	addi x1,x1,8	5	8		9	Wait for bne
2	bne x2,x3,Loop	6	13			Wait for addi
3	ld x2,0(x1)	7	14	15	16	Wait for bne
3	addi x2,x2,1	7	17		18	Wait for ld
3	sd x2,0(x1)	8	15	19		Wait for addi
3	addi x1,x1,8	8	14		15	Wait for bne
3	bne x2,x3,Loop	9	19			Wait for addi

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Example (Mutiple Issue with Speculation)

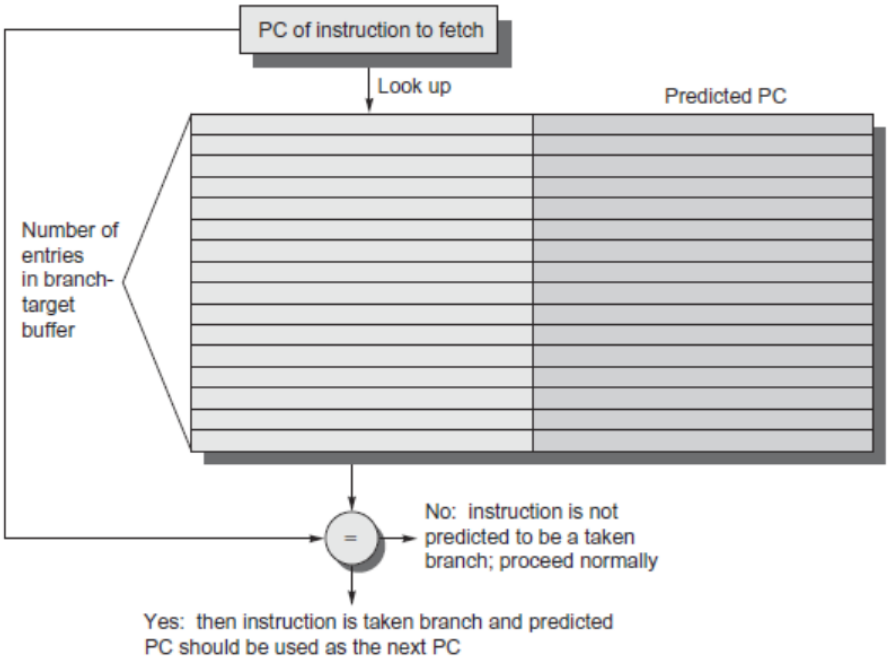
Iteration number	Instructions	Issues at clock number	Executes at clock number	Read access at clock number	Write CDB at clock number	Commits at clock number	Comment
1	ld x2,0(x1)	1	2	3	4	5	First issue
1	addi x2,x2,1	1	5		6	7	Wait for ld
1	sd x2,0(x1)	2	3			7	Wait for addi
1	addi x1,x1,8	2	3		4	8	Commit in order
1	bne x2,x3,Loop	3	7			8	Wait for addi
2	ld x2,0(x1)	4	5	6	7	9	No execute delay
2	addi x2,x2,1	4	8		9	10	Wait for ld
2	sd x2,0(x1)	5	6			10	Wait for addi
2	addi x1,x1,8	5	6		7	11	Commit in order
2	bne x2,x3,Loop	6	10			11	Wait for addi
3	ld x2,0(x1)	7	8	9	10	12	Earliest possible
3	addi x2,x2,1	7	11		12	13	Wait for ld
3	sd x2,0(x1)	8	9			13	Wait for addi
3	addi x1,x1,8	8	9		10	14	Executes earlier
3	bne x2,x3,Loop	9	13			14	Wait for addi

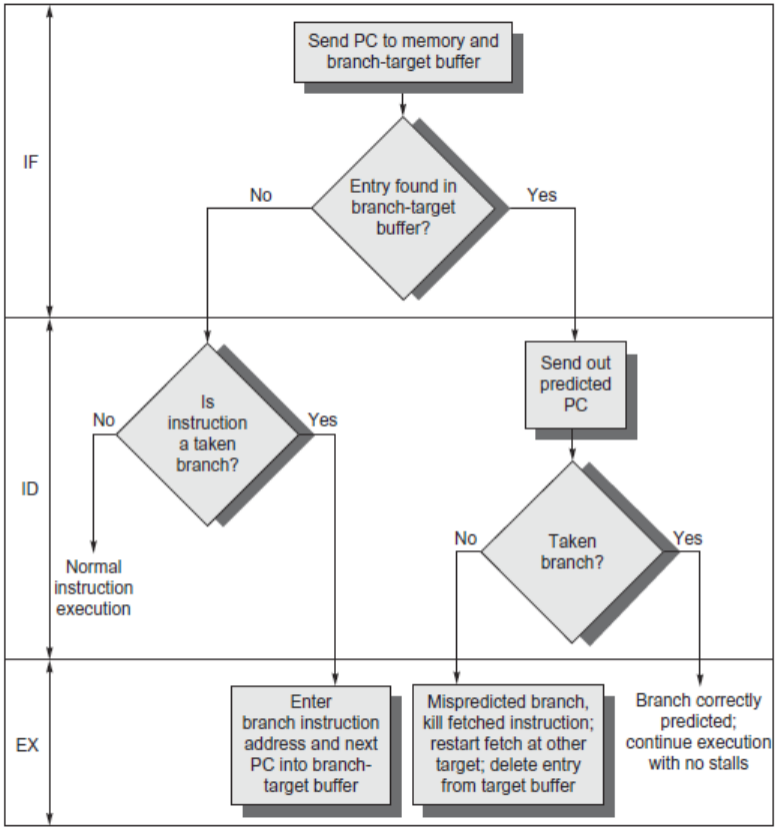
MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Branch-Target Buffer

- Need high instruction bandwidth
  - Branch-Target buffers
    - Next PC prediction buffer, indexed by current PC





MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Branch Folding

- Optimization:
  - Larger branch-target buffer
  - Add target instruction into buffer to deal with longer decoding time required by larger buffer
  - “Branch folding”

MORGAN KAUFMANN

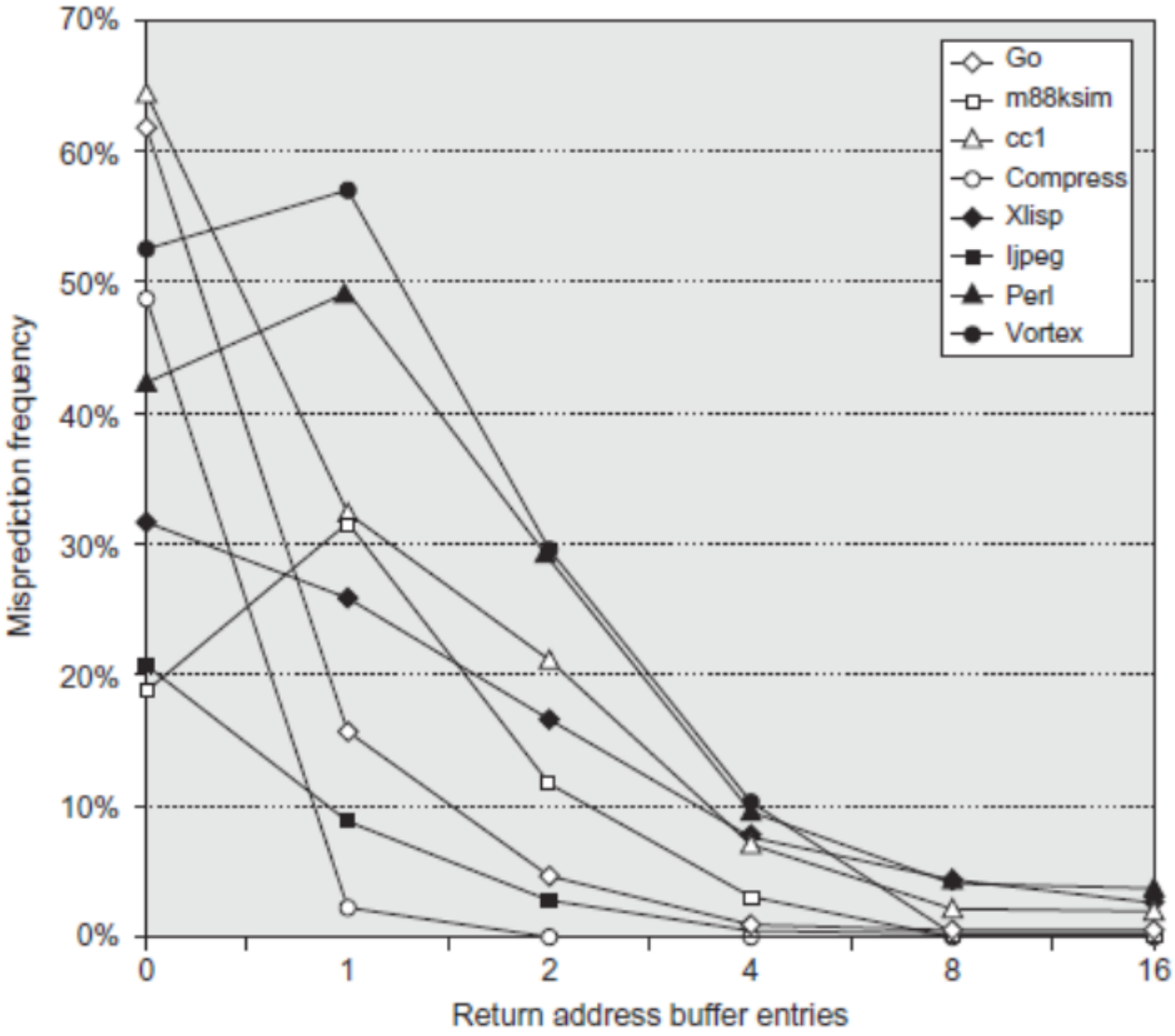
Copyright © 2019, Elsevier Inc. All rights Reserved

Return Address Predictor

- Most unconditional branches come from function returns
- The same procedure can be called from multiple sites
  - Causes the buffer to potentially forget about the return address from previous calls
- Create return address buffer organized as a stack

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Return Address Predictor



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Integrated Instruction Fetch Unit

- Design monolithic unit that performs:
  - Branch prediction
  - Instruction prefetch
    - Fetch ahead
  - Instruction memory access and buffering
    - Deal with crossing cache lines

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Register Renaming

- Register renaming vs. reorder buffers
  - Instead of virtual registers from reservation stations and reorder buffer, create a single register pool
    - Contains visible registers and virtual registers
  - Use hardware-based map to rename registers during issue
  - WAW and WAR hazards are avoided
  - Speculation recovery occurs by copying during commit
  - Still need a ROB-like queue to update table in order
  - Simplifies commit:
    - Record that mapping between architectural register and physical register is no longer speculative
    - Free up physical register used to hold older value
    - In other words: SWAP physical registers on commit
  - Physical register de-allocation is more difficult
    - Simple approach: deallocate virtual register when next instruction writes to its mapped architecturally-visibly register

Copyright © 2019, Elsevier Inc. All rights Reserved

Integrated Issue and Renaming

- Combining instruction issue with register renaming:
  - Issue logic pre-reserves enough physical registers for the bundle
  - Issue logic finds dependencies within bundle, maps registers as necessary
  - Issue logic finds dependencies between current bundle and already in-flight bundles, maps registers as necessary

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	add x1,x2,x3	p32	add p32,p2,p3	x1-> p32
2	sub x1,x1,x2	p33	sub p33,p32,p2	x1->p33
3	add x2,x1,x2	p34	add p34,p33,x2	x2->p34
4	sub x1,x3,x2	p35	sub p35,p3,p34	x1->p35
5	add x1,x1,x2	p36	add p36,p35,p34	x1->p36
6	sub x1,x3,x1	p37	sub p37,p3,p36	x1->p37

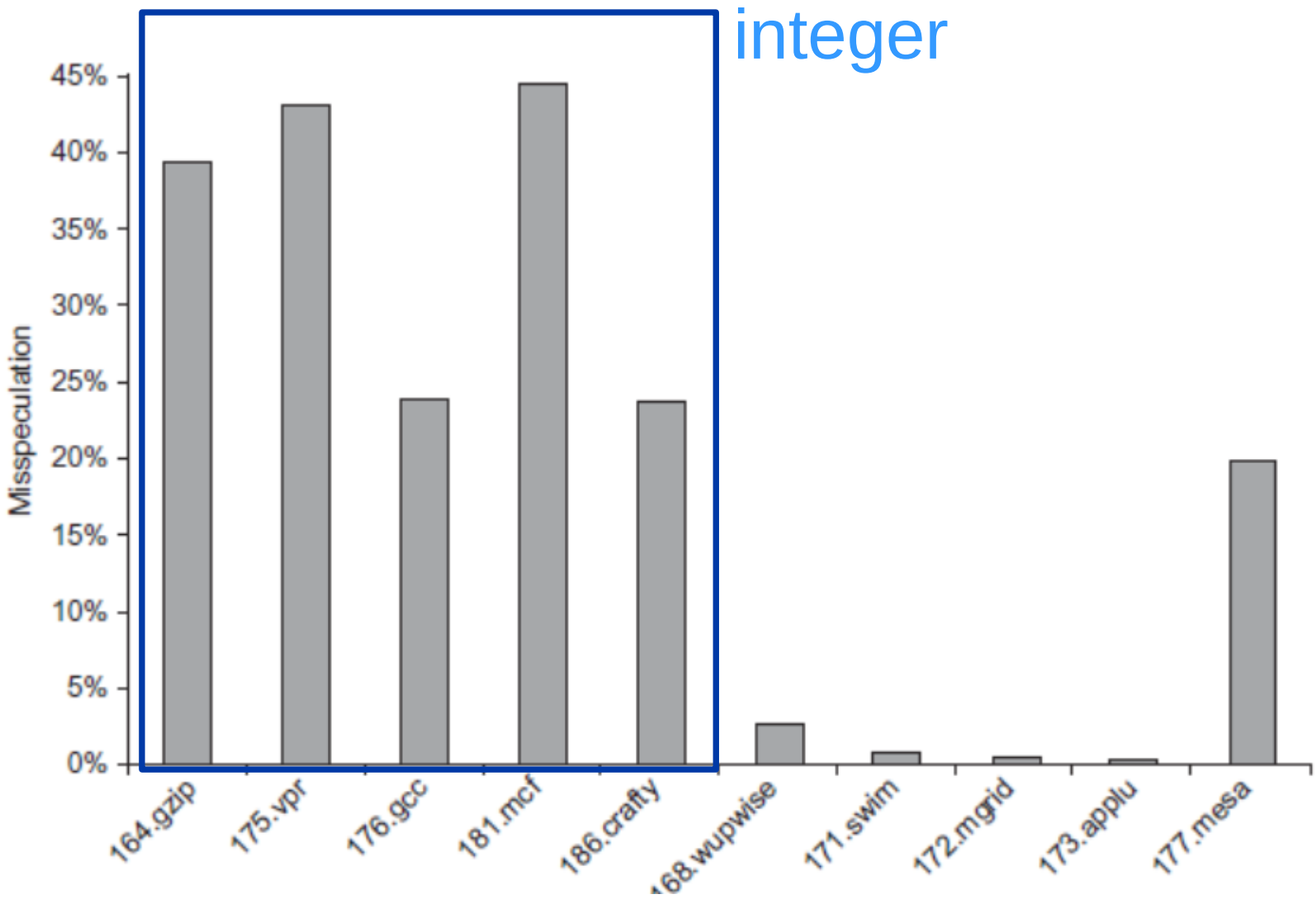
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

How Much?

- How much to speculate
  - Mis-speculation degrades performance and power relative to no speculation
    - May cause additional misses (cache, TLB)
  - Prevent speculative code from causing higher costing misses (e.g. L2)
- Speculating through multiple branches
  - Complicates speculation recovery
- Speculation and energy efficiency
  - Note: speculation is only energy efficient when it significantly improves performance

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

How Much?



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

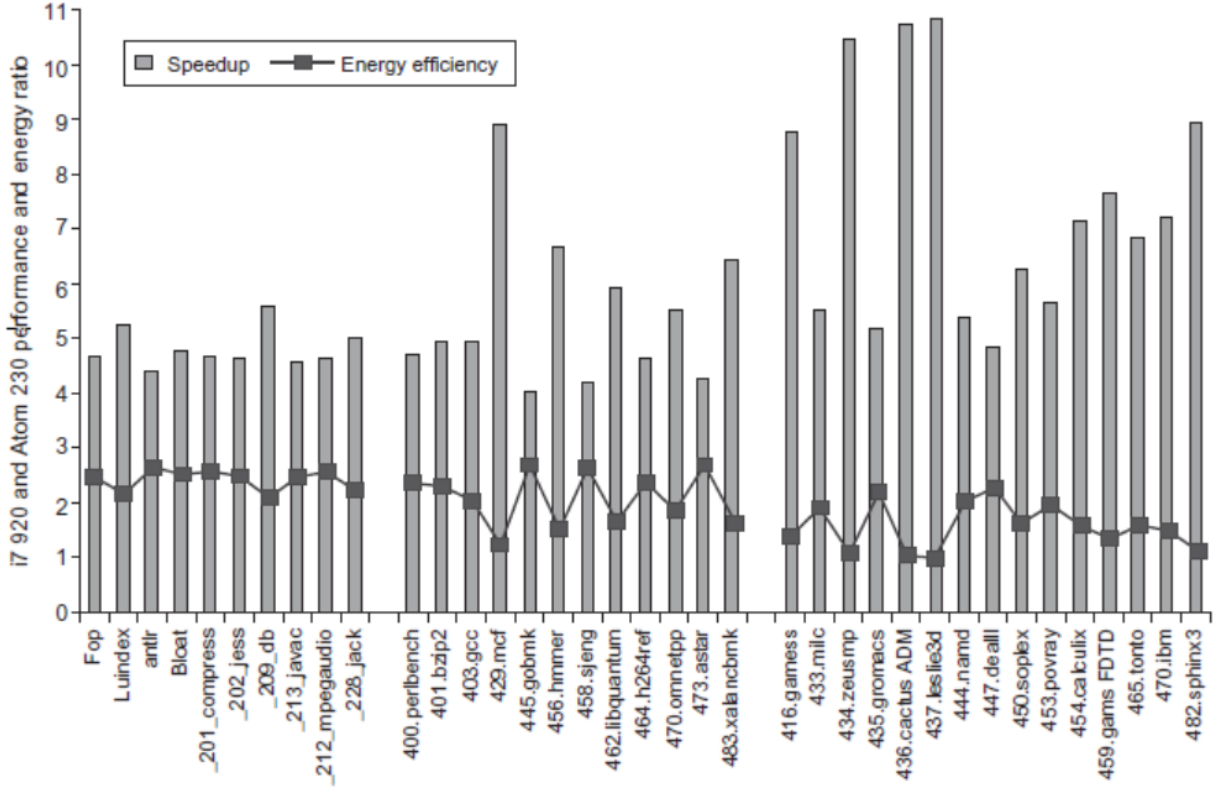
Energy Efficiency

- Value prediction
  - Uses:
    - Loads that load from a constant pool
    - Instruction that produces a value from a small set of values
  - Not incorporated into modern processors
  - Similar idea--*address aliasing prediction*--is used on some processors to determine if two stores or a load and a store reference the same address to allow for reordering

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- It is easy to predict the performance/energy efficiency of two different versions of the same ISA if we hold the technology constant



Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- Processors with lower CPIs / faster clock rates will also be faster

Processor	Implementation technology	Clock rate	Power	SPECCInt2006 base	SPECCFP2006 baseline
Intel Pentium 4 670	90 nm	3.8 GHz	115 W	11.5	12.2
Intel Itanium 2	90 nm	1.66 GHz	104 W approx. 70 W one core	14.5	17.3
Intel i7 920	45 nm	3.3 GHz	130 W total approx. 80 W one core	35.5	38.4

  - Pentium 4 had higher clock, lower CPI
  - Itanium had same CPI, lower clock

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

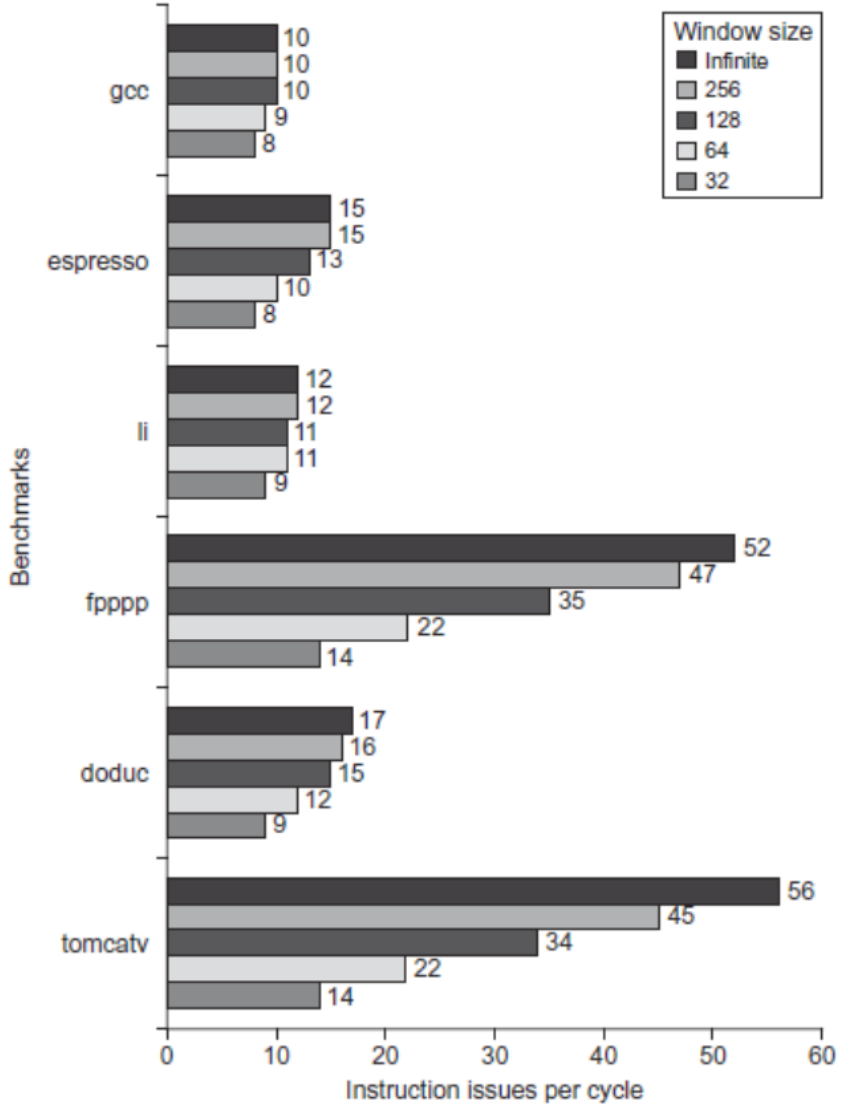
- Sometimes bigger and dumber is better
  - Pentium 4 and Itanium were advanced designs, but could not achieve their peak instruction throughput because of relatively small caches as compared to i7
- And sometimes smarter is better than bigger and dumber
  - TAGE branch predictor outperforms gshare with less stored predictions

MORGAN KAUFMANN

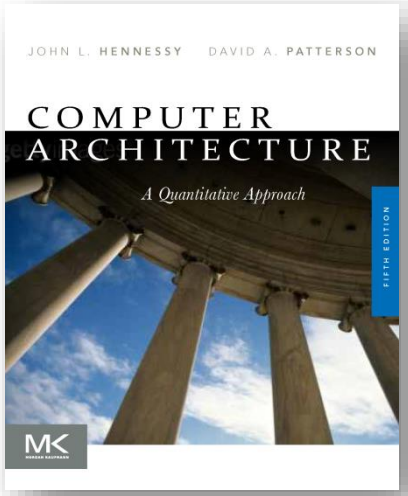
Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- Believing that there are large amounts of ILP available, if only we had the right techniques



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved



Chapter 4

Data-Level Parallelism in Vector, SIMD, and GPU Architectures

Copyright © 2019, Elsevier Inc. All rights Reserved

Introduction

- SIMD architectures can exploit significant data-level parallelism for:
  - Matrix-oriented scientific computing
  - Media-oriented image and sound processors
- SIMD is more energy efficient than MIMD
  - Only needs to fetch one instruction per data operation
  - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

Copyright © 2019, Elsevier Inc. All rights Reserved

SIMD Parallelism

- Vector architectures
- SIMD extensions
- Graphics Processor Units (GPUs)

- For x86 processors:
  - Expect two additional cores per chip per year
  - SIMD width to double every four years
  - Potential speedup from SIMD to be twice that from MIMD!

Copyright © 2019, Elsevier Inc. All rights Reserved

Vector Architectures

- Basic idea:
  - Read sets of data elements into “vector registers”
  - Operate on those registers
  - Disperse the results back into memory
- Registers are controlled by compiler
  - Used to hide memory latency
  - Leverage memory bandwidth

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

VMIPS

- Example architecture: RV64V
  - Loosely based on Cray-1
  - 32 62-bit vector registers
    - Register file has 16 read ports and 8 write ports
  - Vector functional units
    - Fully pipelined
    - Data and control hazards are detected
  - Vector load-store unit
    - Fully pipelined
    - One word per clock cycle after initial latency
  - Scalar registers
    - 31 general-purpose registers
    - 32 floating-point registers

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

VMIPS Instructions

- .vv: two vector operands
- .vs and .sv: vector and scalar operands
- LV/SV: vector load and vector store from address
- Example: DAXPY

```
vsetdcfg   4*FP64      # Enable 4 DP FP vregs
fld        f0,a        # Load scalar a
vld        v0,x5       # Load vector X
vmul       v1,v0,f0    # Vector-scalar mult
vld        v2,x6       # Load vector Y
vadd       v3,v1,v2    # Vector-vector add
vst        v3,x6       # Store the sum
vdisable               # Disable vector regs
```

- 8 instructions, 258 for RV64V (scalar code)

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Vector Execution Time

- Execution time depends on three factors:
  - Length of operand vectors
  - Structural hazards
  - Data dependencies
- RV64V functional units consume one element per clock cycle
  - Execution time is approximately the vector length
- *Convey*
  - Set of vector instructions that could potentially execute together

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Chimes

- Sequences with read-after-write dependency hazards placed in same convey via *chaining*
- *Chaining*
  - Allows a vector operation to start as soon as the individual elements of its vector source operand become available
- *Chime*
  - Unit of time to execute one convey
  - *m* conveys executes in *m* chimes for vector length *n*
  - For vector length of *n*, requires *m* x *n* clock cycles

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Example

```
vld     v0,x5       # Load vector X
vmul    v1,v0,f0    # Vector-scalar multiply
vld     v2,x6       # Load vector Y
vadd    v3,v1,v2    # Vector-vector add
vst     v3,x6       # Store the sum
```

Convoys:

1	vld	vmul
2	vld	vadd
3	vst	

3 chimes, 2 FP ops per result, cycles per FLOP = 1.5

For 64 element vectors, requires 32 x 3 = 96 clock cycles

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Challenges

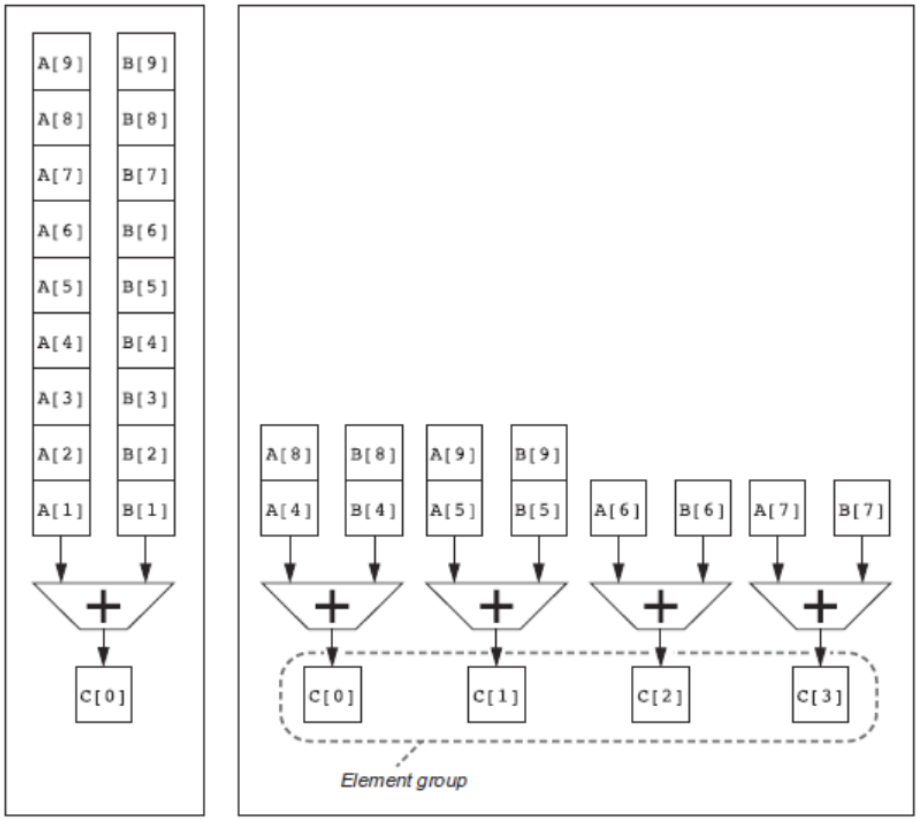
- Start up time
  - Latency of vector functional unit
  - Assume the same as Cray-1
    - Floating-point add => 6 clock cycles
    - Floating-point multiply => 7 clock cycles
    - Floating-point divide => 20 clock cycles
    - Vector load => 12 clock cycles
- Improvements:
  - > 1 element per clock cycle
  - Non-64 wide vectors
  - IF statements in vector code
  - Memory system optimizations to support vector processors
  - Multiple dimensional matrices
  - Sparse matrices
  - Programming a vector computer

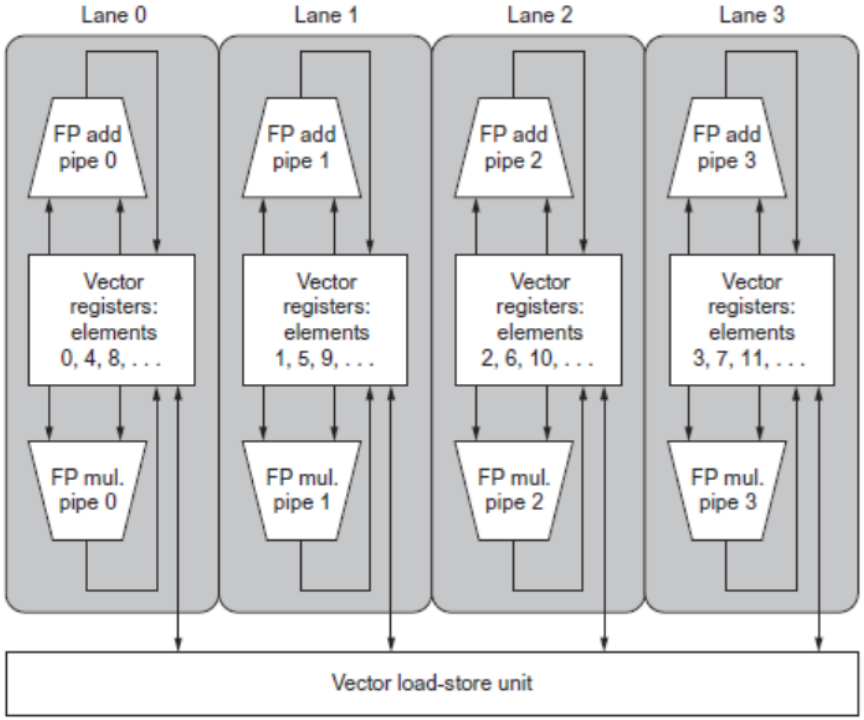
MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Multiple Lanes

- Element *n* of vector register *A* is “hardwired” to element *n* of vector register *B*
  - Allows for multiple hardware lanes





MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Vector Length Register

```
for (i=0; i <n; i=i+1) Y[i] = a * X[i] + Y[i];
        vsetdcfg 2 DP FP # Enable 2 64b Fl.Pt. registers
        fld f0,a         # Load scalar a
loop:   setvl t0,a0      # vl = t0 = min(mvl,n)
        vld v0,x5        # Load vector X
        slli t1,t0,3     # t1 = vl * 8 (in bytes)
        add x5,x5,t1     # Increment pointer to X by vl*8
        vmul v0,v0,f0    # Vector-scalar mult
        vld v1,x6        # Load vector Y
        vadd v1,v0,v1    # Vector-vector add
        sub a0,a0,t0     # n -= vl (t0)
        vst v1,x6        # Store the sum into Y
        add x6,x6,t1     # Increment pointer to Y by vl*8
        bnez a0,loop     # Repeat if n != 0
        vdisable         # Disable vector regs}
```

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Vector Mask Registers

- Consider:

```
for (i = 0; i < 64; i=i+1)
        if (X[i] != 0)
                X[i] = X[i] – Y[i];
```

- Use predicate register to “disable” elements:

```
vsetdcfg        2*FP64      # Enable 2 64b FP vector regs
vsetpcfgi       1           # Enable 1 predicate register
vld             v0,x5       # Load vector X into v0
vld             v1,x6       # Load vector Y into v1
fmv.d.x         f0,x0       # Put (FP) zero into f0
vpne            p0,v0,f0    # Set p0(i) to 1 if v0(i)!=f0
vsub            v0,v0,v1    # Subtract under vector mask
vst             v0,x5       # Store the result in X
vdisable                    # Disable vector registers
vpdisable                   # Disable predicate registers
```

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Memory Banks

- Memory system must be designed to support high bandwidth for vector loads and stores
- Spread accesses across multiple banks
  - Control bank addresses independently
  - Load or store non sequential words (need independent bank addressing)
  - Support multiple vector processors sharing the same memory
- Example:
  - 32 processors, each generating 4 loads and 2 stores/cycle
  - Processor cycle time is 2.167 ns, SRAM cycle time is 15 ns
  - How many memory banks needed?
    - 32x(4+2)x15/2.167 = ~1330 banks

Copyright © 2019, Elsevier Inc. All rights Reserved

Stride

- Consider:

```
for (i = 0; i < 100; i=i+1)
        for (j = 0; j < 100; j=j+1) {
                A[i][j] = 0.0;
                for (k = 0; k < 100; k=k+1)
                A[i][j] = A[i][j] + B[i][k] * D[k][j];
        }
```

- Must vectorize multiplication of rows of B with columns of D
- Use *non-unit stride*
- Bank conflict (stall) occurs when the same bank is hit faster than bank busy time:
  - #banks / LCM(stride,#banks) < bank busy time

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Scatter-Gather

- Consider:

```
for (i = 0; i < n; i=i+1)
        A[K[i]] = A[K[i]] + C[M[i]];
```

- Use index vector:

```
vsetdcfg        4*FP64          # 4 64b FP vector registers
vld             v0, x7          # Load K[]
vldx            v1, x5, v0      # Load A[K[]]
vld             v2, x28         # Load M[]
vldi            v3, x6, v2      # Load C[M[]]
vadd            v1, v1, v3      # Add them
vstx            v1, x5, v0      # Store A[K[]]
vdisable                        # Disable vector registers
```

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Programming Vec. Architectures

- Compilers can provide feedback to programmers
- Programmers can provide hints to compiler

Benchmark name	Operations executed in vector mode, compiler-optimized	Operations executed in vector mode, with programmer aid	Speedup from hint optimization
BDNA	96.1%	97.2%	1.52
MG3D	95.1%	94.5%	1.00
FLO52	91.5%	88.7%	N/A
ARC3D	91.1%	92.0%	1.01
SPEC77	90.3%	90.4%	1.07
MDG	87.7%	94.2%	1.49
TRFD	69.8%	73.7%	1.67
DYFESM	68.8%	65.6%	N/A
ADM	42.9%	59.6%	3.60
OCEAN	42.8%	91.2%	3.92
TRACK	14.4%	54.6%	2.52
SPICE	11.5%	79.9%	4.06
QCD	4.2%	75.1%	2.15

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

SIMD Extensions

- Media applications operate on data types narrower than the native word size
  - Example: disconnect carry chains to “partition” adder
- Limitations, compared to vector instructions:
  - Number of data operands encoded into op code
  - No sophisticated addressing modes (strided, scatter-gather)
  - No mask registers

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

SIMD Implementations

- Implementations:
  - Intel MMX (1996)
    - Eight 8-bit integer ops or four 16-bit integer ops
  - Streaming SIMD Extensions (SSE) (1999)
    - Eight 16-bit integer ops
    - Four 32-bit integer/fp ops or two 64-bit integer/fp ops
  - Advanced Vector Extensions (2010)
    - Four 64-bit integer/fp ops
  - AVX-512 (2017)
    - Eight 64-bit integer/fp ops
  - Operands must be consecutive and aligned memory locations

Copyright © 2019, Elsevier Inc. All rights Reserved

Example SIMD Code

- Example DXPY:

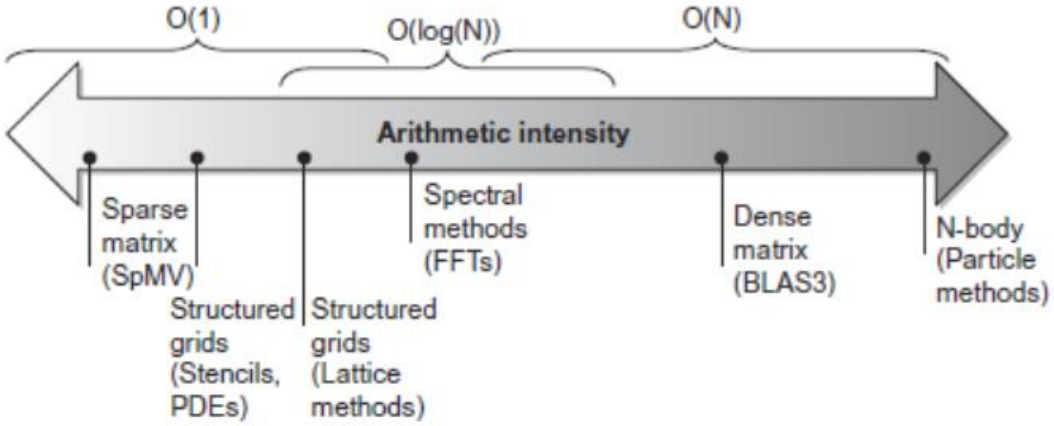
```
        fld       f0,a          # Load scalar a
        splat.4D  f0,f0         # Make 4 copies of a
        addi      x28,x5,#256   # Last address to load
Loop:   fld.4D    f1,0(x5)      # Load X[i] ... X[i+3]
        fmul.4D   f1,f1,f0      # a x X[i] ... a x X[i+3]
        fld.4D    f2,0(x6)      # Load Y[i] ... Y[i+3]
        fadd.4D   f2,f2,f1      # a x X[i]+Y[i]...
                                # a x X[i+3]+Y[i+3]
        fsd.4D    f2,0(x6)      # Store Y[i]... Y[i+3]
        addi      x5,x5,#32     # Increment index to X
        addi      x6,x6,#32     # Increment index to Y
        bne       x28,x5,Loop   # Check if done
```


MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Roofline Performance Model

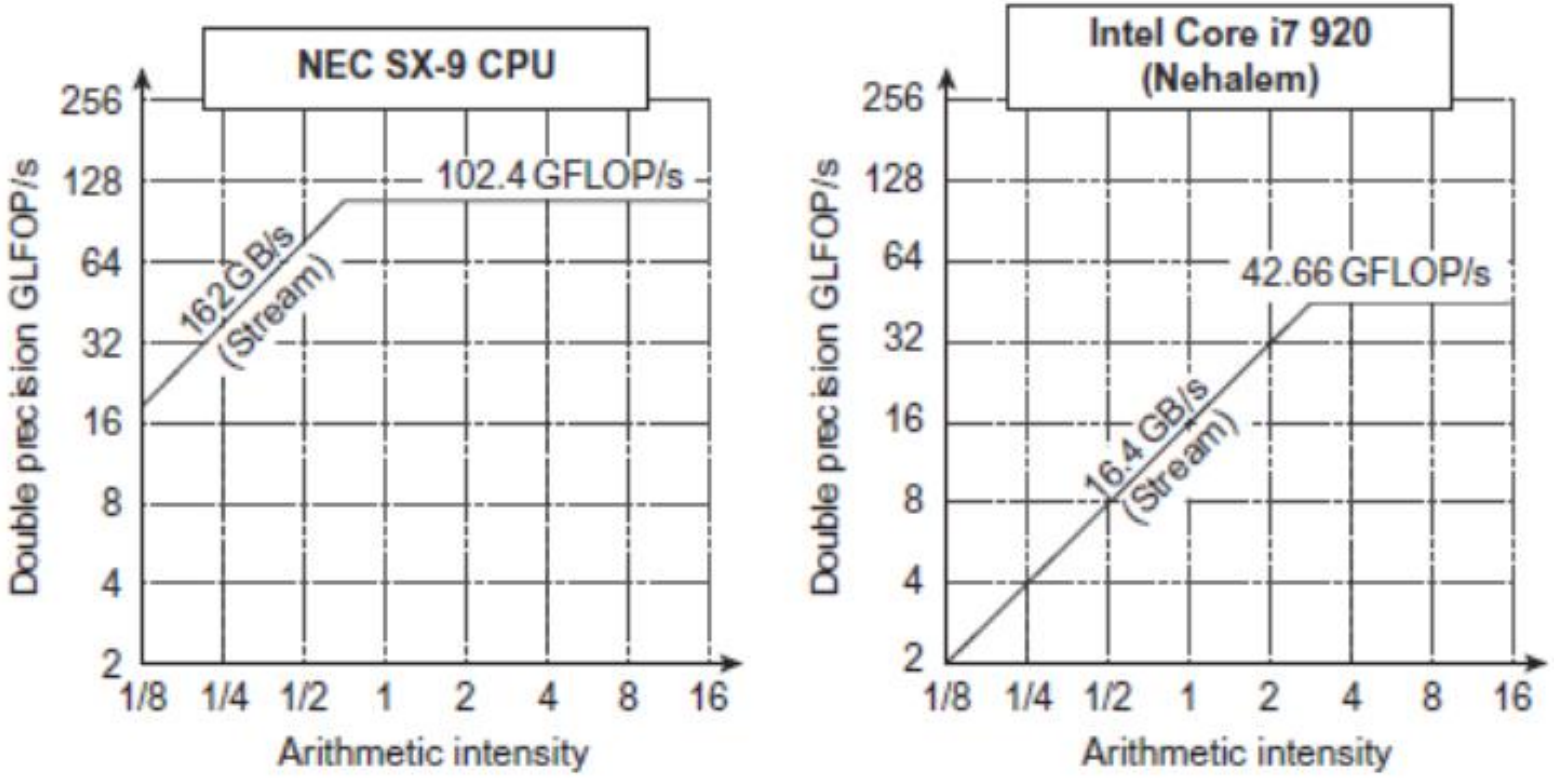
- Basic idea:
  - Plot peak floating-point throughput as a function of arithmetic intensity
  - Ties together floating-point performance and memory performance for a target machine
- Arithmetic intensity
  - Floating-point operations per byte read



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Examples

- Attainable GFLOPs/sec = (Peak Memory BW × Arithmetic Intensity, Peak Floating Point Perf.)



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Graphical Processing Units

- Basic idea:
  - Heterogeneous execution model
    - CPU is the *host*, GPU is the *device*
  - Develop a C-like programming language for GPU
  - Unify all forms of GPU parallelism as *CUDA thread*
  - Programming model is “Single Instruction Multiple Thread”

Copyright © 2019, Elsevier Inc. All rights Reserved

Threads and Blocks

- A thread is associated with each data element
- Threads are organized into blocks
- Blocks are organized into a grid

- GPU hardware handles thread management, not applications or OS

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

NVIDIA GPU Architecture

- Similarities to vector machines:
  - Works well with data-level parallel problems
  - Scatter-gather transfers
  - Mask registers
  - Large register files
- Differences:
  - No scalar processor
  - Uses multithreading to hide memory latency
  - Has many functional units, as opposed to a few deeply pipelined units like a vector processor

Copyright © 2019, Elsevier Inc. All rights Reserved

Example

- Code that works over all elements is the grid
- Thread blocks break this down into manageable sizes
  - 512 threads per block
- SIMD instruction executes 32 elements at a time
- Thus grid size = 16 blocks
- Block is analogous to a strip-mined vector loop with vector length of 32
- Block is assigned to a multithreaded SIMD processor by the thread block scheduler
- Current-generation GPUs have 7-15 multithreaded SIMD processors

MORGAN KAUFMANN

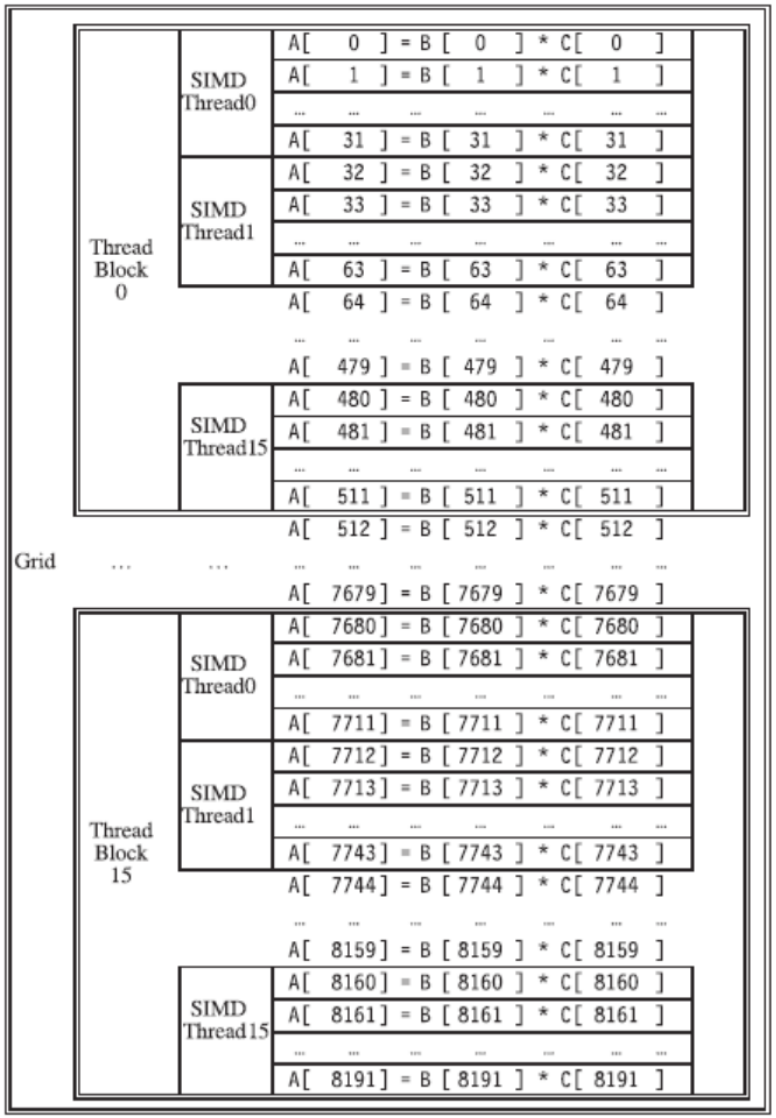
Copyright © 2019, Elsevier Inc. All rights Reserved

Terminology

- Each thread is limited to 64 registers
- Groups of 32 threads combined into a SIMD thread or “warp”
  - Mapped to 16 physical lanes
- Up to 32 warps are scheduled on a single SIMD processor
  - Each warp has its own PC
  - Thread scheduler uses scoreboard to dispatch warps
  - By definition, no data dependencies between warps
  - Dispatch warps into pipeline, hide memory latency
- Thread block scheduler schedules blocks to SIMD processors
- Within each SIMD processor:
  - 32 SIMD lanes
  - Wide and shallow compared to vector processors

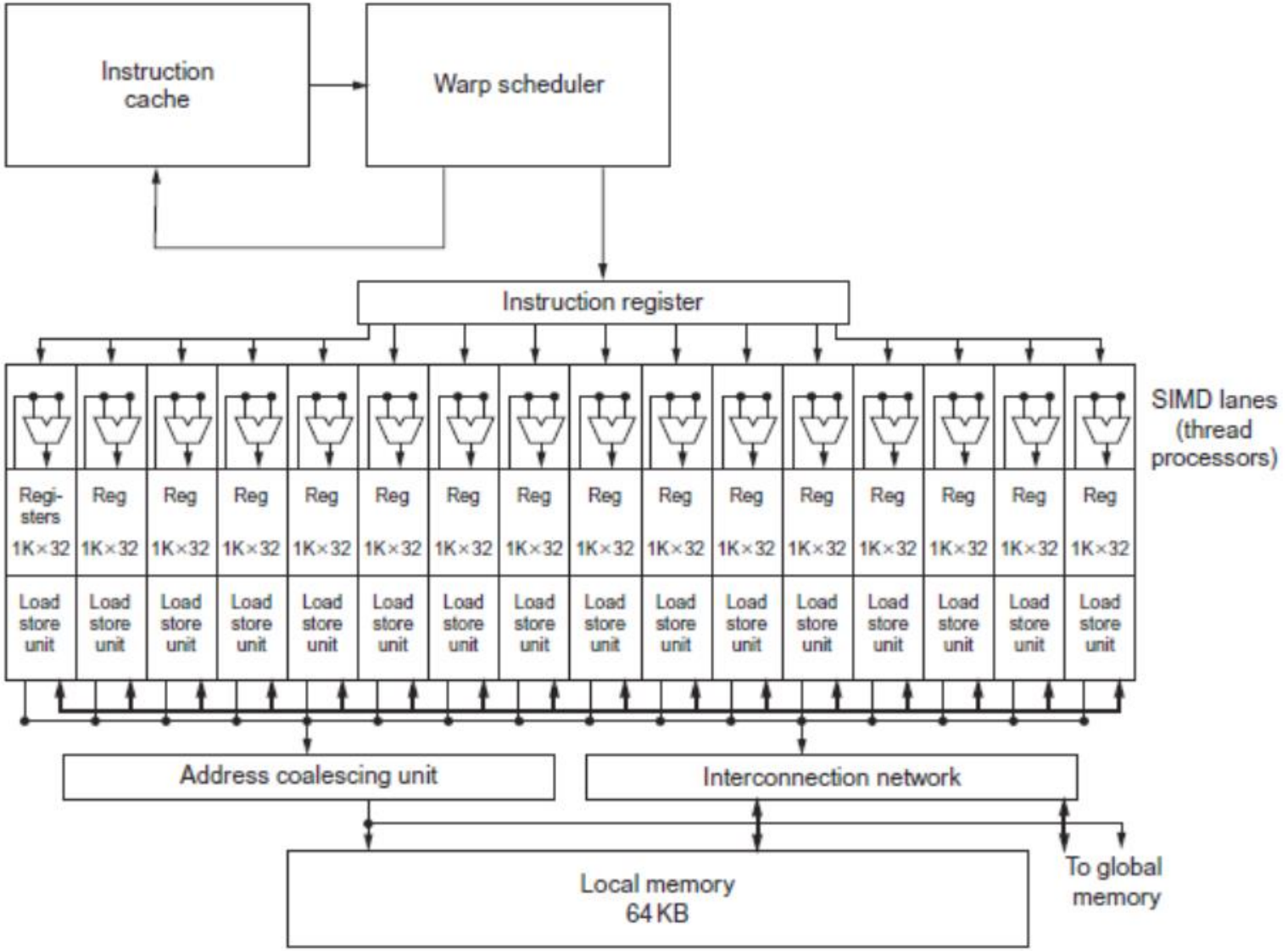
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Example



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

GPU Organization



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

NVIDIA Instruction Set Arch.

- ISA is an abstraction of the hardware instruction set
  - "Parallel Thread Execution (PTX)"
    - opcode.type d,a,b,c;
  - Uses virtual registers
  - Translation to machine code is performed in software
  - Example:

```
shl.s32         R8, blockIdx, 9      ; Thread Block ID * Block size (512 or 29)
add.s32         R8, R8, threadIdx    ; R8 = i = my CUDA thread ID
ld.global.f64   RD0, [X+R8]          ; RD0 = X[i]
ld.global.f64   RD2, [Y+R8]          ; RD2 = Y[i]
mul.f64 R0D, RD0, RD4  ; Product in RD0 = RD0 * RD4 (scalar a)
add.f64 R0D, RD0, RD2  ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0            ; Y[i] = sum (X[i]*a + Y[i])
```


MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Conditional Branching

- Like vector architectures, GPU branch hardware uses internal masks
- Also uses
  - Branch synchronization stack
    - Entries consist of masks for each SIMD lane
    - I.e. which threads commit their results (all threads execute)
  - Instruction markers to manage when a branch diverges into multiple execution paths
    - Push on divergent branch
  - …and when paths converge
    - Act as barriers
    - Pops stack
- Per-thread-lane 1-bit predicate register, specified by programmer

Copyright © 2019, Elsevier Inc. All rights Reserved

Example

```
    if (X[i] != 0)
            X[i] = X[i] – Y[i];
    else X[i] = Z[i];

    ld.global.f64     RD0, [X+R8]              ; RD0 = X[i]
    setp.neq.s32      P1, RD0, #0              ; P1 is predicate register 1
    @!P1, bra         ELSE1, *Push             ; Push old mask, set new mask bits
                                               ; if P1 false, go to ELSE1
    ld.global.f64     RD2, [Y+R8]              ; RD2 = Y[i]
    sub.f64           RD0, RD0, RD2            ; Difference in RD0
    st.global.f64     [X+R8], RD0              ; X[i] = RD0
    @P1, bra          ENDIF1, *Comp            ; complement mask bits
                                               ; if P1 true, go to ENDIF1
ELSE1:              ld.global.f64 RD0, [Z+R8]  ; RD0 = Z[i]
                    st.global.f64 [X+R8], RD0  ; X[i] = RD0
ENDIF1:  <next instruction>, *Pop              ; pop to restore old mask
```


MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

NVIDIA GPU Memory Structures

- Each SIMD Lane has private section of off-chip DRAM
  - “Private memory”
  - Contains stack frame, spilling registers, and private variables
- Each multithreaded SIMD processor also has local memory
  - Shared by SIMD lanes / threads within a block
- Memory shared by SIMD processors is GPU Memory
  - Host can read and write GPU memory

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Pascal Architecture Innovations

- Each SIMD processor has
  - Two or four SIMD thread schedulers, two instruction dispatch units
  - 16 SIMD lanes (SIMD width=32, chime=2 cycles), 16 load-store units, 4 special function units
  - Two threads of SIMD instructions are scheduled every two clock cycles
- Fast single-, double-, and half-precision
- High Bandwith Memory 2 (HBM2) at 732 GB/s
- NVLink between multiple GPUs (20 GB/s in each direction)
- Unified virtual memory and paging support

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Pascal Multithreaded SIMD Proc.



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Vector Architectures vs GPUs

- SIMD processor analogous to vector processor, both have MIMD
- Registers
  - RV64V register file holds entire vectors
  - GPU distributes vectors across the registers of SIMD lanes
  - RV64 has 32 vector registers of 32 elements (1024)
  - GPU has 256 registers with 32 elements each (8K)
  - RV64 has 2 to 8 lanes with vector length of 32, chime is 4 to 16 cycles
  - SIMD processor chime is 2 to 4 cycles
  - GPU vectorized loop is grid
  - All GPU loads are gather instructions and all GPU stores are scatter instructions

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

SIMD Architectures vs GPUs

- GPUs have more SIMD lanes
- GPUs have hardware support for more threads
- Both have 2:1 ratio between double- and single-precision performance
- Both have 64-bit addresses, but GPUs have smaller memory
- SIMD architectures have no scatter-gather support

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Loop-Level Parallelism

- Focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iterations
  - Loop-carried dependence

- Example 1:

```
for (i=999; i>=0; i=i-1)
        x[i] = x[i] + s;
```

- No loop-carried dependence

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Loop-Level Parallelism

- Example 2:

```
for (i=0; i<100; i=i+1) {
        A[i+1] = A[i] + C[i]; /* S1 */
        B[i+1] = B[i] + A[i+1]; /* S2 */
}
```

- S1 and S2 use values computed by S1 in previous iteration
- S2 uses value computed by S1 in same iteration

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Loop-Level Parallelism

- Example 3:

```
for (i=0; i<100; i=i+1) {
        A[i] = A[i] + B[i]; /* S1 */
        B[i+1] = C[i] + D[i]; /* S2 */
}
```

- S1 uses value computed by S2 in previous iteration but dependence is not circular so loop is parallel
- Transform to:

```
A[0] = A[0] + B[0];
for (i=0; i<99; i=i+1) {
        B[i+1] = C[i] + D[i];
        A[i+1] = A[i+1] + B[i+1];
}
B[100] = C[99] + D[99];
```


MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Loop-Level Parallelism

- Example 4:

```
for (i=0;i<100;i=i+1)  {
        A[i] = B[i] + C[i];
        D[i] = A[i] * E[i];
}
```

- Example 5:

```
for (i=1;i<100;i=i+1)  {
        Y[i] = Y[i-1] + Y[i];
}
```


MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Finding dependencies

- Assume indices are affine:
  - a x i + b (i is loop index)
- Assume:
  - Store to a x i + b, then
  - Load from c x i + d
  - i runs from m to n
  - Dependence exists if:
    - Given j, k such that $m \leq j \leq n$, $m \leq k \leq n$
    - Store to a x j + b, load from a x k + d, and $a \times j + b = c \times k + d$

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Finding dependencies

- Generally cannot determine at compile time
- Test for absence of a dependence:
  - GCD test:
    - If a dependency exists, GCD(*c*,*a*) must evenly divide (*d*-*b*)
- Example:

```
for (i=0; i<100; i=i+1) {
    X[2*i+3] = X[2*i] * 5.0;
}
```


MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Finding dependencies

- Example 2:

```
for (i=0; i<100; i=i+1) {
    Y[i] = X[i] / c; /* S1 */
    X[i] = X[i] + c; /* S2 */
    Z[i] = Y[i] + c; /* S3 */
    Y[i] = c - Y[i]; /* S4 */
}
```

- Watch for antidependencies and output dependencies

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Finding dependencies

- Example 2:

```
for (i=0; i<100; i=i+1) {
    Y[i] = X[i] / c; /* S1 */
    X[i] = X[i] + c; /* S2 */
    Z[i] = Y[i] + c; /* S3 */
    Y[i] = c - Y[i]; /* S4 */
}
```

- Watch for antidependencies and output dependencies

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Reductions

- Reduction Operation:

```
for (i=9999; i>=0; i=i-1)
        sum = sum + x[i] * y[i];
```

- Transform to…

```
for (i=9999; i>=0; i=i-1)
        sum [i] = x[i] * y[i];
for (i=9999; i>=0; i=i-1)
        finalsum = finalsum + sum[i];
```

- Do on p processors:

```
for (i=999; i>=0; i=i-1)
        finalsum[p] = finalsum[p] + sum[i+1000*p];
```

- Note: assumes associativity!

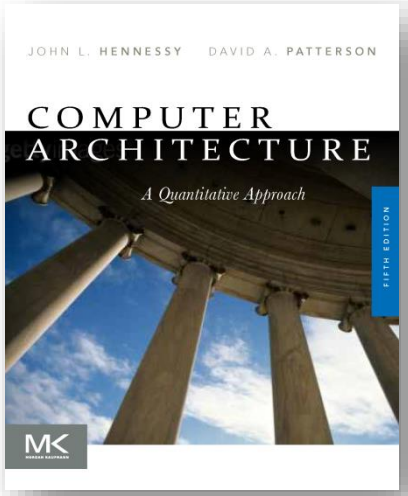
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- GPUs suffer from being coprocessors
  - GPUs have flexibility to change ISA
- Concentrating on peak performance in vector architectures and ignoring start-up overhead
  - Overheads require long vector lengths to achieve speedup
- Increasing vector performance without comparable increases in scalar performance
- You can get good vector performance without providing memory bandwidth
- On GPUs, just add more threads if you don’t have enough memory performance

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved



Chapter 5

Thread-Level Parallelism

Copyright © 2019, Elsevier Inc. All rights Reserved

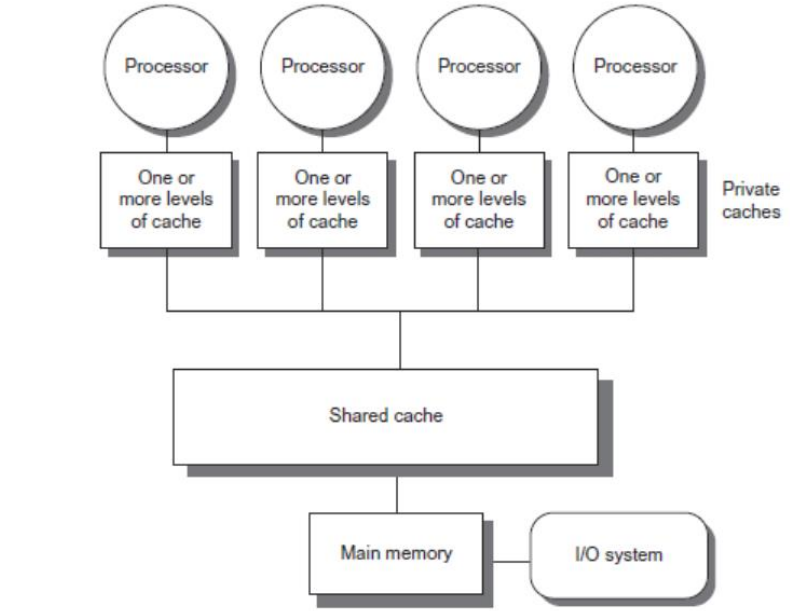
Introduction

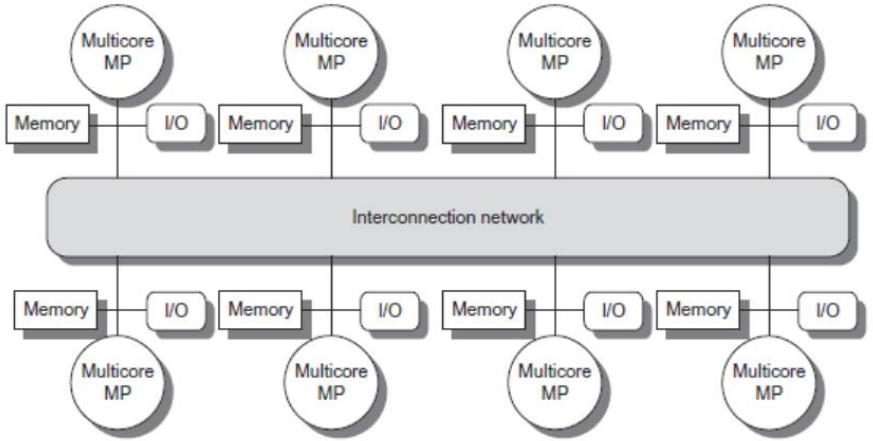
- Thread-Level parallelism
  - Have multiple program counters
  - Uses MIMD model
  - Targeted for tightly-coupled shared-memory multiprocessors
- For *n* processors, need *n* threads
- Amount of computation assigned to each thread = grain size
  - Threads can be used for data-level parallelism, but the overheads may outweigh the benefit

Copyright © 2019, Elsevier Inc. All rights Reserved

Types

- Symmetric multiprocessors (SMP)
  - Small number of cores
  - Share single memory with uniform memory latency
- Distributed shared memory (DSM)
  - Memory distributed among processors
  - Non-uniform memory access/latency (NUMA)
  - Processors connected via direct (switched) and non-direct (multi-hop) interconnection networks





Copyright © 2019, Elsevier Inc. All rights Reserved

Cache Coherence

- Processors may see different values through their caches:

Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1	0

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Cache Coherence

- Coherence
  - All reads by any processor must return the most recently written value
  - Writes to the same location by any two processors are seen in the same order by all processors
- Consistency
  - When a written value will be returned by a read
  - If a processor writes location A followed by location B, any processor that sees the new value of B must also see the new value of A

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Enforcing Coherence

- Coherent caches provide:
  - *Migration*: movement of data
  - *Replication*: multiple copies of data

- Cache coherence protocols
  - Directory based
    - Sharing status of each block kept in one location
  - Snooping
    - Each core tracks sharing status of each block

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Snoopy Coherence Protocols

- Write invalidate
  - On write, invalidate all other copies
  - Use bus itself to serialize
    - Write cannot complete until bus access is obtained

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

  - On write, update all copies

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Snoopy Coherence Protocols

- Locating an item when a read miss occurs
  - In write-back cache, the updated value must be sent to the requesting processor
- Cache lines marked as shared or exclusive/modified
  - Only writes to shared lines need an invalidate broadcast
    - After this, the line is marked as exclusive

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

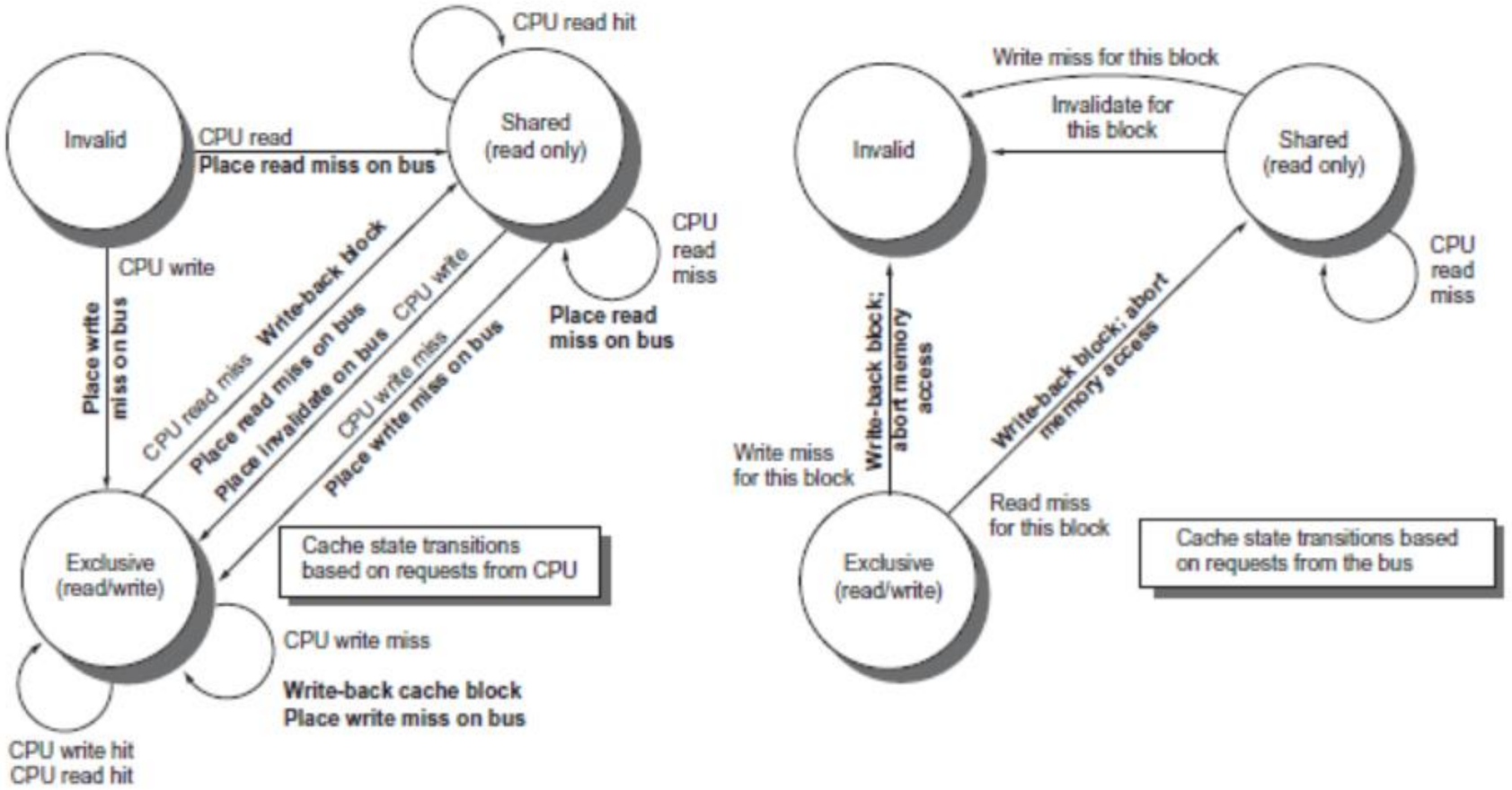
Snoopy Coherence Protocols

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block; then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, because they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block; then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to read shared data: place cache block on bus, write-back block, and change state to shared.
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Snoopy Coherence Protocols



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

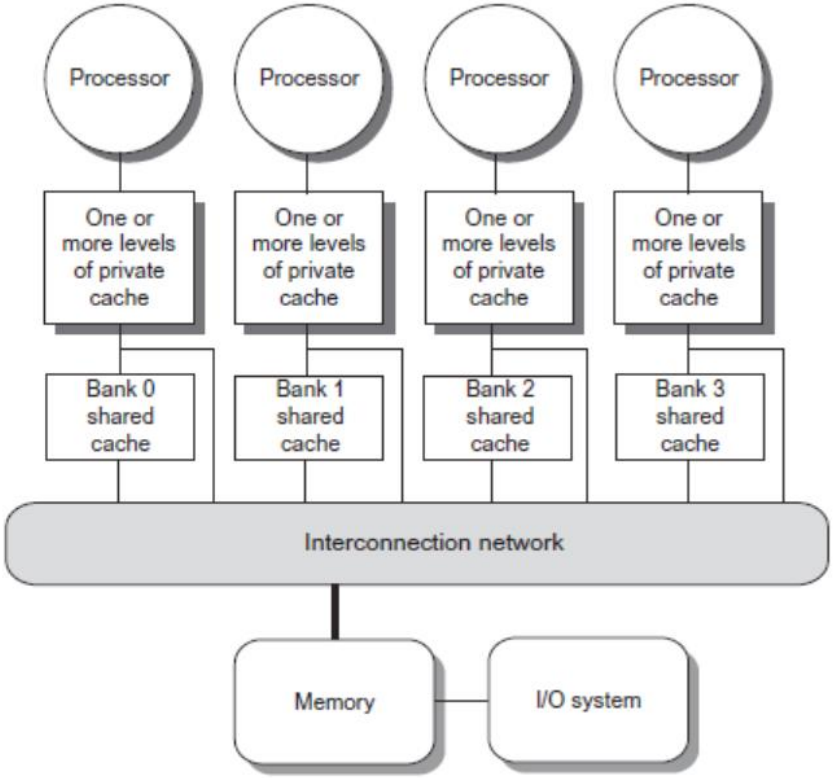
Snoopy Coherence Protocols

- Complications for the basic MSI protocol:
  - Operations are not atomic
    - E.g. detect miss, acquire bus, receive a response
    - Creates possibility of deadlock and races
    - One solution: processor that sends invalidate can hold bus until other processors receive the invalidate
- Extensions:
  - Add exclusive state to indicate clean block in only one cache (MESI protocol)
    - Prevents needing to write invalidate on a write
  - Owned state

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Coherence Protocols: Extensions

- Shared memory bus and snooping bandwidth is bottleneck for scaling symmetric multiprocessors
  - Duplicating tags
  - Place directory in outermost cache
  - Use crossbars or point-to-point networks with banked memory



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Coherence Protocols

- Every multicore with >8 processors uses an interconnect other than bus
  - Makes it difficult to serialize events
  - Write and upgrade misses are not atomic
  - How can the processor know when all invalidates are complete?
  - How can we resolve races when two processors write at the same time?
  - Solution: associate each block with a single bus

Copyright © 2019, Elsevier Inc. All rights Reserved

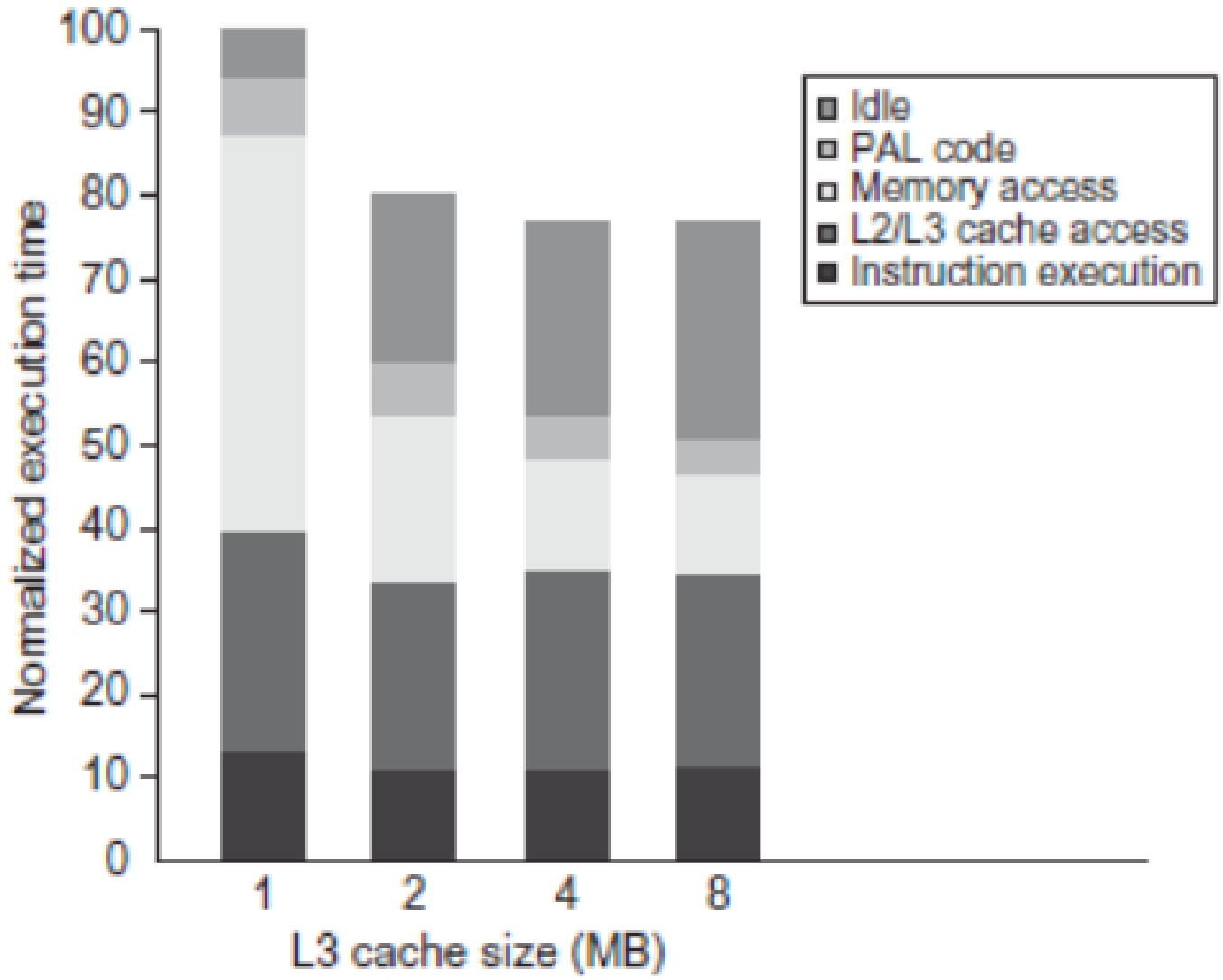
Performance

- Coherence influences cache miss rate
  - Coherence misses
    - True sharing misses
      - Write to shared block (transmission of invalidation)
      - Read an invalidated block
    - False sharing misses
      - Read an unmodified word in an invalidated block

MORGAN KAUFMANN

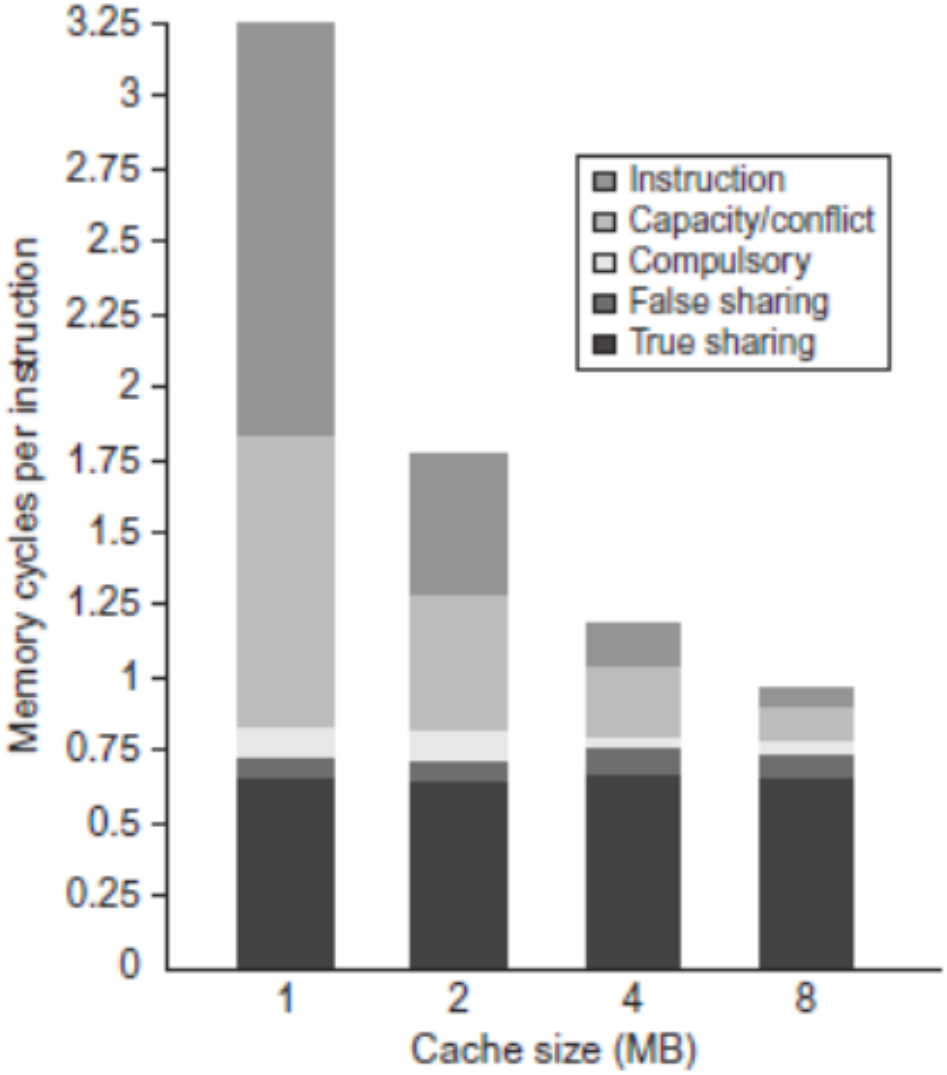
Copyright © 2019, Elsevier Inc. All rights Reserved

Performance Study: Commercial Workload



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

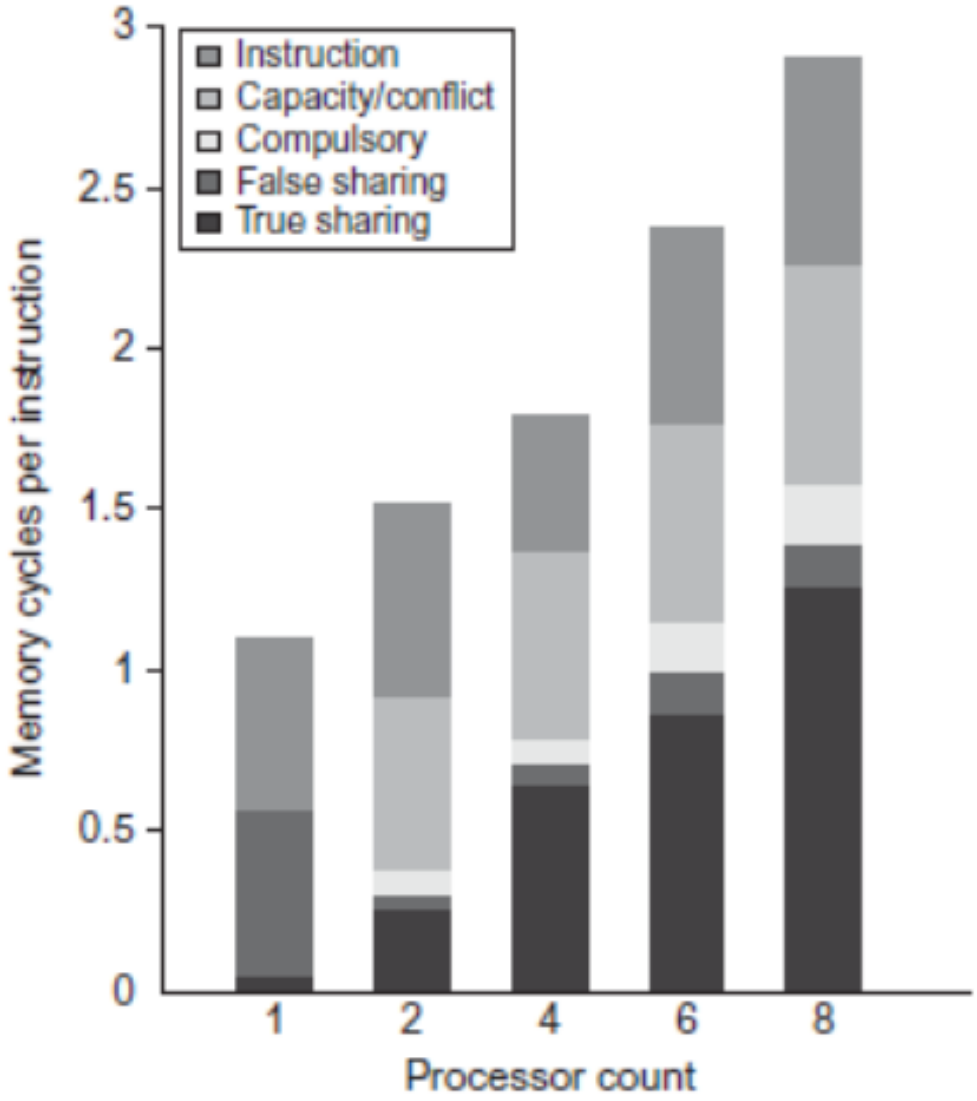
Performance Study: Commercial Workload



MORGAN KAUFMANN

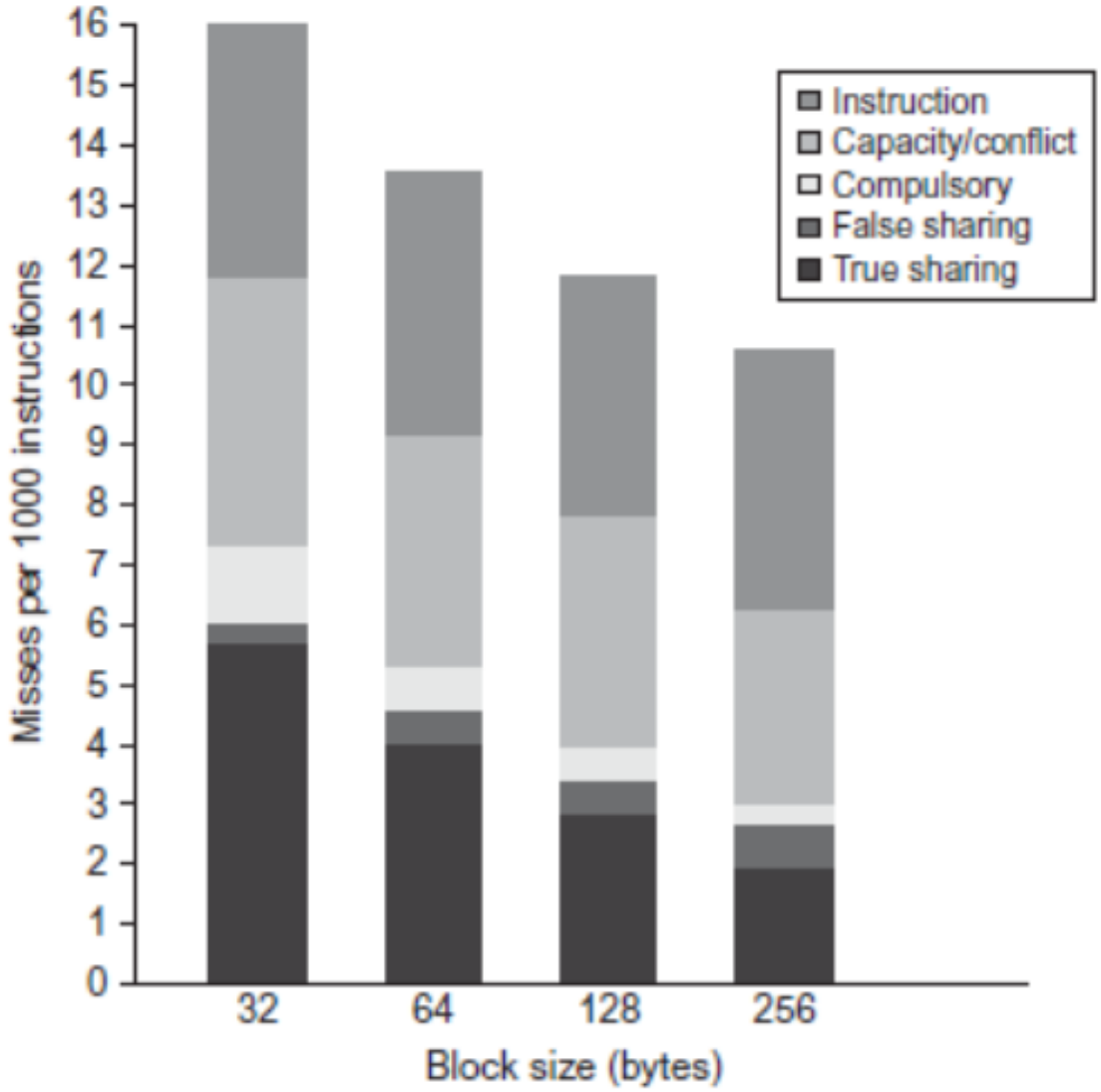
Copyright © 2019, Elsevier Inc. All rights Reserved

Performance Study: Commercial Workload



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Performance Study: Commercial Workload



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

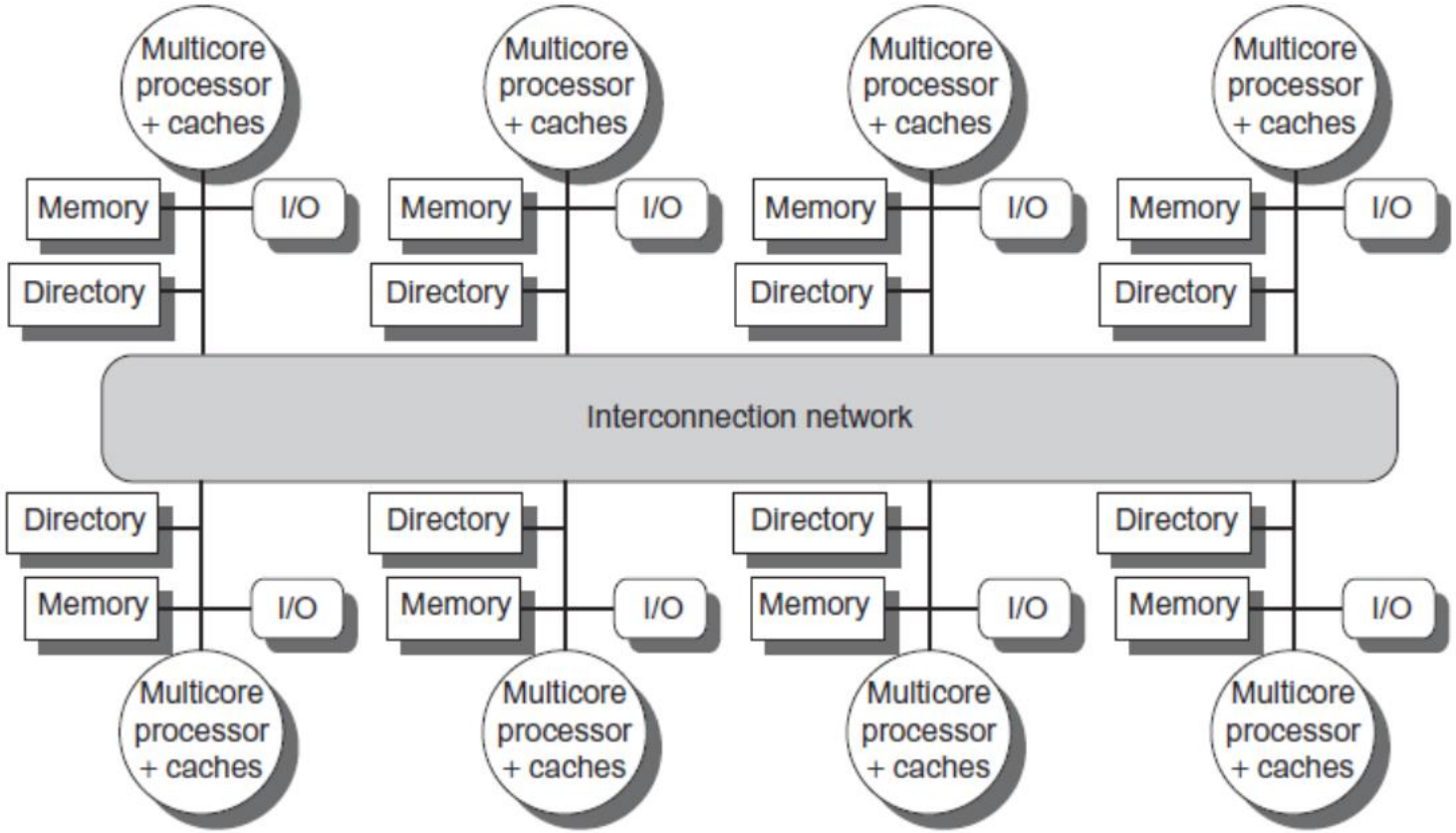
Directory Protocols

- Snooping schemes require communication among all caches on every cache miss
  - Limits scalability
  - Another approach: Use centralized directory to keep track of every block
    - Which caches have each block
    - Dirty status of each block
- Implement in shared L3 cache
  - Keep bit vector of size = # cores for each block in L3
  - Not scalable beyond shared L3

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Directory Protocols

- Alternative approach:
  - Distribute memory



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Directory Protocols

- For each block, maintain state:
  - Shared
    - One or more nodes have the block cached, value in memory is up-to-date
    - Set of node IDs
  - Uncached
  - Modified
    - Exactly one node has a copy of the cache block, value in memory is out-of-date
    - Owner node ID
- Directory maintains block states and sends invalidation messages

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

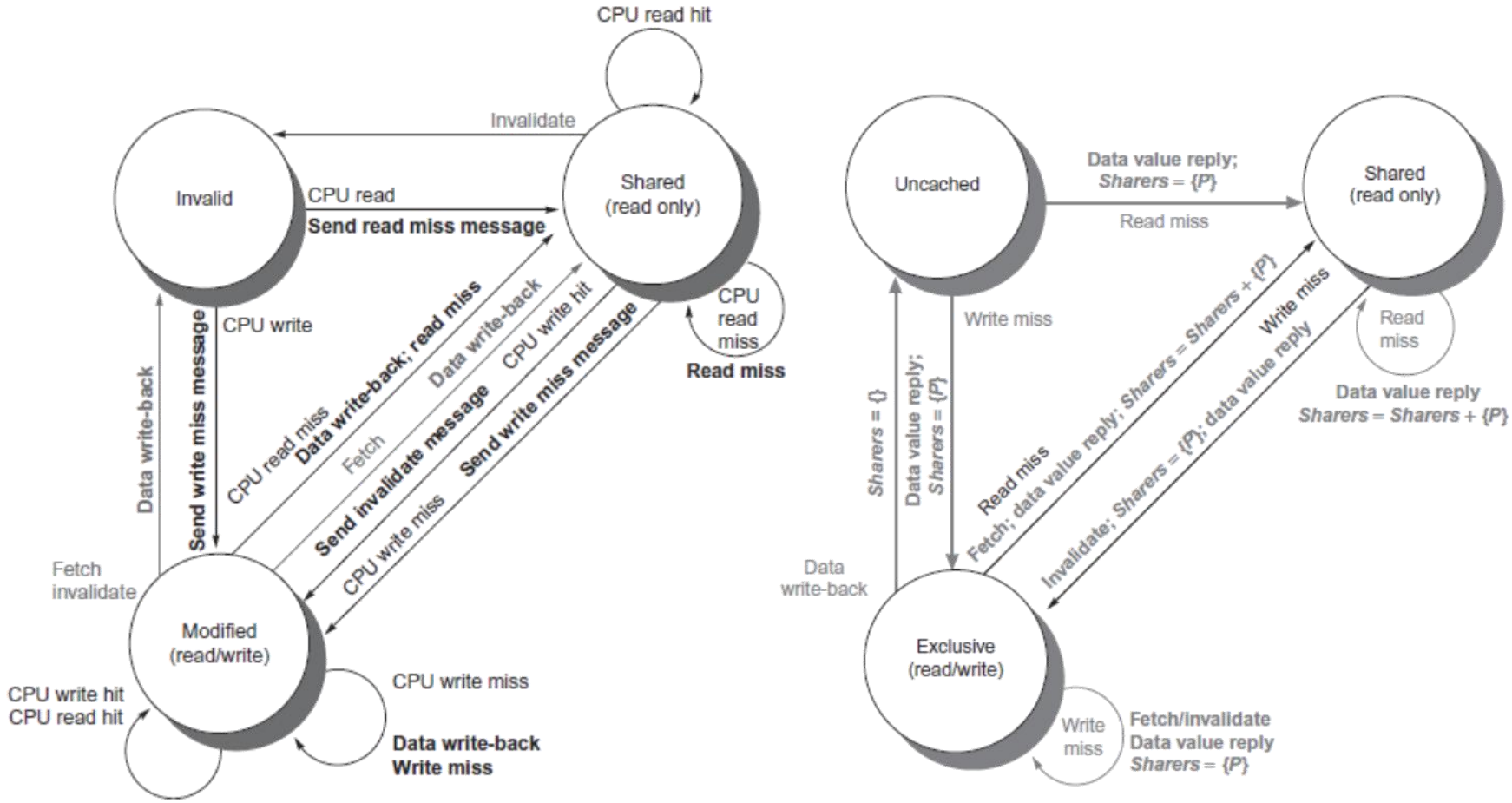
Messages

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/ invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write back a data value for address A.

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Directory Protocols



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Directory Protocols

- For uncached block:
  - Read miss
    - Requesting node is sent the requested data and is made the only sharing node, block is now shared
  - Write miss
    - The requesting node is sent the requested data and becomes the sharing node, block is now exclusive
- For shared block:
  - Read miss
    - The requesting node is sent the requested data from memory, node is added to sharing set
  - Write miss
    - The requesting node is sent the value, all nodes in the sharing set are sent invalidate messages, sharing set only contains requesting node, block is now exclusive

Copyright © 2019, Elsevier Inc. All rights Reserved

Directory Protocols

- For exclusive block:
  - Read miss
    - The owner is sent a data fetch message, block becomes shared, owner sends data to the directory, data written back to memory, sharers set contains old owner and requestor
  - Data write back
    - Block becomes uncached, sharer set is empty
  - Write miss
    - Message is sent to old owner to invalidate and send the value to the directory, requestor becomes new owner, block remains exclusive

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Synchronization

- Basic building blocks:
  - Atomic exchange
    - Swaps register with memory location
  - Test-and-set
    - Sets under condition
  - Fetch-and-increment
    - Reads original value from memory and increments it in memory
  - Requires memory read and write in uninterruptable instruction

  - RISC-V: load reserved/store conditional
    - If the contents of the memory location specified by the load linked are changed before the store conditional to the same address, the store conditional fails

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Implementing Locks

- Atomic exchange (EXCH):

```
try:    mov x3,x4       ;mov exchange value
        lr x2,x1        ;load reserved from
        sc x3,0(x1)     ;store conditional
        bnez x3,try     ;branch store fails
        mov x4,x2       ;put load value in x4?
```

- Atomic increment:

```
try:    lr x2,x1        ;load reserved 0(x1)
        addi x3,x2,1    ;increment
        sc x3,0(x1)     ;store conditional
        bnez x3,try     ;branch store fails
```

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Implementing Locks

- Lock (no cache coherence)

```
addi x2,R0,#1
lockit:   EXCH x2,0(x1)          ;atomic exchange
bnez x2,locket            ;already locked?
```

- Lock (cache coherence):

```
lockit:   ld x2,0(x1)            ;load of lock
bnez x2,locket            ;not available-spin
addi x2,R0,#1             ;load locked value
EXCH x2,0(x1)             ;swap
bnez x2,locket            ;branch if lock wasn't 0
```

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Implementing Locks

- Advantage of this scheme: reduces memory traffic

Step	P0	P1	P2	Coherence state of lock at end of step	Bus/directory activity
1	Has lock	Begins spin, testing if lock = 0	Begins spin, testing if lock = 0	Shared	Cache misses for P1 and P2 satisfied in either order. Lock state becomes shared.
2	Set lock to 0	(Invalidate received)	(Invalidate received)	Exclusive (P0)	Write invalidate of lock variable from P0.
3		Cache miss	Cache miss	Shared	Bus/directory services P2 cache miss; write-back from P0; state shared.
4		(Waits while bus/ directory busy)	Lock = 0 test succeeds	Shared	Cache miss for P2 satisfied.
5		Lock = 0	Executes swap, gets cache miss	Shared	Cache miss for P1 satisfied.
6		Executes swap, gets cache miss	Completes swap: returns 0 and sets lock = 1	Exclusive (P2)	Bus/directory services P2 cache miss; generates invalidate; lock is exclusive.
7		Swap completes and returns 1, and sets lock = 1	Enter critical section	Exclusive (P1)	Bus/directory services P1 cache miss; sends invalidate and generates write-back from P2.
8		Spins, testing if lock = 0			None

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Models of Memory Consistency

Processor 1:	Processor 2:
A=0	B=0
…	…
A=1	B=1
if (B==0) …	if (A==0) …

- Should be impossible for both if-statements to be evaluated as true
  - Delayed write invalidate?
- Sequential consistency:
  - Result of execution should be the same as long as:
    - Accesses on each processor were kept in order
    - Accesses on different processors were arbitrarily interleaved

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Implementing Locks

- To implement, delay completion of all memory accesses until all invalidations caused by the access are completed
  - Reduces performance!
- Alternatives:
  - Program-enforced synchronization to force write on processor to occur before read on the other processor
    - Requires synchronization object for A and another for B
      - “Unlock” after write
      - “Lock” after read

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

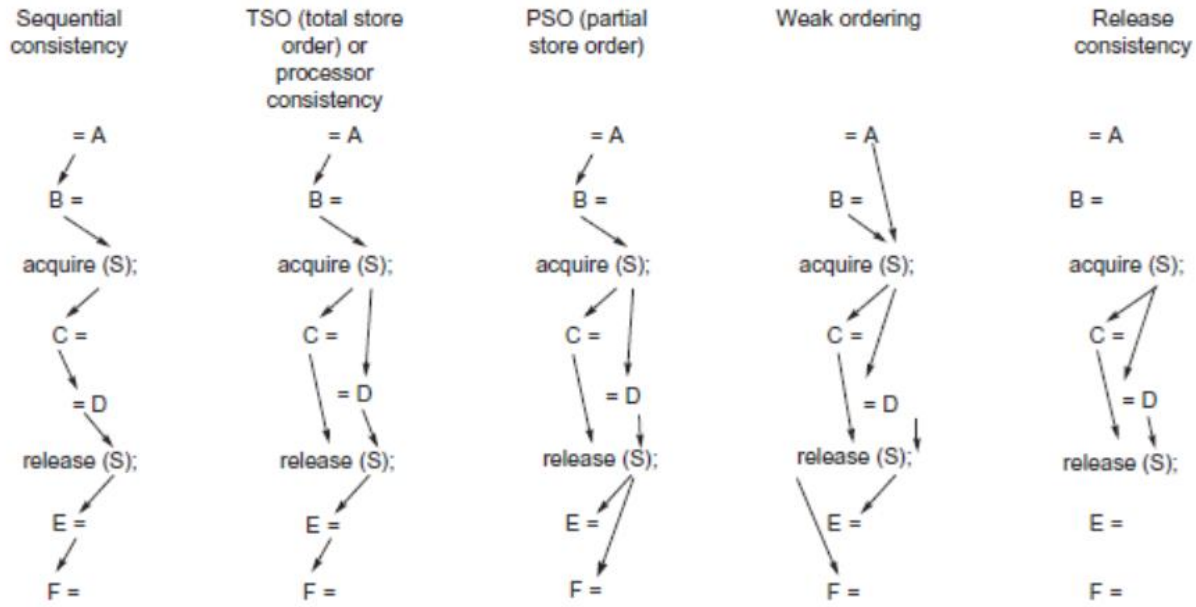
Relaxed Consistency Models

- Rules:
  - X → Y
    - Operation X must complete before operation Y is done
    - Sequential consistency requires:
      - R → W, R → R, W → R, W → W
  - Relax W → R
    - "Total store ordering"
  - Relax W → W
    - "Partial store order"
  - Relax R → W and R → R
    - "Weak ordering" and "release consistency"

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Relaxed Consistency Models

Model	Used in	Ordinary orderings	Synchronization orderings
Sequential consistency	Most machines as an optional mode	R → R, R → W, W → R, W → W	S → W, S → R, R → S, W → S, S → S
Total store order or processor consistency	IBMS/370, DEC VAX, SPARC	R → R, R → W, W → W	S → W, S → R, R → S, W → S, S → S
Partial store order	SPARC	R → R, R → W	S → W, S → R, R → S, W → S, S → S
Weak ordering	PowerPC		S → W, S → R, R → S, W → S, S → S
Release consistency	MIPS, RISC V, Armv8, C, and C++ specifications		$S_A \rightarrow W$, $S_A \rightarrow R$, $R \rightarrow S_R$, $W \rightarrow S_R$, $S_A \rightarrow S_A$, $S_A \rightarrow S_R$, $S_R \rightarrow S_A$, $S_R \rightarrow S_R$



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Relaxed Consistency Models

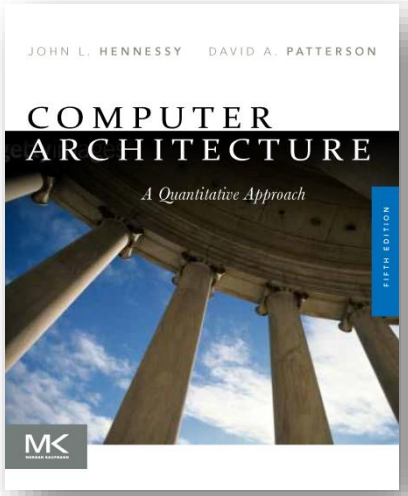
- Consistency model is multiprocessor specific
- Programmers will often implement explicit synchronization
- Speculation gives much of the performance advantage of relaxed models with sequential consistency
  - Basic idea: if an invalidation arrives for a result that has not been committed, use speculation recovery

Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- Measuring performance of multiprocessors by linear speedup versus execution time
- Amdahl's Law doesn't apply to parallel computers
- Linear speedups are needed to make multiprocessors cost-effective
  - Doesn't consider cost of other system components
- Not developing the software to take advantage of, or optimize for, a multiprocessor architecture

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved



Chapter 6

Warehouse-Scale Computers to Exploit Request-Level and Data-Level Parallelism

Copyright © 2019, Elsevier Inc. All rights Reserved

Introduction

- Warehouse-scale computer (WSC)
  - Provides Internet services
    - Search, social networking, online maps, video sharing, online shopping, email, cloud computing, etc.
  - Differences with HPC “clusters”:
    - Clusters have higher performance processors and network
    - Clusters emphasize thread-level parallelism, WSCs emphasize request-level parallelism
  - Differences with datacenters:
    - Datacenters consolidate different machines and software into one location
    - Datacenters emphasize virtual machines and hardware heterogeneity in order to serve varied customers

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Introduction

- Important design factors for WSC:
  - Cost-performance
    - Small savings add up
  - Energy efficiency
    - Affects power distribution and cooling
    - Work per joule
  - Dependability via redundancy
  - Network I/O
  - Interactive and batch processing workloads

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

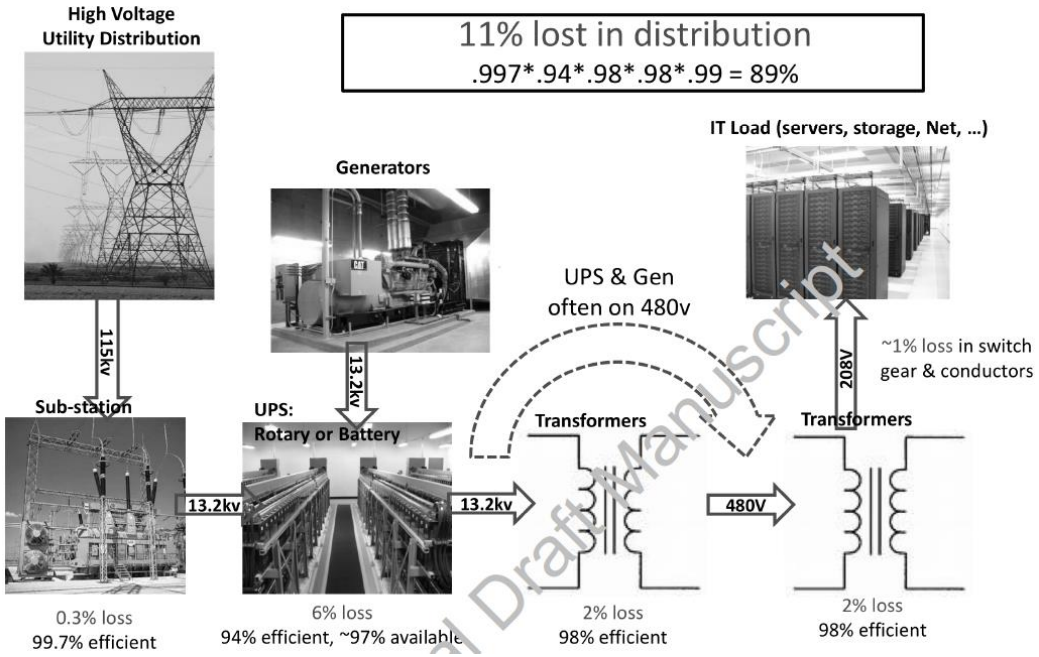
Introduction

- Ample computational parallelism is not important
  - Most jobs are totally independent
  - “Request-level parallelism”
- Operational costs count
  - Power consumption is a primary, not secondary, constraint when designing system
- Scale and its opportunities and problems
  - Can afford to build customized systems since WSC require volume purchase
- Location counts
  - Real estate, power cost; Internet, end-user, and workforce availability
- Computing efficiently at low utilization
- Scale and the opportunities/problems associated with scale
  - Unique challenges: custom hardware, failures
  - Unique opportunities: bulk discounts

Copyright © 2019, Elsevier Inc. All rights Reserved

Efficiency and Cost of WSC

- Location of WSC
  - Proximity to Internet backbones, electricity cost, property tax rates, low risk from earthquakes, floods, and hurricanes
- Power distribution



Final Draft Manuscript

Copyright © 2019, Elsevier Inc. All rights Reserved

Prgrm'g Models and Workloads

- Batch processing framework: MapReduce
  - **Map:** applies a programmer-supplied function to each logical input record
    - Runs on thousands of computers
    - Provides new set of key-value pairs as intermediate values
  - **Reduce:** collapses values using another programmer-supplied function

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Prgrm'g Models and Workloads

- Example:
  - **map (String key, String value)**:
    - **// key: document name**
    - **// value: document contents**
    - **for each word w in value**
      - **EmitIntermediate(w,"1"); // Produce list of all words**
  - **reduce (String key, Iterator values):**
    - **// key: a word**
    - **// value: a list of counts**
    - **int result = 0;**
    - **for each v in values:**
      - **result += ParseInt(v); // get integer from key-value pair**
    - **Emit(AsString(result));**

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Prgrm'g Models and Workloads

- Availability:
  - Use replicas of data across different servers
  - Use relaxed consistency:
    - No need for all replicas to always agree
- File systems: GFS and Colossus
- Databases: Dynamo and BigTable

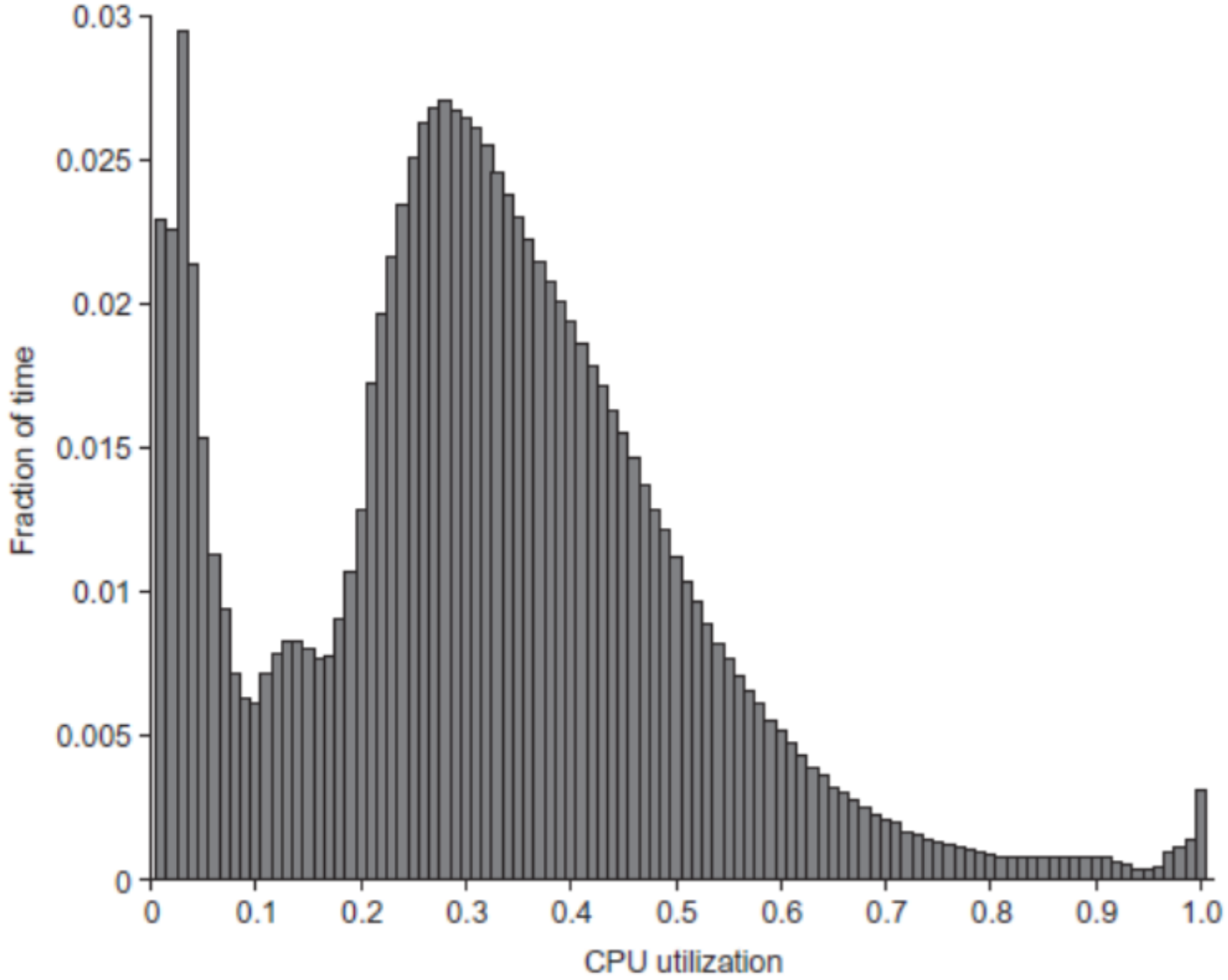
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Prgrm'g Models and Workloads

- MapReduce runtime environment schedules map and reduce task to WSC nodes
  - Workload demands often vary considerably
  - Scheduler assigns tasks based on completion of prior tasks
  - Tail latency/execution time variability: single slow task can hold up large MapReduce job
  - Runtime libraries replicate tasks near end of job

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Prgrm'g Models and Workloads



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Computer Architecture of WSC

- WSC often use a hierarchy of networks for interconnection
- Each 19” rack holds 48 1U servers connected to a rack switch
- Rack switches are uplinked to switch higher in hierarchy
  - Uplink has 6-24X times lower bandwidthGoal is to maximize locality of communication relative to the rack

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Storage

- **Storage options:**
  - **Use disks inside the servers, or**
  - **Network attached storage through Infiniband**

  - **WSCs generally rely on local disks**
  - **Google File System (GFS) uses local disks and maintains at least three relicas**

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Array Switch

- Switch that connects an array of racks
  - Array switch should have 10 X the bisection bandwidth of rack switch
  - Cost of *n*-port switch grows as n^2
  - Often utilize content addressible memory chips and FPGAs

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

WSC Memory Hierarchy

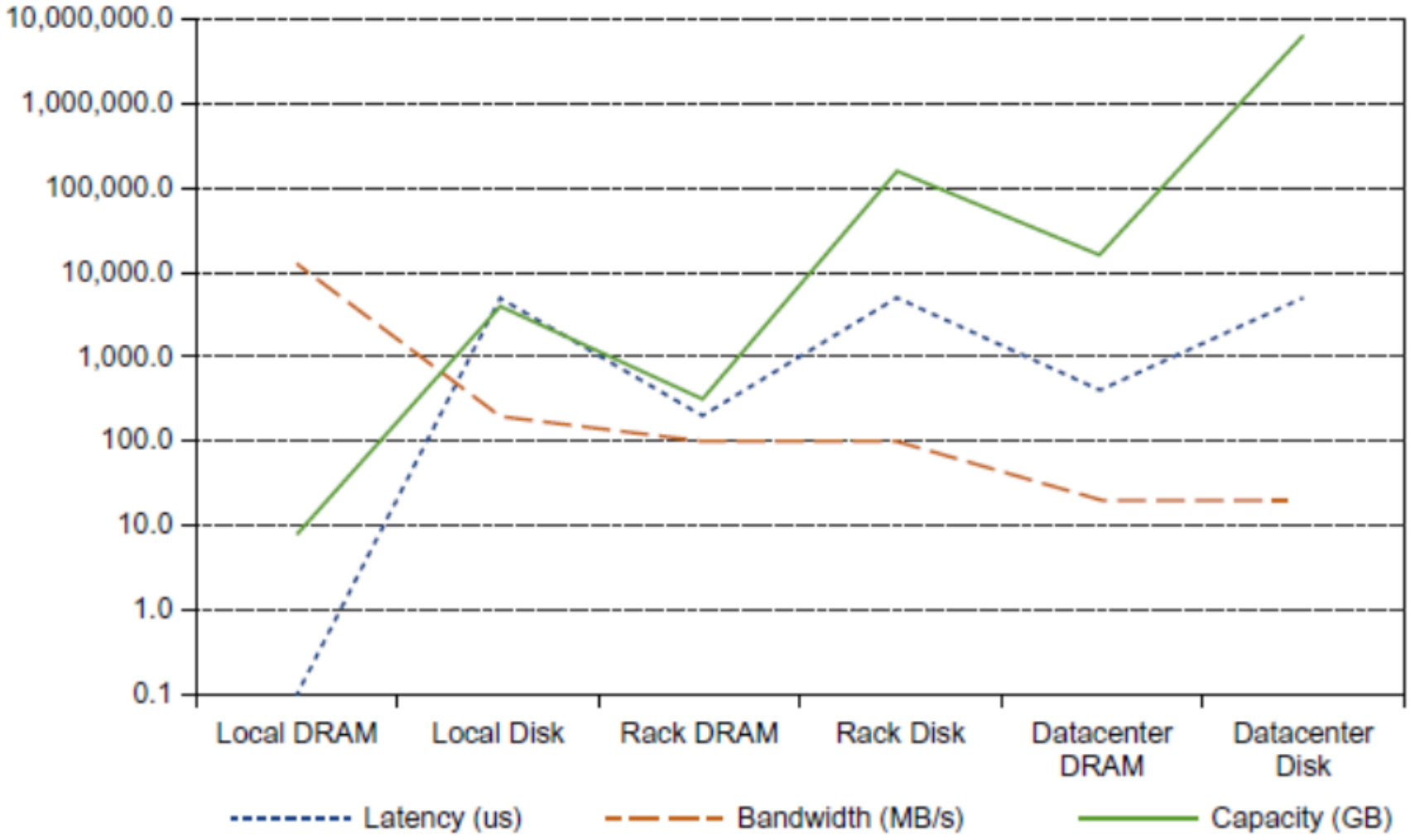
- Servers can access DRAM and disks on other servers using a NUMA-style interface

	Local	Rack	Array
DRAM latency (μs)	0.1	300	500
Flash latency (μs)	100	400	600
Disk latency (μs)	10,000	11,000	12,000
DRAM bandwidth (MB/s)	20,000	100	10
Flash bandwidth (MB/s)	1000	100	10
Disk bandwidth (MB/s)	200	100	10
DRAM capacity (GB)	16	1024	31,200
Flash capacity (GB)	128	20,000	600,000
Disk capacity (GB)	2000	160,000	4,800,000

MORGAN KAUFMANN

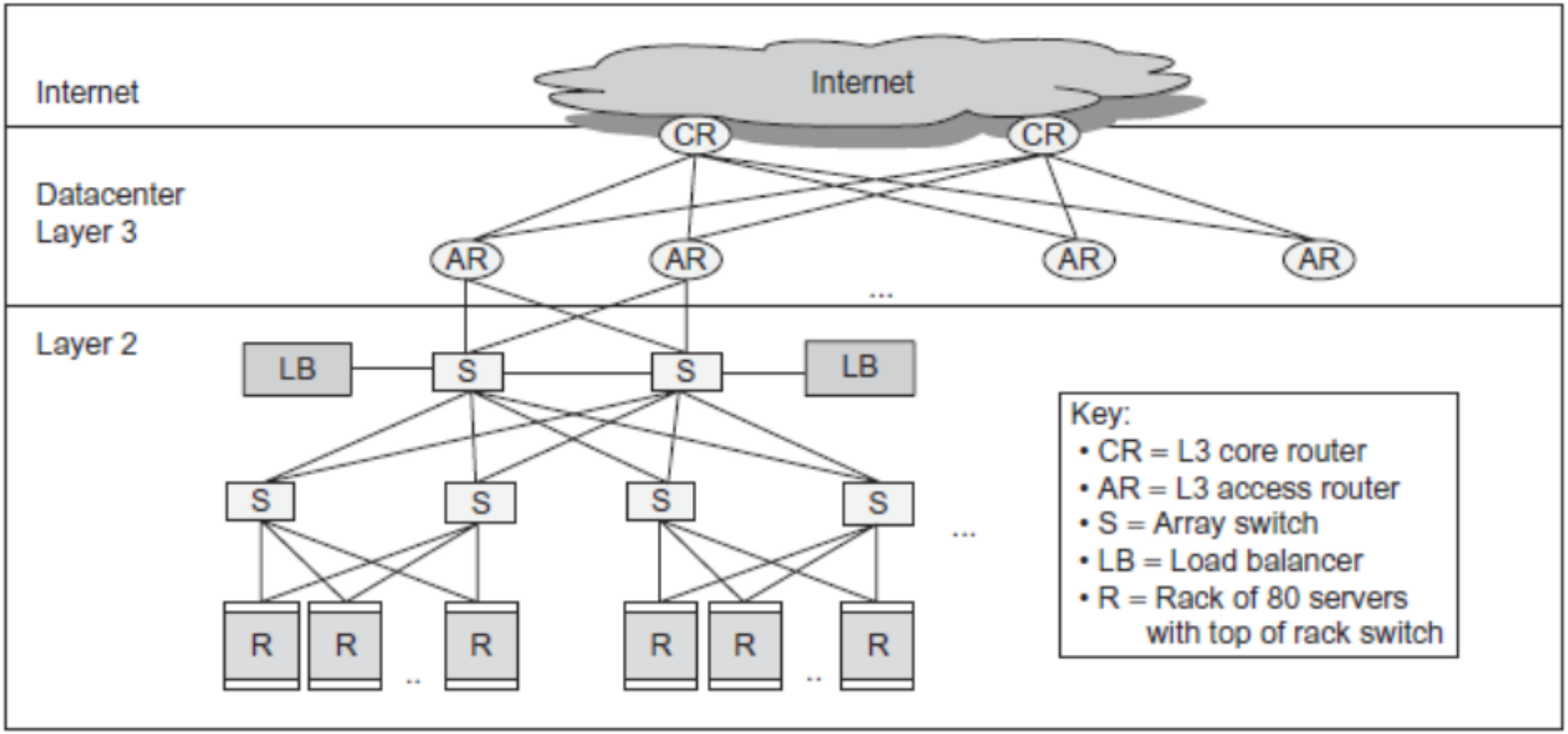
Copyright © 2019, Elsevier Inc. All rights Reserved

WSC Memory Hierarchy



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

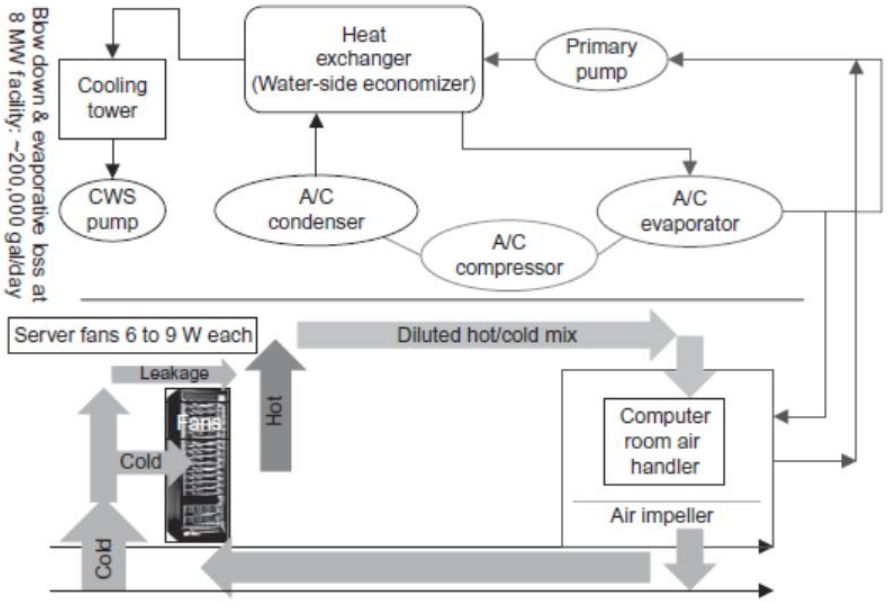
WSC Memory Hierarchy



Copyright © 2019, Elsevier Inc. All rights Reserved

Infrastructure and Costs of WSC

- Cooling
  - Air conditioning used to cool server room
  - 64 F – 71 F
    - Keep temperature higher (closer to 71 F)
  - Cooling towers can also be used
    - Minimum temperature is “wet bulb temperature”



Copyright © 2019, Elsevier Inc. All rights Reserved

Infrastructure and Costs of WSC

- Cooling system also uses water (evaporation and spills)
  - E.g. 70,000 to 200,000 gallons per day for an 8 MW facility
- Power cost breakdown:
  - Chillers: 30-50% of the power used by the IT equipment
  - Air conditioning: 10-20% of the IT power, mostly due to fans
- How man servers can a WSC support?
  - Each server:
    - "Nameplate power rating" gives maximum power consumption
    - To get actual, measure power under actual workloads
  - Oversubscribe cumulative server power by 40%, but monitor power closely

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Infrastructure and Costs of WSC

- Determining the maximum server capacity
  - Nameplate power rating: maximum power that a server can draw
  - Better approach: measure under various workloads
  - Oversubscribe by 40%
- Typical power usage by component:
  - Processors: 42%
  - DRAM: 12%
  - Disks: 14%
  - Networking: 5%
  - Cooling: 15%
  - Power overhead: 8%
  - Miscellaneous: 4%

Copyright © 2019, Elsevier Inc. All rights Reserved

Measuring Efficiency of a WSC

- **Power Utilization Effectiveness (PEU)**
  - = Total facility power / IT equipment power
  - Median PUE on 2006 study was 1.69
- **Performance**
  - Latency is important metric because it is seen by users
  - Bing study: users will use search less as response time increases
  - Service Level Objectives (SLOs)/Service Level Agreements (SLAs)
    - E.g. 99% of requests be below 100 ms

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Measuring Efficiency of a WSC

Server delay (ms)	Increased time to next click (ms)	Queries/ user	Any clicks/ user	User satisfaction	Revenue/ user
50	–	–	–	–	–
200	500	–	–0.3%	–0.4%	–
500	1200	–	–1.0%	–0.9%	–1.2%
1000	1900	–0.7%	–1.9%	–1.6%	–2.8%
2000	3100	–1.8%	–4.4%	–3.8%	–4.3%

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Cost of a WSC

- Capital expenditures (CAPEX)
  - Cost to build a WSC
  - $9 to 13/watt
- Operational expenditures (OPEX)
  - Cost to operate a WSC

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

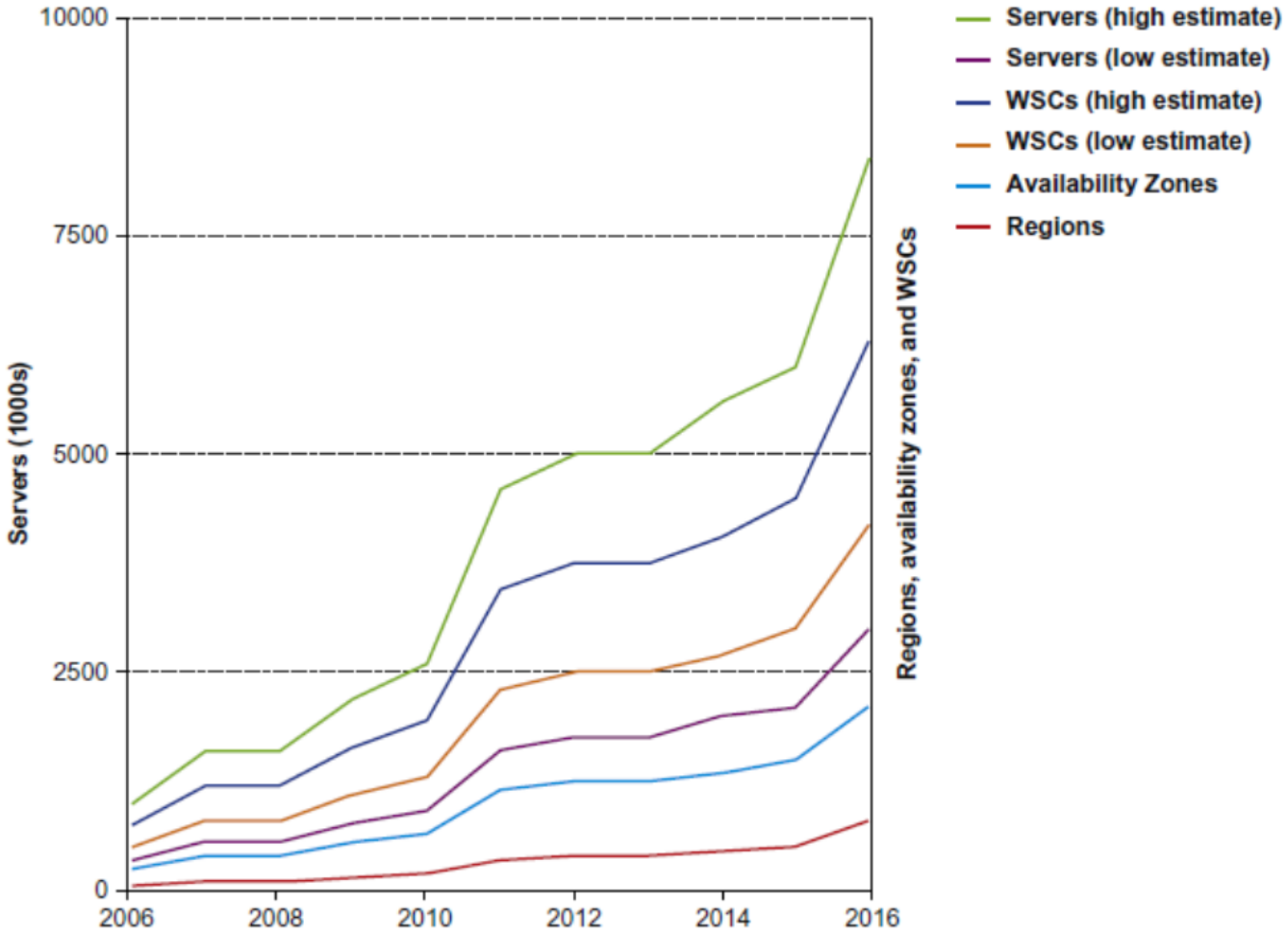
Cloud Computing

- Amazon Web Services
  - Virtual Machines: Linux/Xen
  - Low cost
  - Open source software
  - Initially no guarantee of service
  - No contract

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Cloud Computing

- **Cloud Computing Growth**



Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- Cloud computing providers are losing money
  - AWS has a margin of 25%, Amazon retail 3%
- Focusing on average performance instead of 99th percentile performance
- Using too wimpy a processor when trying to improve WSC cost-performance
- Inconsistent Measure of PUE by different companies
- Capital costs of the WSC facility are higher than for the servers that it houses

MORGAN KAUFMANN

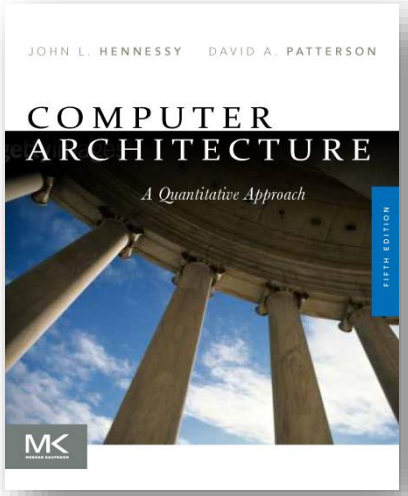
Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- Trying to save power with inactive low power modes versus active low power modes
- Given improvements in DRAM dependability and the fault tolerance of WSC systems software, there is no need to spend extra for ECC memory in a WSC
- Coping effectively with microsecond (e.g. Flash and 100 GbE) delays as opposed to nansecond or millisecond delays
- Turning off hardware during periods of low activity improves the cost-performance of a WSC

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved



Chapter 7

Domain-Specific Architectures

Copyright © 2019, Elsevier Inc. All rights Reserved

Introduction

- Moore's Law enabled:
  - Deep memory hierarchy
  - Wide SIMD units
  - Deep pipelines
  - Branch prediction
  - Out-of-order execution
  - Speculative prefetching
  - Multithreading
  - Multiprocessing

- Objective:
  - Extract performance from software that is oblivious to architecture

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Introduction

- Need factor of 100 improvements in number of operations per instruction
  - Requires domain specific architectures
  - For ASICs, NRE cannot be amoratized over large volumes
  - FPGAs are less efficient than ASICs

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Guidelines for DSAs

- Use dedicated memories to minimize data movement
- Invest resources into more arithmetic units or bigger memories
- Use the easiest form of parallelism that matches the domain
- Reduce data size and type to the simplest needed for the domain
- Use a domain-specific programming language

Copyright © 2019, Elsevier Inc. All rights Reserved

Guidelines for DSAs

Guideline	TPU	Catapult	Crest	Pixel Visual Core
Design target	Data center ASIC	Data center FPGA	Data center ASIC	PMD ASIC/SOC IP
1. Dedicated memories	24 MiB Unified Buffer, 4 MiB Accumulators	Varies	N.A.	Per core: 128 KiB line buffer, 64 KiB P.E. memory
2. Larger arithmetic unit	65,536 Multiply-accumulators	Varies	N.A.	Per core: 256 Multiply-accumulators (512 ALUs)
3. Easy parallelism	Single-threaded, SIMD, in-order	SIMD, MISD	N.A.	MPMD, SIMD, VLIW
4. Smaller data size	8-Bit, 16-bit integer	8-Bit, 16-bit integer 32-bit Fl. Pt.	21-bit Fl. Pt.	8-bit, 16-bit, 32-bit integer
5. Domain-specific lang.	TensorFlow	Verilog	TensorFlow	Halide/TensorFlow

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Example: Deep Neural Networks

- Inpired by neuron of the brain
- Computes non-linear “activiation” function of the weighted sum of input values
- Neurons arranged in layers

Name	DNN layers	Weights	Operations/Weight
MLP0	5	20M	200
MLP1	4	5M	168
LSTM0	58	52M	64
LSTM1	56	34M	96
CNN0	16	8M	2888
CNN1	89	100M	1750

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Example: Deep Neural Networks

- Most practioners will choose an existing design
  - Topology
  - Data type
- Training (learning):
  - Calculate weights using backpropagation algorithm
  - Supervised learning: stocastic graduate descent

Type of data	Problem area	Size of benchmark's training set	DNN architecture	Hardware	Training time
text [1]	Word prediction (word2vec)	100 billion words (Wikipedia)	2-layer skip gram	1 NVIDIA Titan X GPU	6.2 hours
audio [2]	Speech recognition	2000 hours (Fisher Corpus)	11-layer RNN	1 NVIDIA K1200 GPU	3.5 days
images [3]	Image classification	1 million images (ImageNet)	22-layer CNN	1 NVIDIA K20 GPU	3 weeks
video [4]	activity recognition	1 million videos (Sports-1M)	8-layer CNN	10 NVIDIA GPUs	1 month

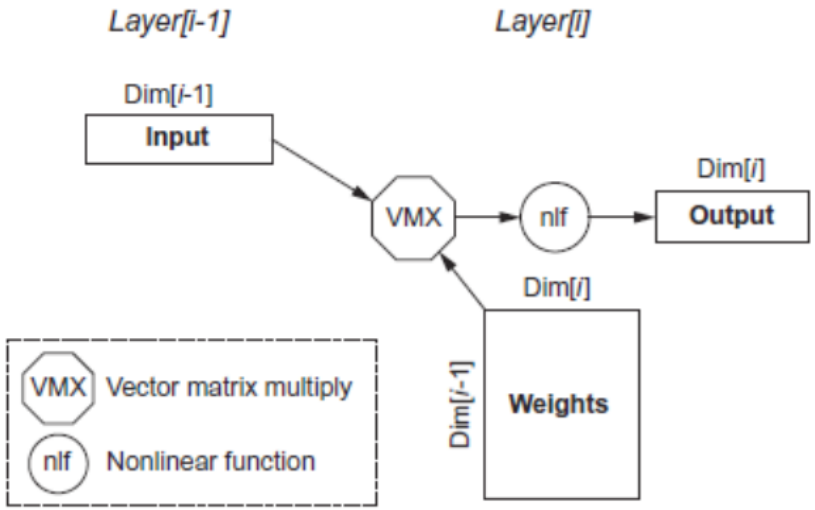
- Inference: use neural network for classification

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Multi-Layer Perceptrons

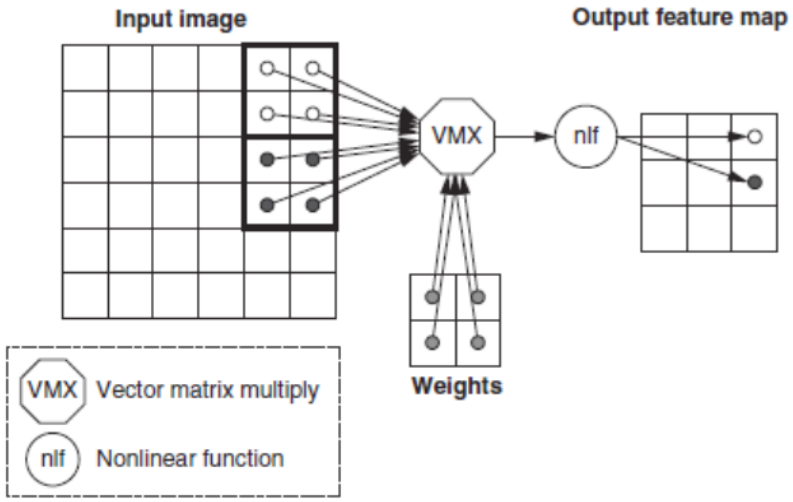
- Parameters:
  - Dim[i]: number of neurons
  - Dim[i-1]: dimension of input vector
  - Number of weights: Dim[i-1] x Dim[i]
  - Operations: 2 x Dim[i-1] x Dim[i]
  - Operations/weight: 2



Copyright © 2019, Elsevier Inc. All rights Reserved

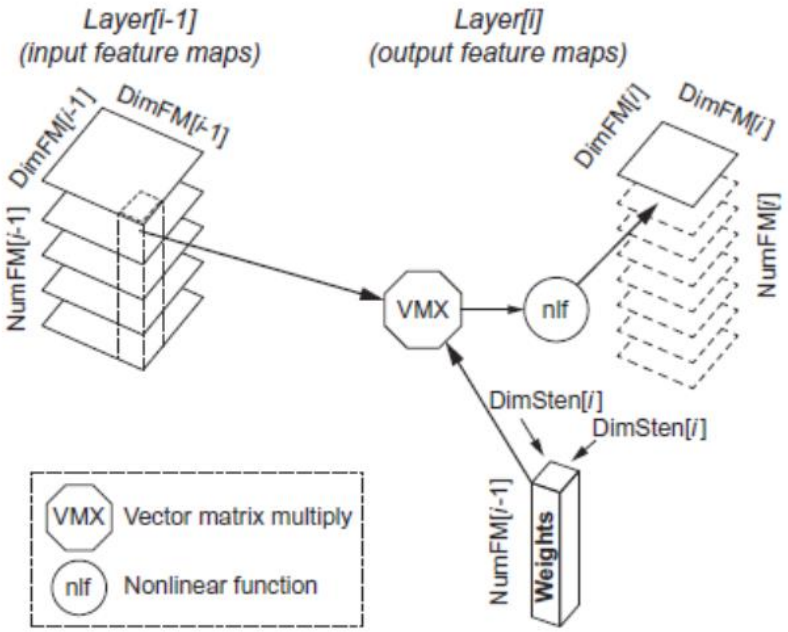
Convolutional Neural Network

- Computer vision
- Each layer raises the level of abstraction
  - First layer recognizes horizontal and vertical lines
  - Second layer recognizes corners
  - Third layer recognizes shapes
  - Fourth layer recognizes features, such as ears of a dog
  - Higher layers recognizes different breeds of dogs



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Convolutional Neural Network



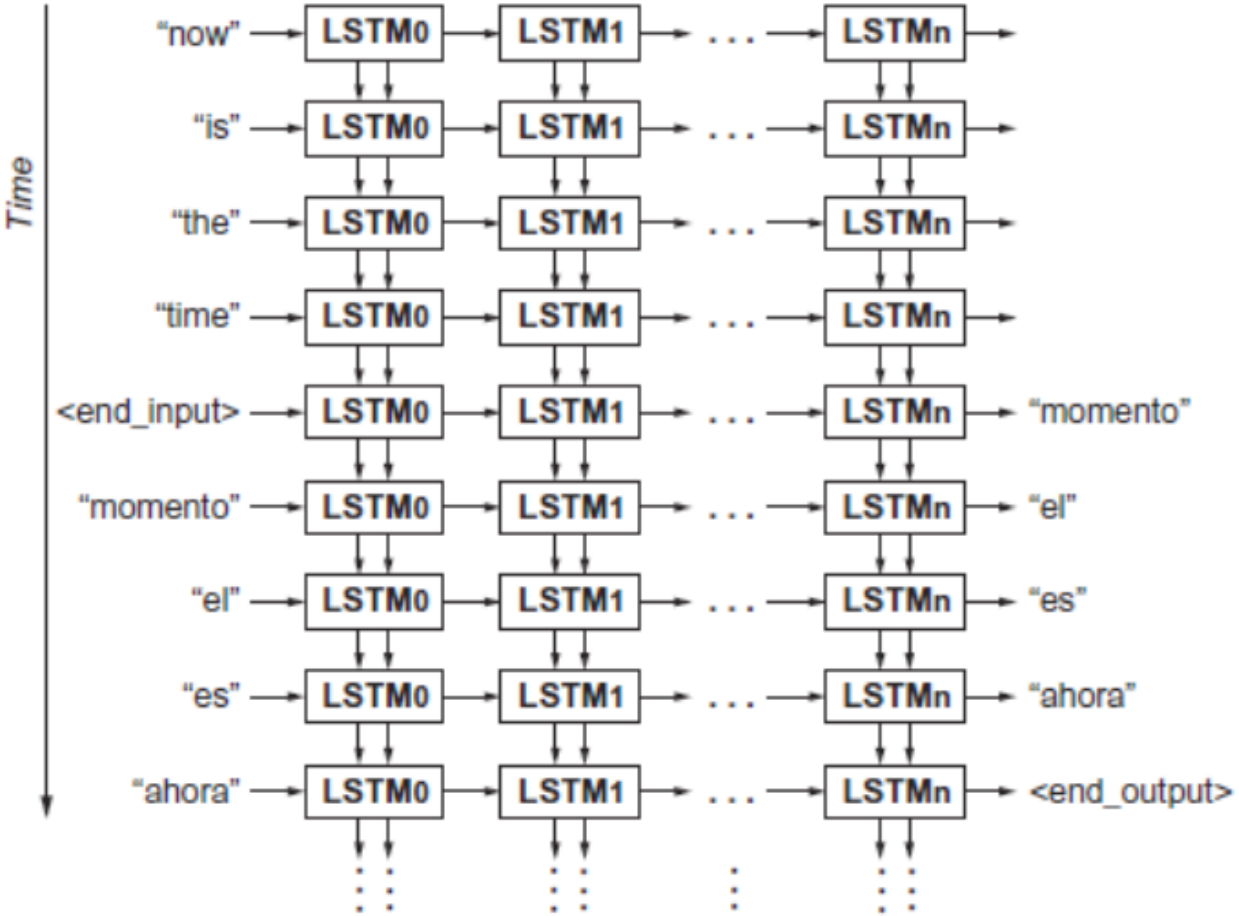
- Parameters:
  - DimFM[i-1]: Dimension of the (square) input Feature Map
  - DimFM[i]: Dimension of the (square) output Feature Map
  - DimSten[i]: Dimension of the (square) stencil
  - NumFM[i-1]: Number of input Feature Maps
  - NumFM[i]: Number of output Feature Maps
  - Number of neurons: NumFM[i] x DimFM[i]^2
  - Number of weights per output Feature Map: NumFM[i-1] x DimSten[i]^2
  - Total number of weights per layer: NumFM[i] x Number of weights per output Feature Map
  - Number of operations per output Feature Map: 2 x DimFM[i]^2 x Number of weights per output Feature Map
  - Total number of operations per layer: NumFM[i] x Number of operations per output Feature Map = 2 x DimFM[i]^2 x NumFM[i] x Number of weights per output Feature Map = 2 x DimFM[i]^2 x Total number of weights per layer
  - Operations/Weight: 2 x DimFM[i]^2

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

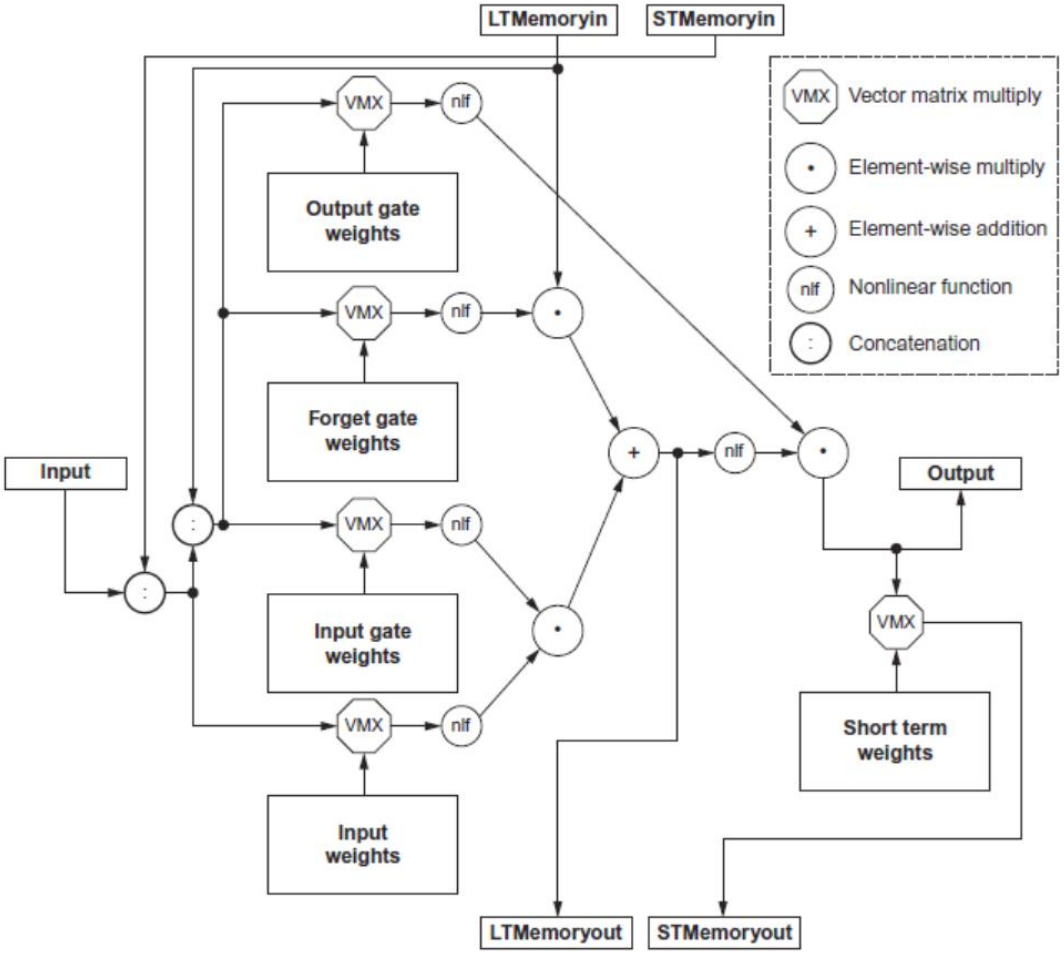
Recurrent Neural Network

- Speech recognition and language translation
- Long short-term memory (LSTM) network



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Recurrent Neural Network



- Parameters:
  - Number of weights per cell: 3 x (3 x Dim x Dim)+(2 x Dim x Dim) + (1 x Dim x Dim) = 12 x Dim^2
  - Number of operations for the 5 vector-matrix multiplies per cell: 2 x Number of weights per cell = 24 x Dim^2
  - Number of operations for the 3 element-wise multiplies and 1 addition (vectors are all the size of the output): 4 x Dim
  - Total number of operations per cell (5 vector-matrix multiplies and the 4 element-wise operations): 24 x Dim^2 + 4 x Dim
  - Operations/Weight: ~2

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Convolutional Neural Network

- Batches:
  - Reuse weights once fetched from memory across multiple inputs
  - Increases operational intensity
- Quantization
  - Use 8- or 16-bit fixed point
- Summary:
  - Need the following kernels:
    - Matrix-vector multiply
    - Matrix-matrix multiply
    - Stencil
    - ReLU
    - Sigmoid
    - Hyperbolic tangeant

MORGAN KAUFMANN

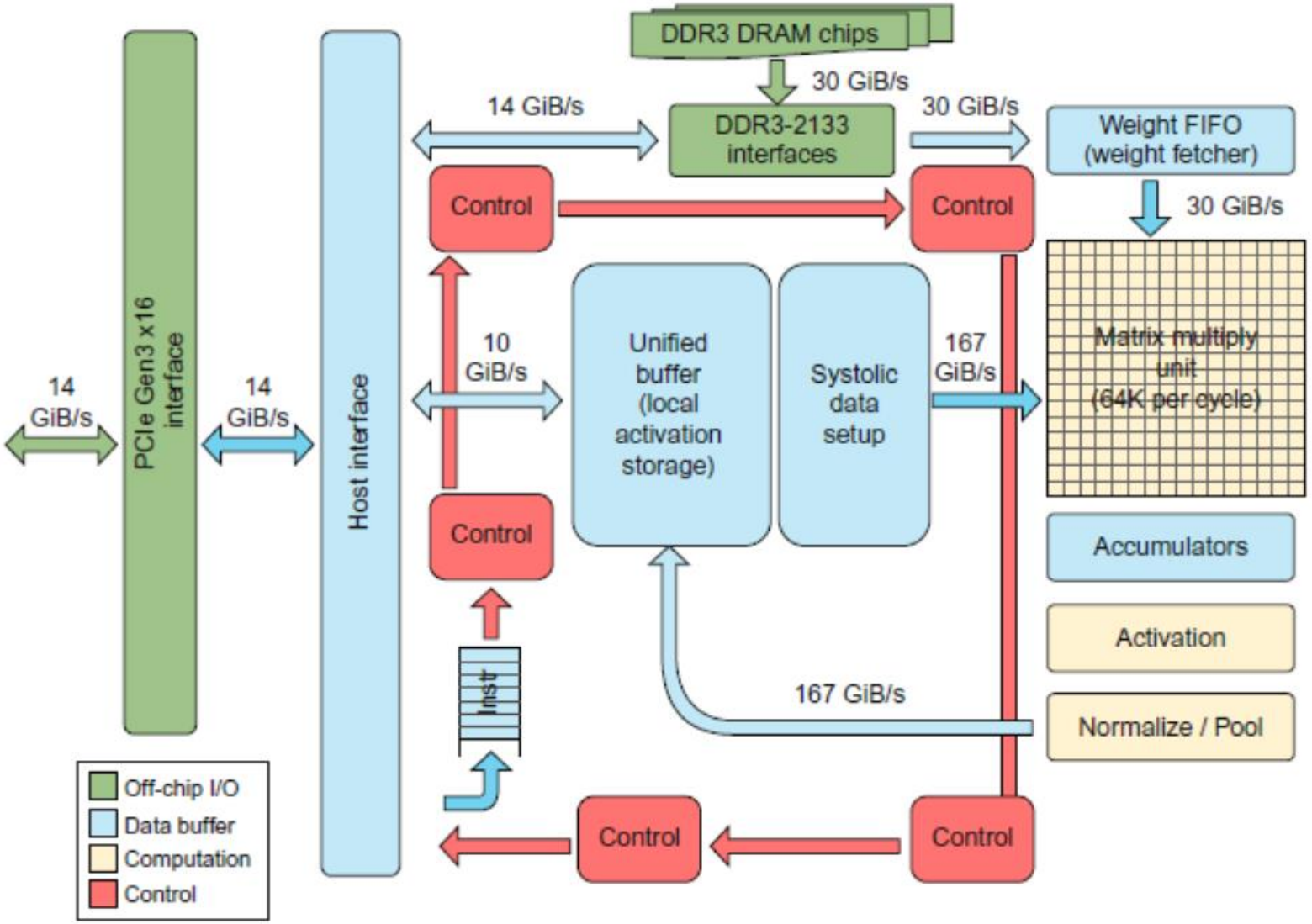
Copyright © 2019, Elsevier Inc. All rights Reserved

Tensor Processing Unit

- Google's DNN ASIC
- 256 x 256 8-bit matrix multiply unit
- Large software-managed scratchpad
- Coprocessor on the PCIe bus

MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Tensor Processing Unit



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

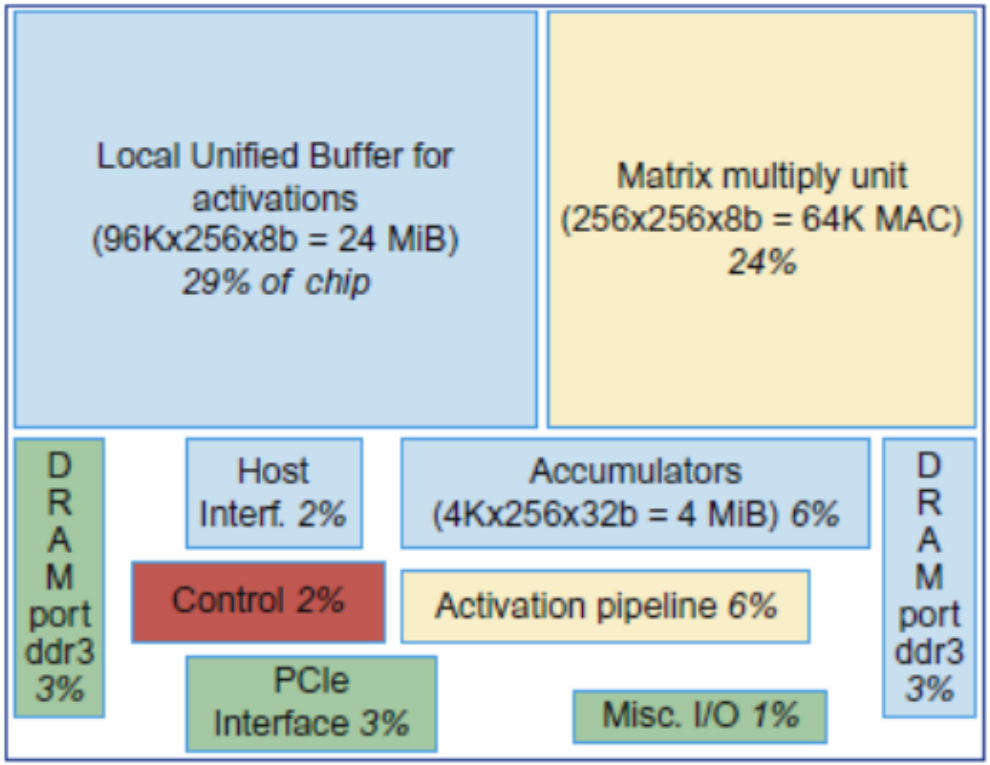
TPU ISA

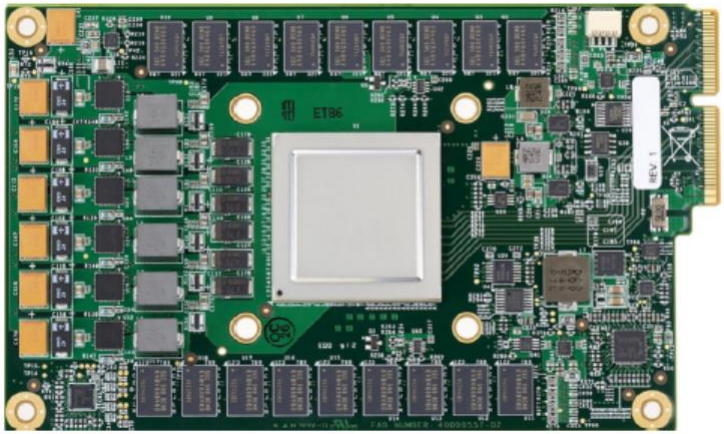
- Read_Host_Memory
  - Reads memory from the CPU memory into the unified buffer
- Read_Weights
  - Reads weights from the Weight Memory into the Weight FIFO as input to the Matrix Unit
- MatrixMatrixMultiply/Convolve
  - Perform a matrix-matrix multiply, a vector-matrix multiply, an element-wise matrix multiply, an element-wise vector multiply, or a convolution from the Unified Buffer into the accumulators
  - takes a variable-sized B*256 input, multiplies it by a 256x256 constant input, and produces a B*256 output, taking B pipelined cycles to complete
- Activate
  - Computes activation function
- Write_Host_Memory
  - Writes data from unified buffer into host memory

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

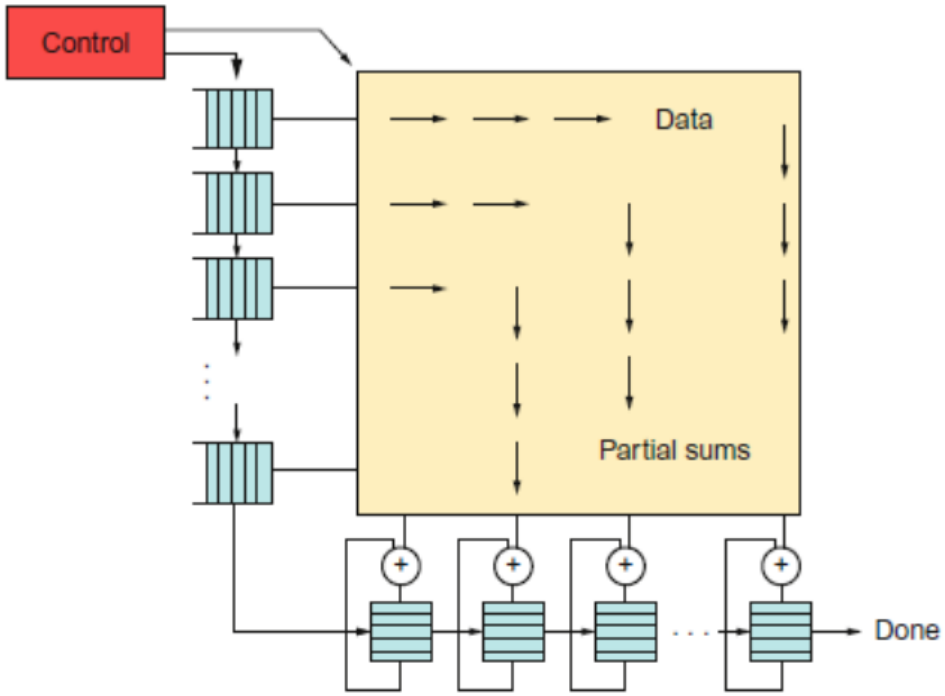
TPU ISA

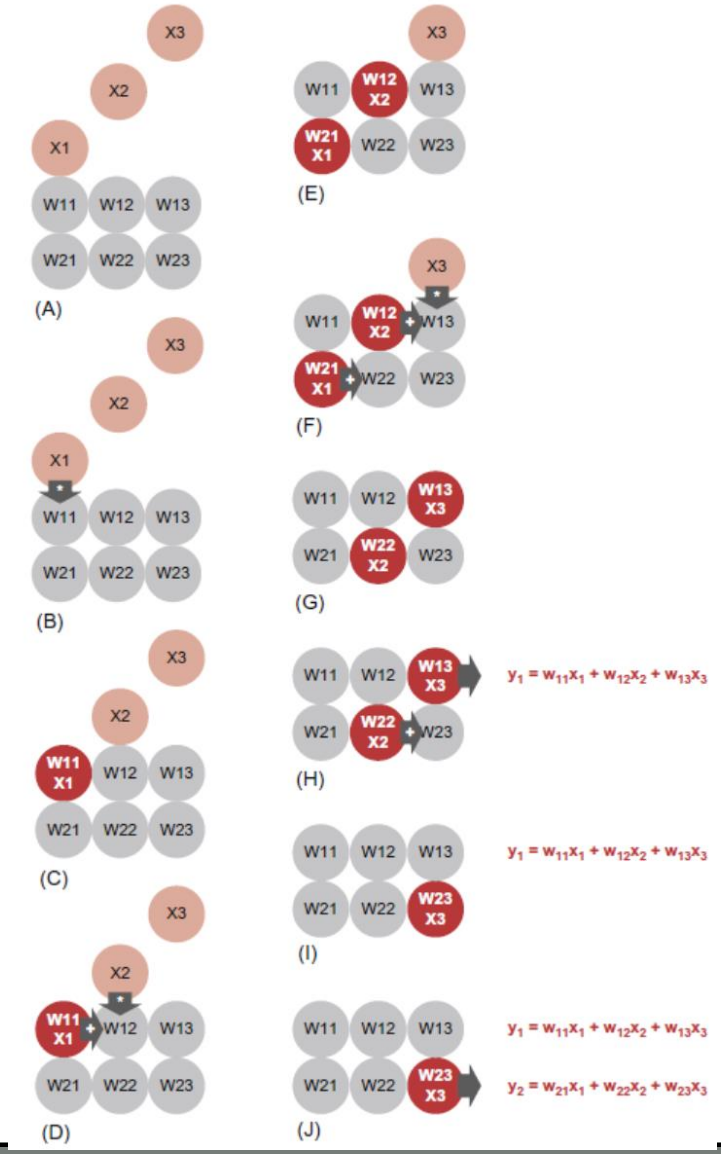




MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

TPU ISA





Copyright © 2019, Elsevier Inc. All rights Reserved

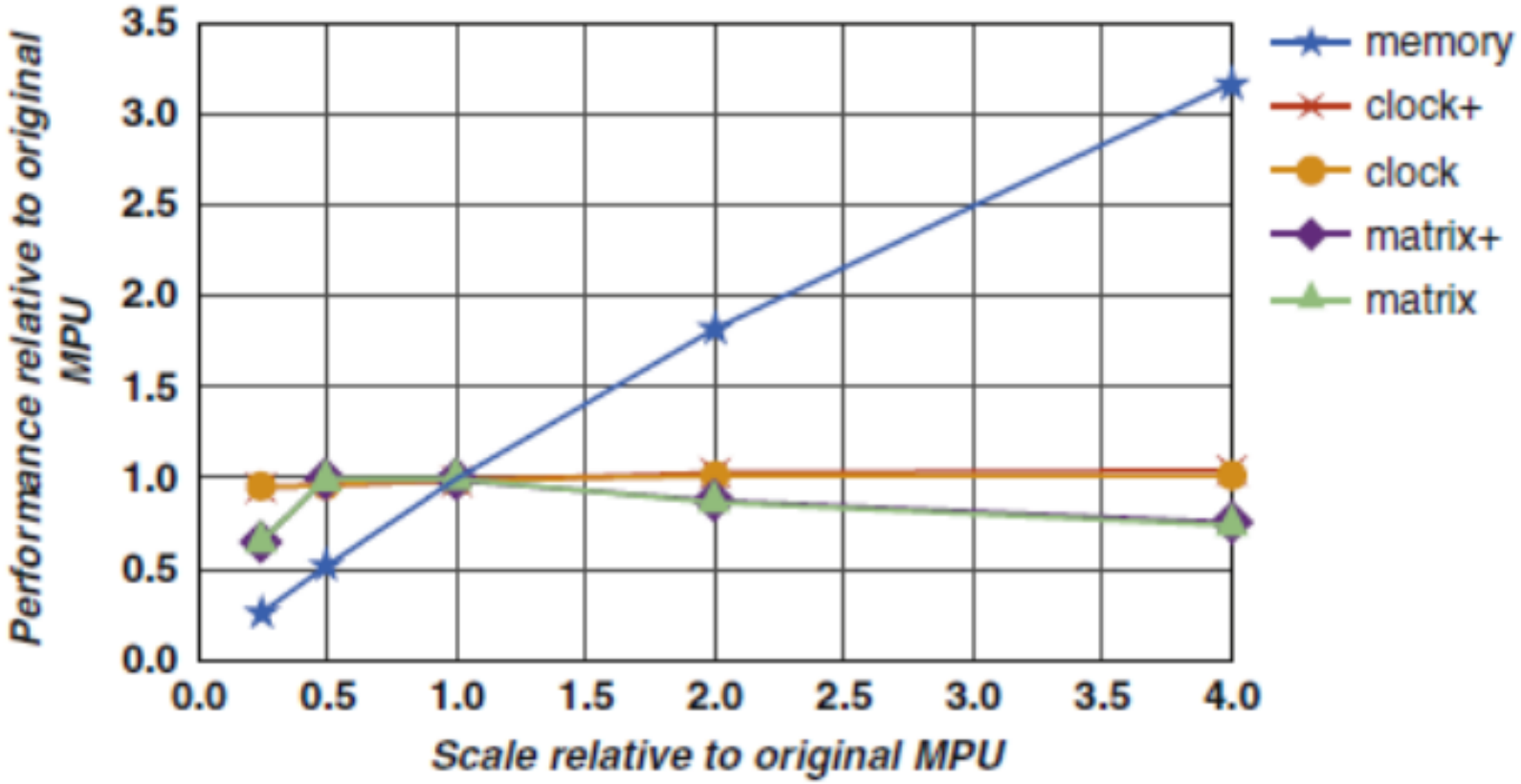
TPU ISA

- Read_Host_Memory
  - Reads memory from the CPU memory into the unified buffer
- Read_Weights
  - Reads weights from the Weight Memory into the Weight FIFO as input to the Matrix Unit
- MatrixMatrixMultiply/Convolve
  - Perform a matrix-matrix multiply, a vector-matrix multiply, an element-wise matrix multiply, an element-wise vector multiply, or a convolution from the Unified Buffer into the accumulators
  - takes a variable-sized B*256 input, multiplies it by a 256x256 constant input, and produces a B*256 output, taking B pipelined cycles to complete
- Activate
  - Computes activation function
- Write_Host_Memory
  - Writes data from unified buffer into host memory

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Improving the TPU



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

The TPU and the Guidelines

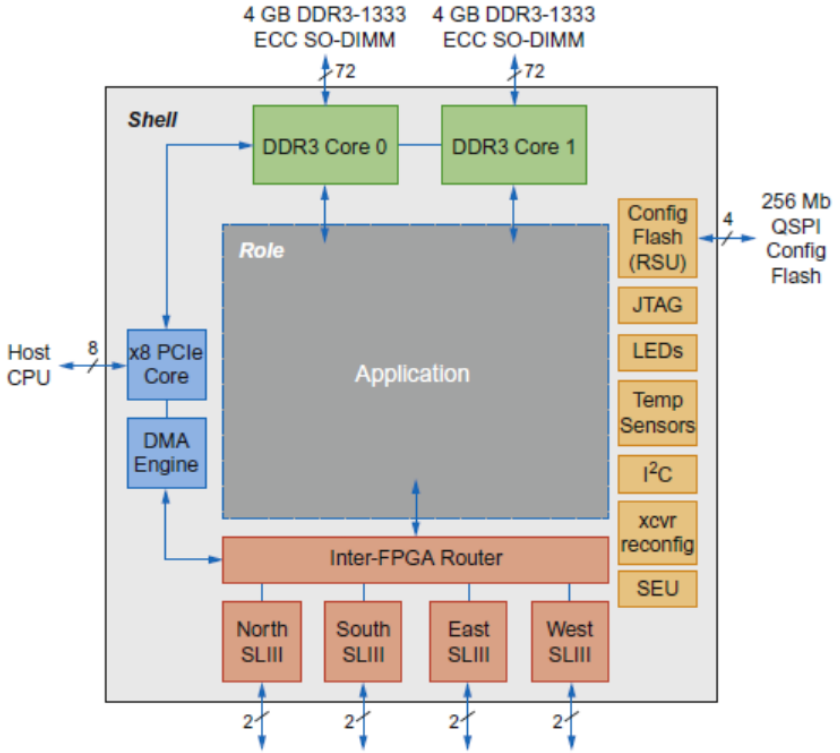
- Use dedicated memories
  - 24 MiB dedicated buffer, 4 MiB accumulator buffers
- Invest resources in arithmetic units and dedicated memories
  - 60% of the memory and 250X the arithmetic units of a server-class CPU
- Use the easiest form of parallelism that matches the domain
  - Exploits 2D SIMD parallelism
- Reduce the data size and type needed for the domain
  - Primarily uses 8-bit integers
- Use a domain-specific programming language
  - Uses TensorFlow

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

Microsoft Catapult

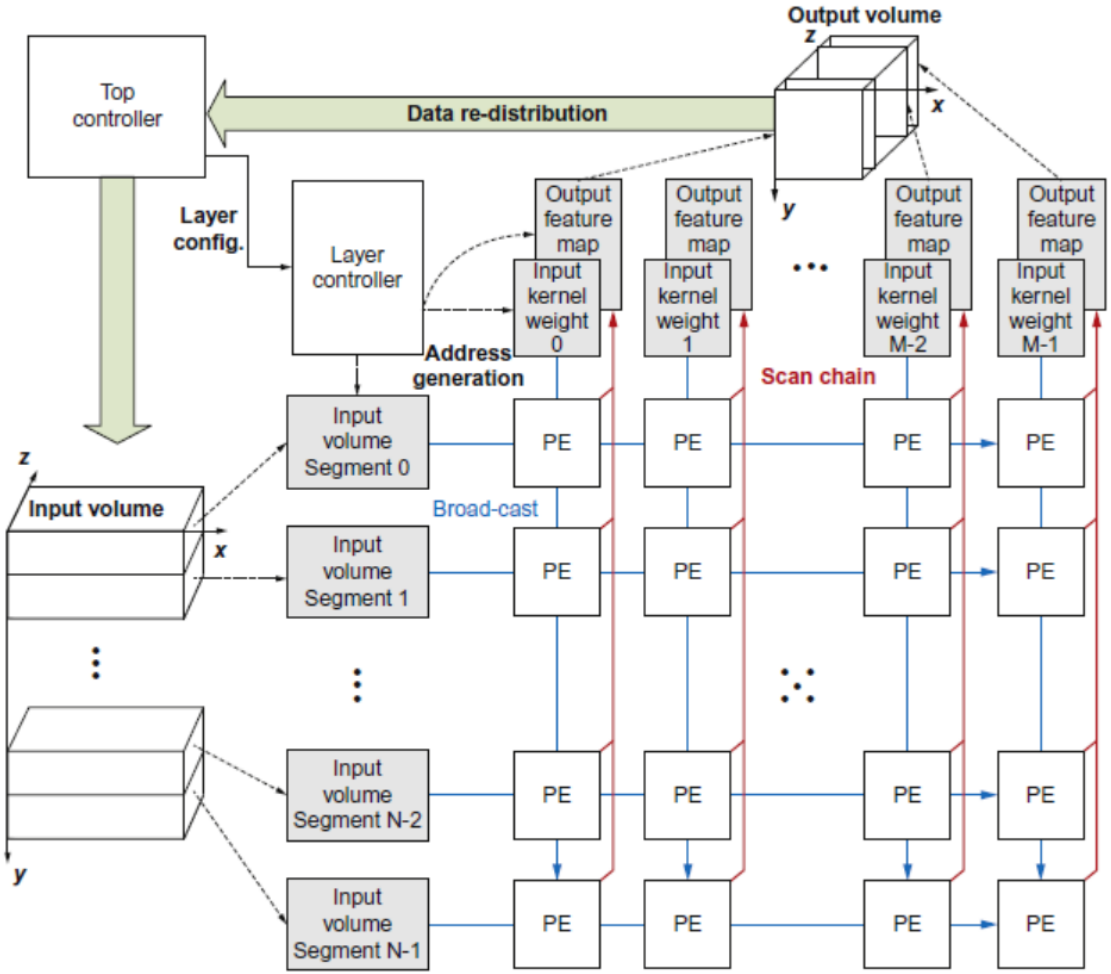
- Needed to be general purpose and power efficient
  - Uses FPGA PCIe board with dedicated 20 Gbps network in 6 x 8 torus
  - Each of the 48 servers in half the rack has a Catapult board
  - Limited to 25 watts
  - 32 MiB Flash memory
  - Two banks of DDR3-1600 (11 GB/s) and 8 GiB DRAM
  - FPGA (unconfigured) has 3962 18-bit ALUs and 5 MiB of on-chip memory
  - Programmed in Verilog RTL
  - Shell is 23% of the FPGA



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Microsoft Catapult: CNN

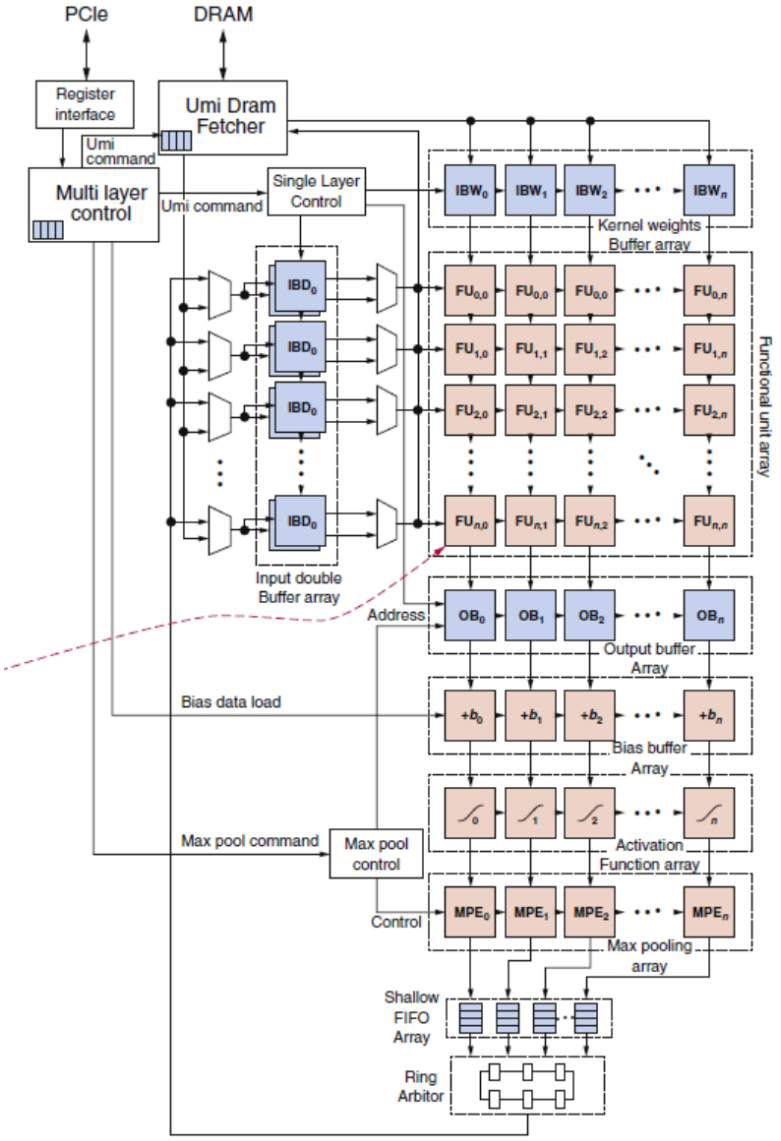
- CNN accelerator, mapped across multiple FPGAs



MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

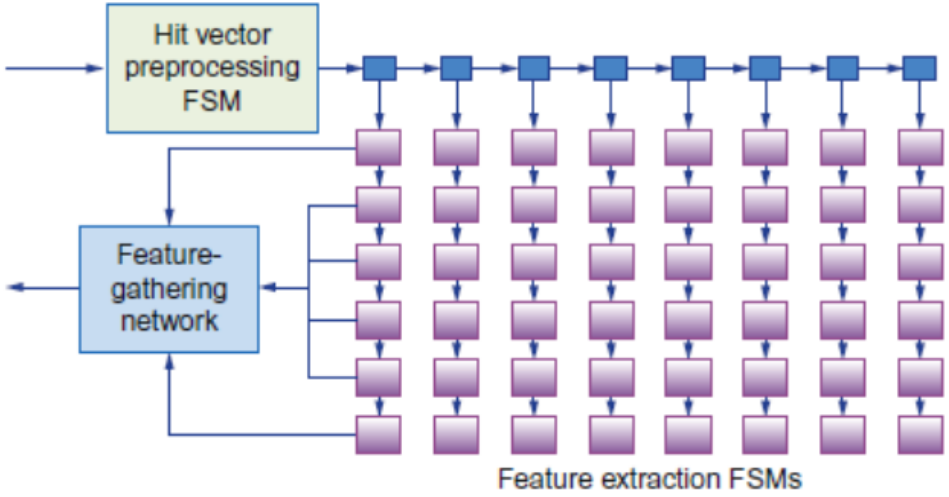
Microsoft Catapult: CNN



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Microsoft Catapult: Search Ranking

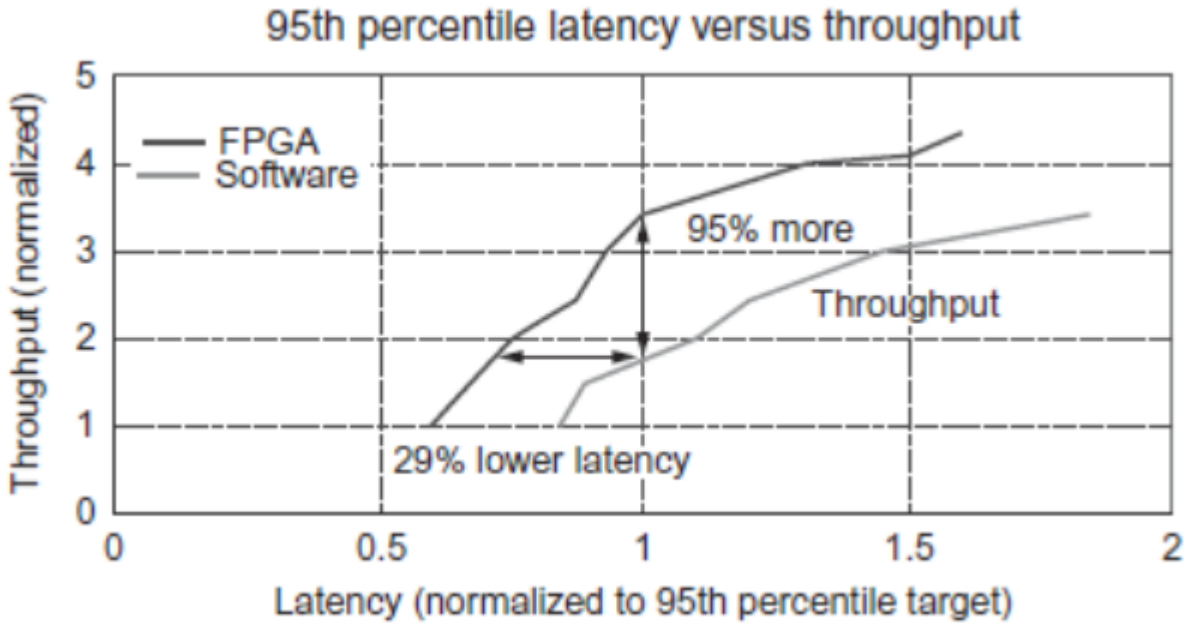
- Feature extraction (1 FPGA)
  - Extracts 4500 features for every document-query pair, e.g. frequency in which the query appears in the page
  - Systolic array of FSMs
- Free-form expressions (2 FPGAs)
  - Calculates feature combinations
- Machine-learned Scoring (1 FPGA for compression, 3 FPGAs calculate score)
  - Uses results of previous two stages to calculate floating-point score
- One FPGA allocated as a hot-spare



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Microsoft Catapult: Search Ranking

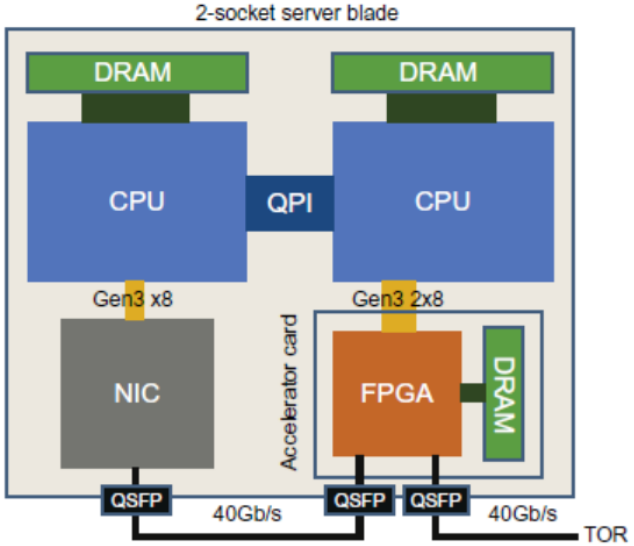
- Free-form expression evaluation
  - 60 core processor
  - Pipelined cores
  - Each core supports four threads that can hide each other's latency
  - Threads are statically prioritized according to thread latency



Copyright © 2019, Elsevier Inc. All rights Reserved

Microsoft Catapult: Search Ranking

- Version 2 of Catapult
  - Placed the FPGA between the CPU and NIC
  - Increased network from 10 Gb/s to 40 Gb/s
  - Also performs network acceleration
  - Shell now consumes 44% of the FPGA
  - Now FPGA performs only feature extraction



Copyright © 2019, Elsevier Inc. All rights Reserved

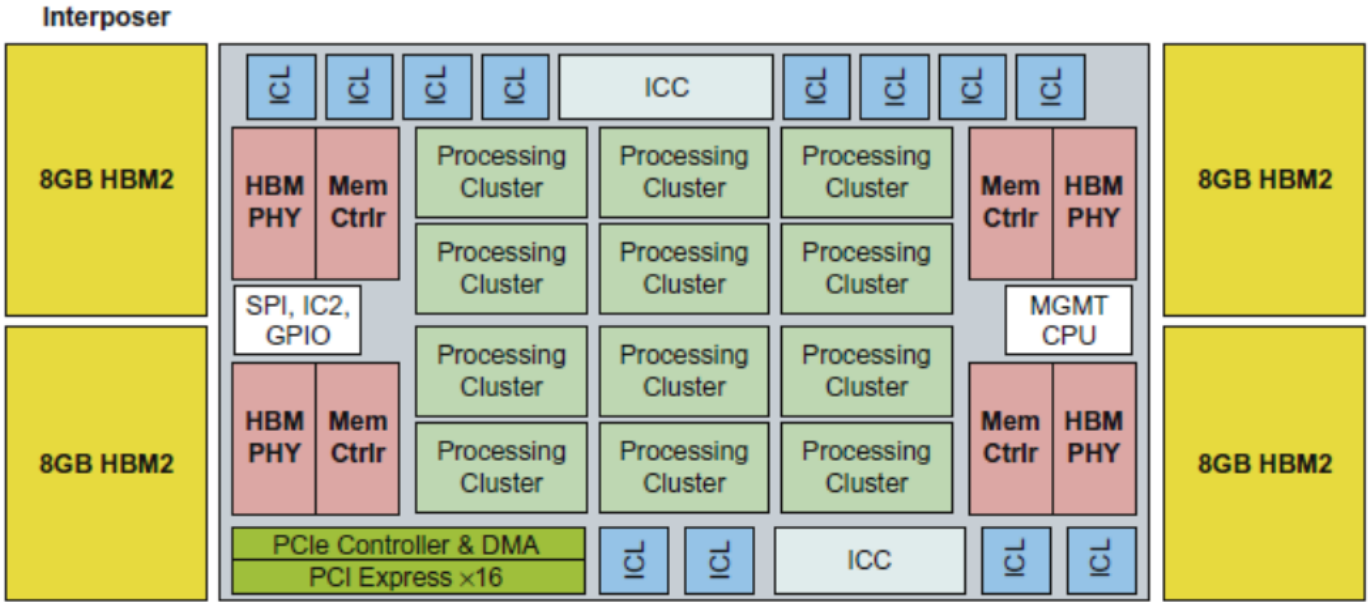
Catapult and the Guidelines

- Use dedicated memories
  - 5 MiB dedicated memory
- Invest resources in arithmetic units and dedicated memories
  - 3926 ALUs
- Use the easiest form of parallelism that matches the domain
  - 2D SIMD for CNN, MISD parallelism for search scoring
- Reduce the data size and type needed for the domain
  - Uses mixture of 8-bit integers and 64-bit floating-point
- Use a domain-specific programming language
  - Uses Verilog RTL; Microsoft did not follow this guideline

Copyright © 2019, Elsevier Inc. All rights Reserved

Intel Crest

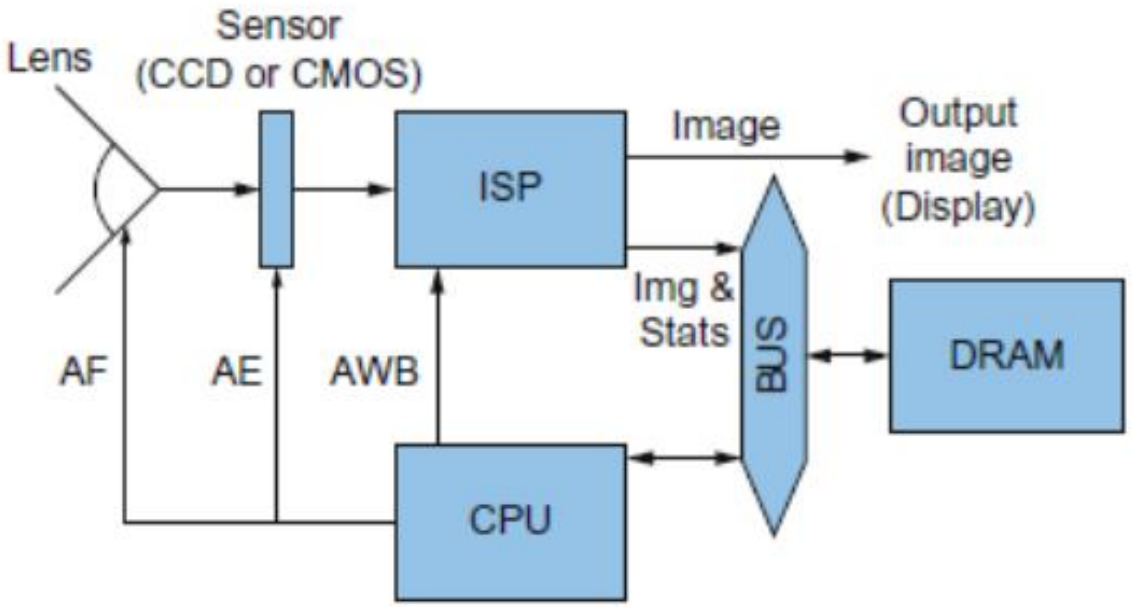
- DNN training
- 16-bit fixed point
- Operates on blocks of 32x32 matrices
- SRAM + HBM2



Copyright © 2019, Elsevier Inc. All rights Reserved

Pixel Visual Core

- Pixel Visual Core
  - Image Processing Unit
  - Performs stencil operations
  - Decended from Image Signal processor



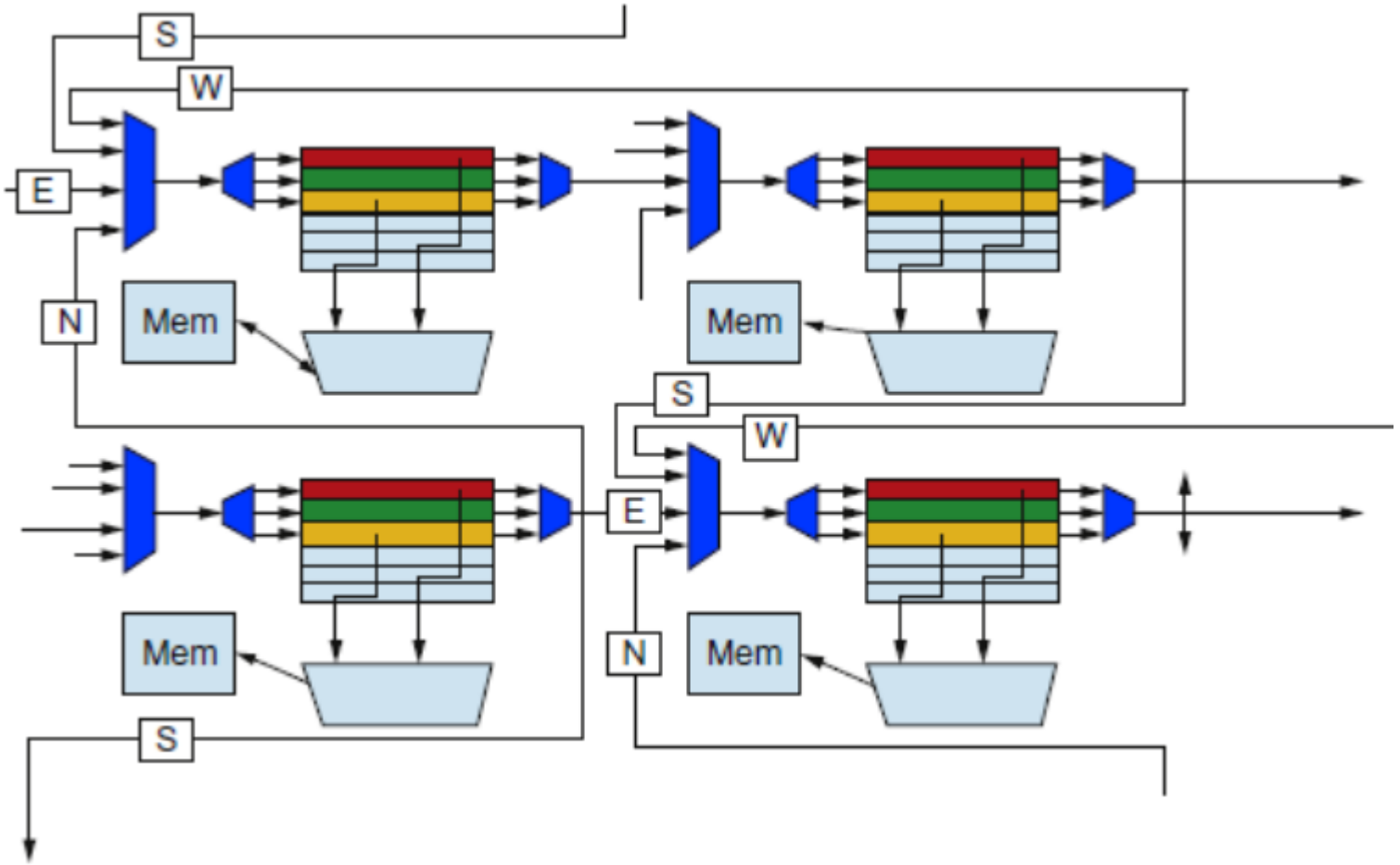
Copyright © 2019, Elsevier Inc. All rights Reserved

Pixel Visual Core

- Software written in Halide, a DSL
  - Compiled to virtual ISA
  - vISA is lowered to physical ISA using application-specific parameters
  - pISA is VLSI
- Optimized for energy
  - Power Budget is 6 to 8 W for bursts of 10-20 seconds, dropping to tens of milliwatts when not in use
  - 8-bit DRAM access equivalent energy as 12,500 8-bit integer operations or 7 to 100 8-bit SRAM accesses
  - IEEE 754 operations require 22X to 150X of the cost of 8-bit integer operations
- Optimized for 2D access
  - 2D SIMD unit
  - On-chip SRAM structured using a square geometry

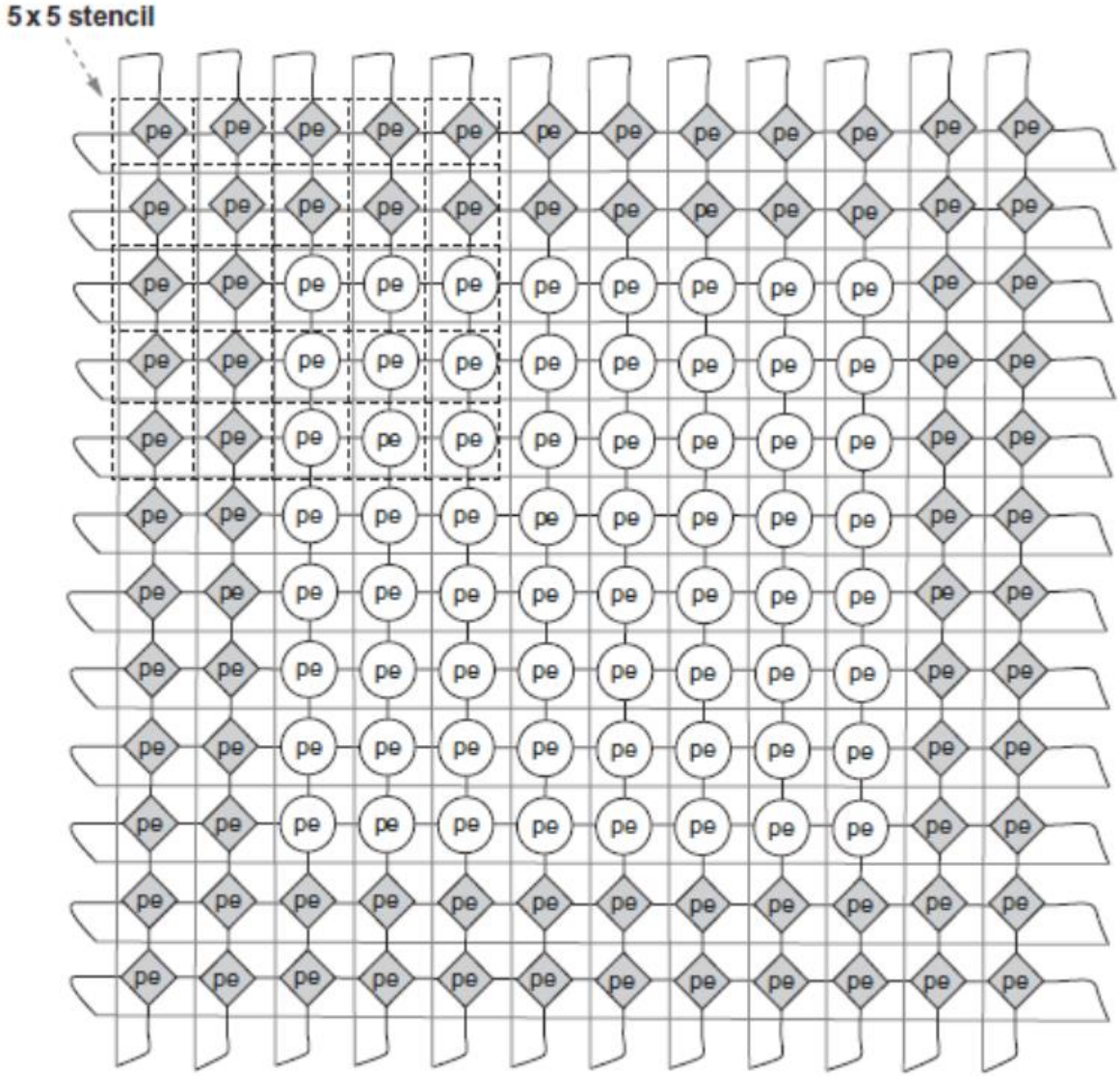
Copyright © 2019, Elsevier Inc. All rights Reserved

Pixel Visual Core



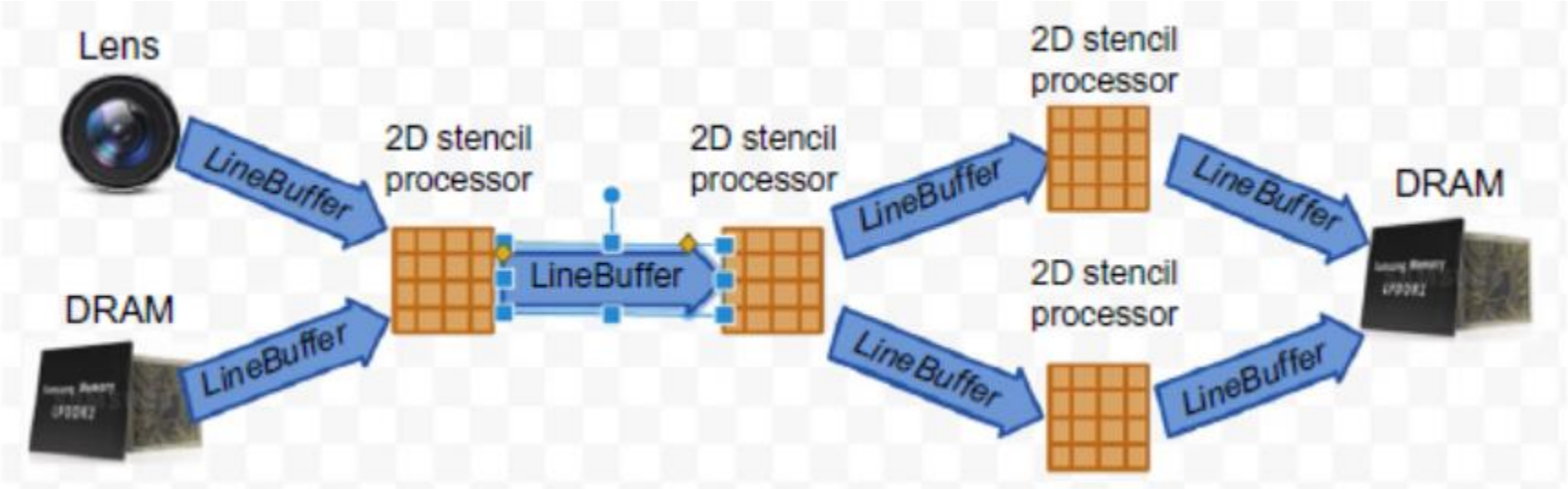
MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Pixel Visual Core



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Pixel Visual Core



Copyright © 2019, Elsevier Inc. All rights Reserved

Visual Core and the Guidelines

- Use dedicated memories
  - 128 + 64 MiB dedicated memory per core
- Invest resources in arithmetic units and dedicated memories
  - 16x16 2D array of processing elements per core and 2D shifting network per core
- Use the easiest form of parallelism that matches the domain
  - 2D SIMD and VLIW
- Reduce the data size and type needed for the domain
  - Uses mixture of 8-bit and 16-bit integers
- Use a domain-specific programming language
  - Halide for image processing and TensorFlow for CNNs

Copyright © 2019, Elsevier Inc. All rights Reserved

Fallacies and Pitfalls

- It costs $100 million to design a custom chip
- Performance counters added as an afterthought
- Architects are tackling the right DNN tasks
- For DNN hardware, inferences per second (IPS) is a fair summary performance metric
- Being ignorant of architecture history when designing an DSA

Copyright © 2019, Elsevier Inc. All rights Reserved